Building Large Language Models from Scratch

A grounded eBook generated from the playlist

Build statistics	
Videos processed	43
Chapters	12
Atomic claims extracted	3,873
Superseded claims	684
Sentences in book	4,841
Citation rate	62.1%
Verification pass rate	75.6%

Every sentence maps back to a transcript span, slide frame, or canonical reference. Citation rate measures claim-bearing sentences with a verified citation; verification pass rate counts all sentences that survived audit.

Source playlist

https://www.youtube.com/watch?v=Xpr8D6LeAtw&list=PLPTV0NXA_ZSgsLAr8YCgCwhPIJNNtexWu

Generated by llm-playlist-book pipeline

May 2026

Contents

Chapter 1

Large Language Models: Foundations and Landscape

If you have spent any time with ChatGPT, Gemini, or similar tools, you have already interacted with a Large Language Model. But knowing how to *use* one and knowing how to *build* one are very different things. This chapter lays the conceptual groundwork for everything that follows: what LLMs actually are, where they sit in the broader landscape of artificial intelligence, how they are trained, and why they represent such a significant departure from the NLP tools that came before them. By the end, you will have a clear mental map of the territory—one that will make every technical detail in subsequent chapters feel purposeful rather than arbitrary.

1.1 What Is a Large Language Model?

Start with the simplest possible definition. A Large Language Model—LLM for short—is a neural network designed to understand, generate, and respond to human-like text. That single sentence contains three important ideas: *neural network*, *massive scale*, and *text*.

At its core, a neural network consists of input data feeding into layers of neurons stacked together, with an output layer. Think of each layer as a transformation: raw numbers go in, slightly more useful numbers come out, and after enough layers the network has learned to map inputs to outputs in surprisingly powerful ways. LLMs are deep neural networks—meaning they have many such layers—trained on massive amounts of data and specifically designed to understand, generate, and respond to human-like text.

The word "large" in the name is not decorative. Model size refers to the number of parameters in a model, and modern LLMs have parameter counts in the billions or even trillions. Those parameters are the learned weights that encode everything the model knows about language, facts, reasoning patterns, and more.

[Diagram: Neural network architecture showing input text flowing through stacked layers with labeled parameters on connections, producing output text.]

1.2 Where Do LLMs Fit? The AI Terminology Hierarchy

The field of AI is littered with overlapping buzzwords—AI, ML, deep learning, generative AI—and it is easy to lose track of how they relate. Here is the precise hierarchy.

Artificial Intelligence is the broadest umbrella term, encompassing any machine that behaves remotely like a human or exhibits some form of intelligence. Inside that umbrella sits Machine Learning, a subset of AI where machines learn and adapt based on how users interact with them. Inside ML sits Deep Learning, which specifically involves neural networks—whereas ML also encompasses other approaches like decision trees.

LLMs are a subset of deep learning that deal exclusively with text and do not involve images. That last qualifier matters: the moment you add images, audio, or video to the mix, you have crossed into a different, though related, territory.

[Diagram: Hierarchical relationship of AI subfields: AI encompasses ML, which encompasses DL, which contains LLMs at the center. Generative AI overlaps the DL and LLM regions.]

That related territory is Generative AI—a mixture of large language models and deep learning that deals with multiple modalities including text, images, sound, and video. Generative AI uses deep neural networks to create new content such as text, images, and various forms of media. So while every LLM is a piece of deep learning, not every piece of generative AI is an LLM.

1.3 How LLMs Differ from Earlier NLP Models

To appreciate what makes LLMs remarkable, it helps to understand what came before them. Earlier NLP models were designed for very specific tasks—a translation model translated, a sentiment analysis model classified sentiment, and that was essentially the end of the story. Each model was a specialist, and specialists are brittle: a translation model cannot answer a question, and a sentiment classifier cannot summarize a document.

LLMs break this mold entirely. A single pretrained LLM can translate, summarize, answer questions, write code, and perform dozens of other tasks—often without any task-specific retraining. This task-agnostic quality is one of the defining characteristics of the LLM era, and understanding *why* it arises requires understanding how LLMs are trained.

1.4 The Two-Stage Development Pipeline

Creating an LLM involves two stages: pre-training and fine-tuning. This pipeline is fundamental enough that the entire book is organized around it, so it is worth spending time here to understand what each stage means and why both are necessary.

Figure 1.1: Two-stage pipeline for building Large Language Models: pre-training produces a foundation model, which is then fine-tuned to create a task-specific model.

Stage 1: Pre-Training

Pre-training means training an LLM on a huge amount of data so that it can perform a wide range of tasks. The key word is *huge*. We are not talking about thousands or even millions of examples—we are talking about hundreds of billions of tokens of text scraped from the internet and other sources.

For the purposes of building intuition, one token is approximately equal to one word. So "hundreds of billions of tokens" translates roughly to hundreds of billions of words—an amount of text no human could read in many lifetimes.

Where does all this text come from? Two prominent sources illustrate the scale:

- **Common Crawl** is a huge and open repository of all the data on the internet.
- **WebText2** is a corpus consisting of Reddit submissions, blog posts, Stack Overflow articles, and code.

The training objective during pre-training is deceptively simple: next-word prediction. Given a sequence of words, the model predicts the next word—also called word completion. LLM pretraining involves this next-word prediction task applied to large text datasets.

What makes this objective so powerful is that it requires no human-labeled data. The text itself provides the supervision: every word in a document is simultaneously a training input (given the words before it) and a training label (the correct next word). This approach is called generative pre-training—an unsupervised learning method where a language model is trained on unlabeled text to predict the next word, using the text itself as training data without requiring external labels. The raw text used for training is regular text without any labeling information, which means training data can be collected at internet scale without expensive human annotation.

In the GPT family of models, this self-supervised structure is made explicit: a sentence is divided into training and testing portions, where the next word serves as the known true label. GPT models are autoregressive, meaning each prediction is conditioned on all previous words in the sequence.

The result of pre-training is a **foundation model** (also called a base model or pre-trained model)—a model trained on an underlying dataset of unlabeled data that has absorbed an enormous amount of world knowledge and linguistic structure, but has not yet been specialized for any particular application.

Stage 2: Fine-Tuning

Once a foundation model exists, it can be adapted for specific tasks through fine-tuning. A pre-trained LLM can be finetuned using smaller labeled datasets for specific tasks like classification, summarization, or question-answering. Fine-tuning uses human-annotated examples to adjust the model's weights so that it performs well on a given task.

The contrast with pre-training is stark: pre-training uses massive unlabeled datasets; fine-tuning uses small labeled ones. Pre-training builds general capability; fine-tuning sharpens it for a purpose.

It is worth noting what fine-tuning for classification does to a model's outputs. A classification-finetuned model is restricted to predicting only the classes it has encountered during training; for example, a model trained to classify emails as "spam" or "not spam" cannot output any other response. This is a deliberate constraint—you want a spam classifier to say "spam" or "not spam," not to compose a poem.

All modern large language models are trained in these two main steps: pre-training on unlabeled data to create a foundational model, followed by fine-tuning on labeled task-specific data.

1.5 The Three-Stage Development Workflow

When building an LLM from scratch, the two-stage pipeline expands into a three-stage development workflow:

1. **Stage 1 — Foundation and Architecture**: This covers data preparation, sampling, and the architectural components of the model. Before training anything, you need to understand how to represent text as numbers, how to feed data into the network, and what the network's internal structure looks like.
2. **Stage 2 — Pre-Training**: With the architecture in place, the model is trained on large unlabeled datasets. At the end of this stage, pre-trained weights—such as those released by OpenAI—can be loaded into the model, and functions to save and load those weights are implemented to avoid retraining from scratch every time.
3. **Stage 3 — Fine-Tuning**: The pre-trained model is adapted for specific applications.

This book follows that three-stage structure. The goal throughout is to understand the nuts and bolts of how large language models are built, rather than simply running pre-built applications.

1.6 The Transformer Architecture: A Brief Introduction

The architectural backbone of virtually every modern LLM is the **transformer**—a deep neural network architecture introduced in 2017. Understanding the transformer is essential to understanding LLMs, and later chapters will build it piece by piece. For now, it is enough to know that the transformer's power comes from a mechanism called **attention**.

The attention mechanism is the core component of the transformer architecture that makes it powerful. Intuitively, attention allows the model to decide which parts of the input are most relevant when producing each part of the output. More precisely, it gives LLMs selective access to the entire input sequence when generating output one word at a time. This is a crucial capability: in a long sentence, the word that best predicts the next word might be ten or twenty words back. The self-attention mechanism captures exactly these long-range dependencies, allowing better prediction of the next word—something earlier architectures struggled with.

Several components work together to make attention function correctly:

- **Attention scores** are a key component of the attention mechanism.
- **Positional encoding** provides information about the order in which words appear in a sentence, since the transformer architecture does not inherently process words in sequence.
- **Vector embeddings** represent words as dense numerical vectors.
- **Tokens** are the basic units of a sentence that the model operates on.

The transformer also uses specialized variants of attention for different purposes. **Multi-head attention** applies the attention mechanism across multiple representation subspaces simultaneously,

allowing the model to attend to different aspects of the input at once. **Masked multi-head attention** is a variant used in transformer decoders to prevent the model from attending to future positions—a necessary constraint during next-word prediction training, since the model should not be able to "cheat" by looking ahead.

The GPT approach specifically combines the transformer architecture with unsupervised pre-training, where labels are derived from the sentences themselves without any pre-labeling of data.

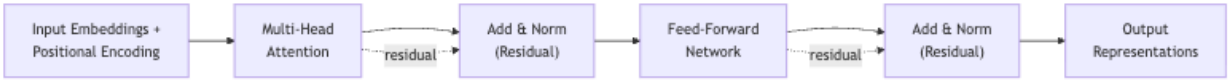


Figure 1.2: Transformer block architecture showing input embeddings with positional encoding flowing through multi-head attention and feed-forward layers with residual connections.

1.7 Zero-Shot and Few-Shot Learning

One of the most striking capabilities of large pre-trained language models is their ability to perform tasks they were never explicitly trained on. This phenomenon has two related names depending on how much task-specific information the model receives at inference time.

Zero-shot learning is the ability to generalize to completely unseen tasks without any prior specific examples. The model predicts the answer given only a description of the task, with no supporting examples whatsoever. For instance, in zero-shot translation, the model is given only the task description "translate English to French" and the input word "breakfast"—no translation examples included.

Few-shot learning is learning from a minimum number of examples provided by the user as input. Rather than updating the model's weights, these examples are supplied directly in the input prompt.

Between these two extremes sits **one-shot learning**, where the model sees a single example of the task in addition to the task description.

These capabilities are closely related to what researchers call **emergent behavior**—the ability of a model to perform tasks it was not explicitly trained to perform. A model trained purely on next-word prediction somehow learns to translate, reason, and answer questions. This emergence is not fully understood theoretically, but it is empirically well-established.

The concept of few-shot learning as a property of large language models was formally introduced with GPT-3, released in 2020, which demonstrated that language models are few-shot learners. Zero-shot versus few-shot learning remains a key concept in understanding what modern LLMs can do.

1.8 The Historical Arc: From Transformers to GPT-4

To understand where we are, it helps to trace the path that got us here.

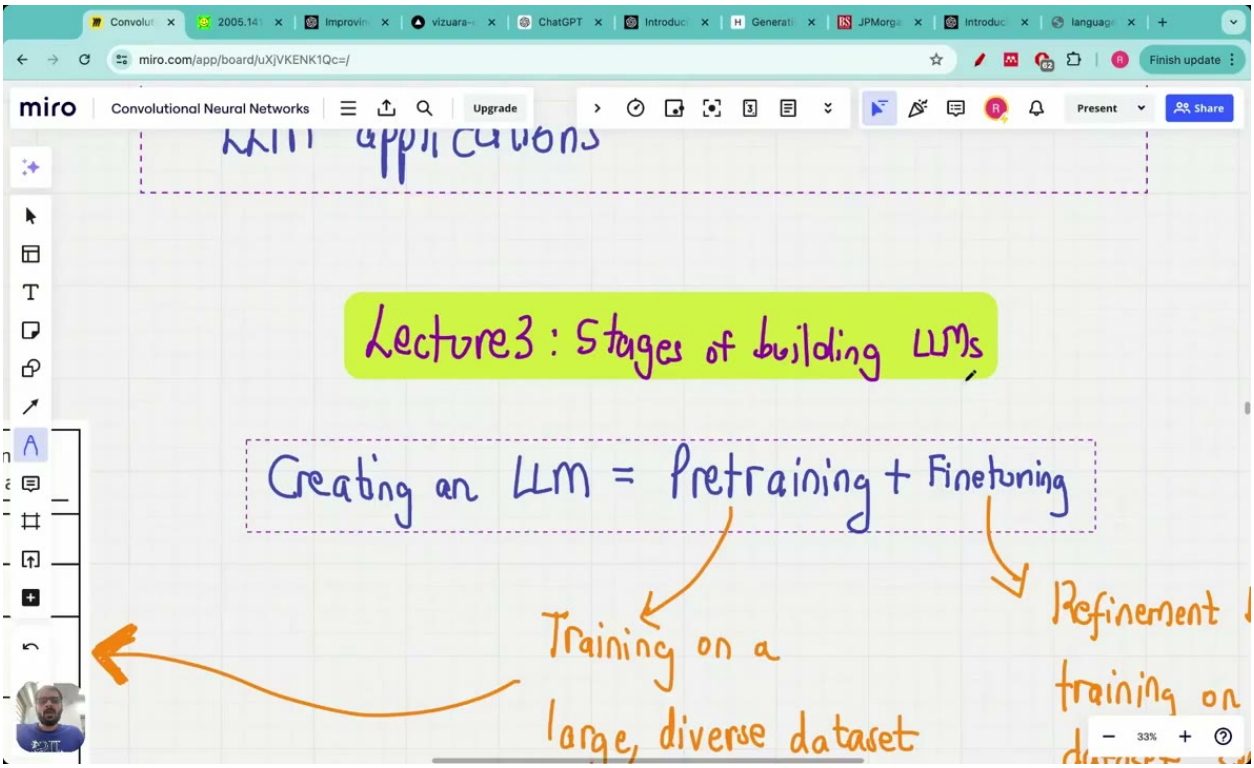


Figure 1.3: Three prompting paradigms: zero-shot (task description only), one-shot (task description plus one example), and few-shot (task description plus multiple examples) for guiding LLM behavior.

The transformer architecture, introduced in 2017, provided the foundation. The GPT family of models then demonstrated that combining transformers with large-scale unsupervised pre-training could produce models with remarkable generalization ability. GPT-3, released in 2020, was a landmark: it showed that scaling up model size and training data produced qualitatively new capabilities, including few-shot learning.

GPT-4 represents the current state of the art in closed-source models—one where the parameters and weights are not publicly known. This opacity is characteristic of the closed-source approach: such models do not release their weights or architecture, and few details about the model itself are made public.

1.9 Open-Source vs. Closed-Source Models

The distinction between open-source and closed-source models is practically important for anyone building with or on top of LLMs.

An open-source model makes its entire architecture available to the public for anyone to see, allowing researchers and engineers to inspect, modify, and build upon it. In 2022, most large language models were closed-source. That landscape has shifted considerably: by 2024, the performance gap between open-source and closed-source models is slowly decreasing, and as of August 2024, open-source language models are comparable to closed-source models on the MMLU benchmark.

Meta's release of Llama 3 405B is a concrete example of this trend—a model that closes the gap between closed-source and open-weight models. All the information needed to build capable language models is now available as open-source. Modern large language models such as GPT are very powerful and sophisticated, but the techniques used to build them are no longer secret, and the open-source ecosystem has made it possible to study, replicate, and build upon state-of-the-art architectures.

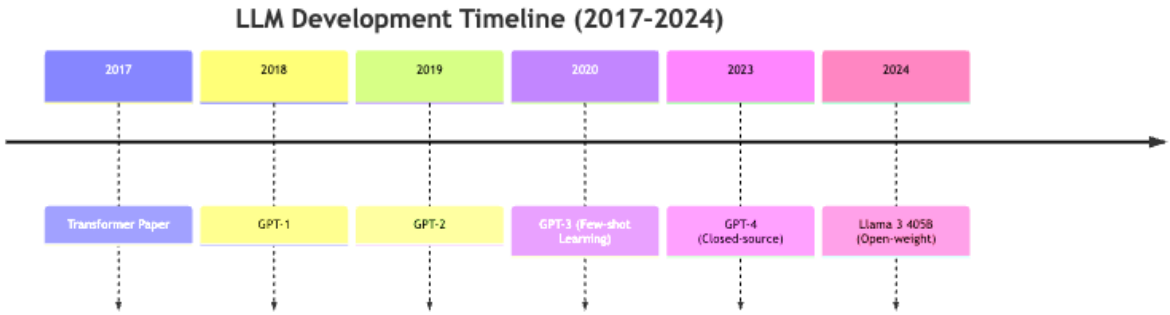


Figure 1.4: Key milestones in large language model development from the Transformer paper (2017) through GPT-4 and open-weight models like Llama 3 (2024).

1.10 Why Build from Scratch?

Given that powerful pre-trained models are freely available, a reasonable question is: why bother building one from scratch? Most existing online courses on building large language models focus on application development rather than teaching how to build an LLM from the ground up. Running a pre-built model is straightforward; understanding what is happening inside it is not.

A common mistake learners make is jumping directly to applications and running code without understanding how LLMs actually work. This shortcut creates a fragile understanding—you can use the tool, but you cannot diagnose problems, adapt architectures, or reason about limitations.

The goal of this book is different: to teach how to build an entire large language model from scratch, ensuring deep understanding of LLM fundamentals rather than focusing on deploying pre-built applications. Every design decision, every hyperparameter, every architectural choice will be motivated from first principles. By the time you finish, you will not just know *that* transformers use multi-head attention—you will know *why*, and you will have implemented it yourself.

1.11 Chapter Summary

This chapter has established the conceptual landscape for everything that follows:

- An LLM is a deep neural network designed to understand, generate, and respond to human-like text, with model size measured in parameters.
- LLMs sit within a hierarchy: AI ⊃ ML ⊃ Deep Learning ⊃ LLMs, with Generative AI overlapping DL and LLMs across multiple modalities.
- Earlier NLP models were task-specific specialists; LLMs are task-agnostic generalists.
- LLM development follows a two-stage pipeline: pre-training on massive unlabeled text using next-word prediction, followed by fine-tuning on smaller labeled datasets for specific tasks.
- The transformer architecture, introduced in 2017, is the backbone of modern LLMs, with the attention mechanism as its core innovation.
- Pre-trained LLMs exhibit zero-shot and few-shot learning—the ability to perform tasks without explicit training examples—a form of emergent behavior.
- GPT-3 (2020) formally established few-shot learning as a property of large language models; GPT-4 represents the closed-source frontier.
- The open-source model landscape has matured rapidly, with performance now comparable to closed-source models on standard benchmarks.

In the next chapter, we begin Stage 1 of the development workflow: preparing text data and understanding how raw text is transformed into the numerical representations that a neural network can process.

Chapter 2

Pre-Training vs. Fine-Tuning: Concepts and Motivation

Before writing a single line of code or tuning a single parameter, it pays to understand the two-stage lifecycle that turns raw text into a useful language model. Understanding what each stage does—and why both exist—is foundational to everything that follows in this book.

2.1 The Two-Stage Lifecycle

At the highest level, creating an LLM consists of two stages: pre-training and fine-tuning.

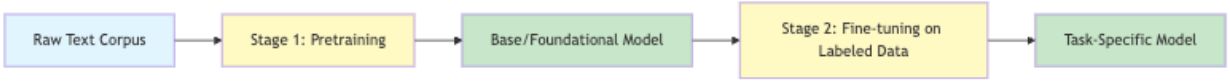


Figure 2.1: Two-stage LLM development pipeline: pretraining on large diverse text corpora produces a foundational model, which is then fine-tuned on labeled task-specific data to create a task-specific model.

A few definitions will keep the rest of the chapter precise. A *token* is approximately equal to one word. LLM stands for Large Language Model. And when we say *transformer*, we mean the neural network architecture that underlies models like GPT and BERT—though it is worth noting upfront that not all transformers are LLMs, and not all LLMs are transformers. We will return to that distinction later in the chapter.

2.2 Pre-Training: Building the Foundation

What Pre-Training Is

Pre-training means training an LLM on a huge amount of data so that it can perform a wide range of tasks. More precisely, it is the first training stage of an LLM, and the model it produces is called a *base model* or *foundational model*. Put simply, pre-training is training on a large, diverse dataset.

Pre-training requires a huge amount of data—billions or even trillions of words—and significant computational power, making it inaccessible to most individuals. In practice, LLMs are typically trained on billions of documents.

The Training Objective: Next-Word Prediction

The core task that drives pre-training is deceptively simple. *Word completion* is the task where an LLM is given a set of words and must predict the next word. The model sees raw text—regular text without any labeling information—and learns by trying to predict what comes next, over and over, across billions of examples.

Emergent Capabilities

What makes next-word prediction so powerful is what it produces as a side effect. LLMs trained only for next-word prediction can perform translation, multiple-choice questions, text summarization, sentiment analysis, linguistic acceptability, and question answering—without specific training on any of these tasks. In traditional NLP, separate models had to be trained for each of those tasks; a pre-trained LLM handles all of them with a single model.

Why does next-word prediction produce such broad capability? The answer lies in the data: LLMs interact effectively with users because they are trained on a huge and diverse set of data. To predict the next word accurately across billions of documents spanning every topic and genre, the model must internalize an enormous amount of world knowledge, linguistic structure, and reasoning patterns.

What Pre-Training Data Looks Like

Two commonly cited data sources illustrate the kind of material used in pre-training:

- **Common Crawl** is a huge and open repository of all the data on the internet.
- **WebText2** is a corpus consisting of Reddit submissions, blog posts, Stack Overflow articles, and code.

Together, these sources expose the model to an enormous variety of writing styles, topics, languages, and domains.

The Output: A Foundational Model

GPT-4, for example, is a pre-trained model capable of text completion and other tasks such as sentiment analysis and question answering. GPT stands for Generative Pre-trained Transformers and is explicitly a pre-trained, or foundational, model. GPT generates one word at a time and processes text from left to right, predicting only the rightmost unknown information.

For contrast, consider BERT—Bidirectional Encoder Representations from Transformers. Rather than predicting the next word, BERT randomly masks some words during training and attempts to predict those masked words. BERT is bidirectional, attending to different parts of the sentence from both left and right directions, and it uses only an encoder architecture without a decoder.

Because BERT can capture nuances and relationships between words by looking at the entire sentence from both directions, it can differentiate between different meanings of the same word—for example, “bank” as a financial institution versus “bank” as a river bank—by examining surrounding

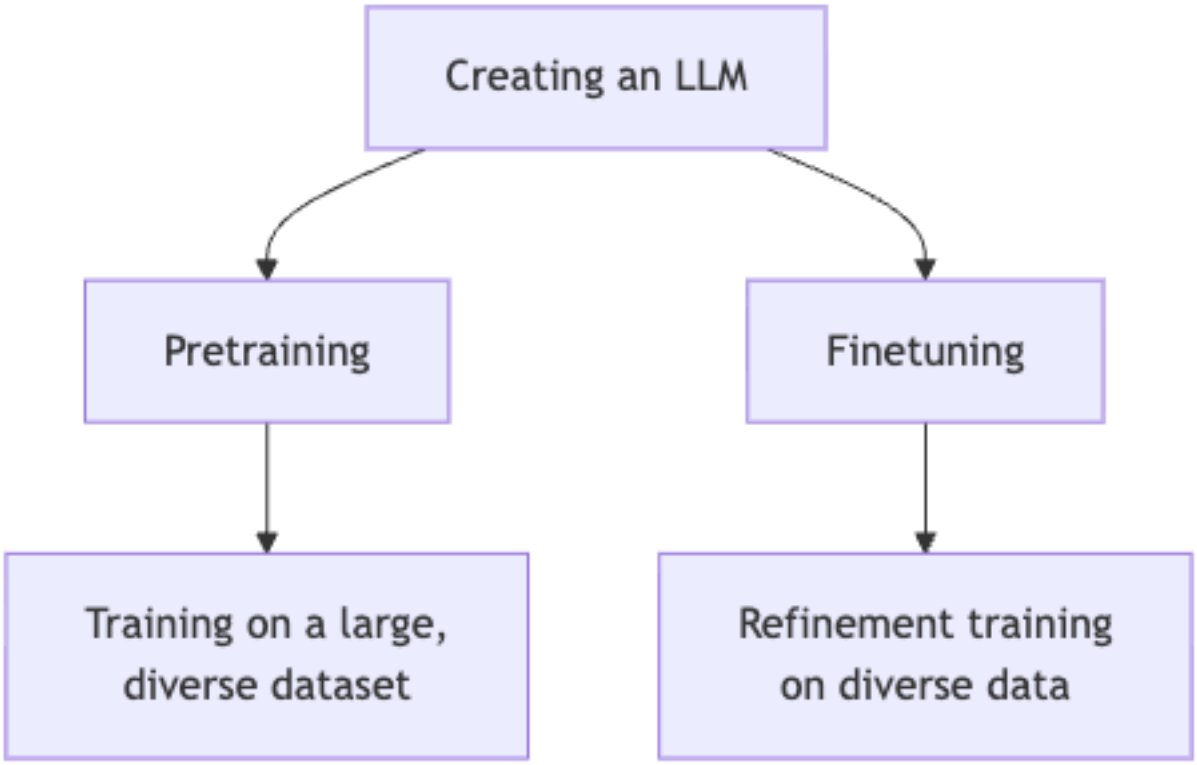


Figure 2.2: Two-stage process for creating an LLM: pretraining on large diverse datasets followed by finetuning on task-specific data.

words. This makes BERT particularly well suited to classification tasks; it is commonly used for sentiment analysis.

Both BERT and GPT have the word "transformers" in their names because they originated from the transformer architecture. The original transformer architecture was developed for machine translation tasks, specifically translating English text into German and French. Its simplified architecture consists of eight steps: tokenization, encoding, decoding with partial output, and word-by-word generation. BERT uses an encoder architecture, while GPT uses a decoder-style design.

2.3 The Limits of Pre-Training Alone

A foundational model is powerful, but it is not sufficient for every use case. Pre-trained models like GPT-4 may not have access to company-specific data and therefore produce generic rather than domain-specific responses.

This gap matters in practice. Fine-tuning is needed when building applications specific to a particular task or domain, rather than for general-purpose use. General users and students can use pre-trained models like GPT-4 directly without fine-tuning, but companies and startups deploying LLM applications in production need fine-tuning.

2.4 Fine-Tuning: Specializing the Foundation

What Fine-Tuning Is

Where pre-training operates on raw, unlabeled text at massive scale, fine-tuning typically requires a labeled dataset. The three main steps for building an LLM are: (1) training on a large corpus of raw text data, (2) pre-training to create a foundational model, and (3) fine-tuning on labeled data for specific tasks. Stage 3 involves fine-tuning on smaller, specific datasets to build applications such as classifiers or personal assistants.

A useful analogy: the foundational model is a generalist who already knows how to read, write, reason, and communicate. Fine-tuning is the specialized training that teaches them the specific knowledge and conventions of a particular domain.

Types of Fine-Tuning

Instruction fine-tuning involves providing instruction-answer pairs as labeled data to teach the model specific tasks like language translation. Training examples take the form: "Translate the following sentence into French: [sentence]" → "[translation]". The model learns to follow instructions by seeing many such pairs.

Classification fine-tuning adapts the model to produce categorical outputs—for example, labeling a piece of text as positive or negative sentiment, or routing a customer query to the correct department. More broadly, fine-tuned LLMs can be used for specific tasks such as classification,

summarization, translation, and building custom chatbots. Large companies building custom chatbots, in particular, perform fine-tuning on foundational models rather than using foundational models alone.

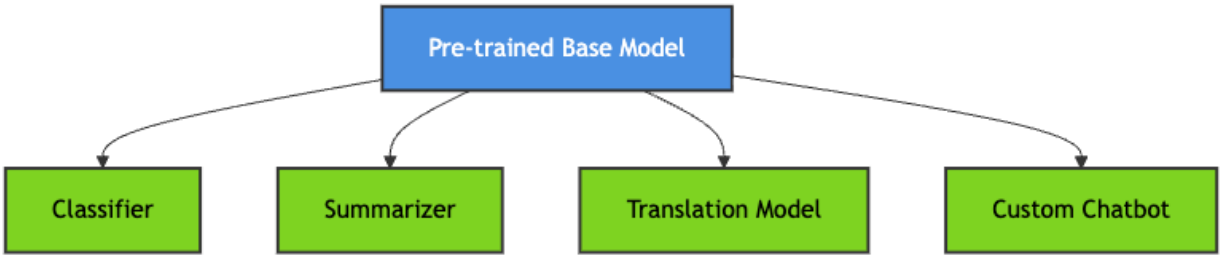


Figure 2.3: Fine-tuning a pre-trained base model into multiple specialized downstream models for different tasks.

2.5 Real-World Case Studies

The following three case studies illustrate different domains and different reasons for choosing fine-tuning over a general-purpose model.

SK Telecom: Customer Service in Korean

SK Telecom fine-tuned a model to improve customer service interactions for telecom-related conversations in Korean, resulting in a 35% increase in conversation summarization quality and a 33% increase in intent recognition accuracy. The results demonstrate that fine-tuning can produce a qualitatively better product for a specific use case—not merely a marginally different one.

Harvey: Legal AI

Harvey is an AI legal tool that was fine-tuned on legal case history data to assist attorneys and lawyers, addressing the limitation that foundational models lacked extensive knowledge of legal cases. Legal reasoning depends on specific precedents, statutes, and jurisdictional rules—content that may appear only sparsely in a general pre-training corpus. Harvey's fine-tuning on legal case data directly addresses this gap, giving the model the specialized knowledge that attorneys actually need.

JPMorgan Chase: Proprietary Financial Data

JPMorgan Chase developed a fine-tuned AI-powered LLM suite specifically for their employees using their proprietary data, rather than relying solely on GPT-4. This case adds a dimension absent from the previous two: data privacy and competitive advantage. By fine-tuning their own model on their own data, JPMorgan Chase keeps that data in-house while still benefiting from the language capabilities of a large pre-trained model. The pattern—fine-tune a foundational model on proprietary data, deploy internally—is increasingly common among large enterprises. The foundational model provides the expensive, compute-intensive language understanding; the fine-tuning supplies the domain-specific knowledge that makes the model useful for the organization's particular needs.

2.6 Connecting the Stages: A Unified View

Building a large language model from scratch typically proceeds in three stages: Stage 1 (understanding LLM basics), Stage 2 (pre-training), and Stage 3 (fine-tuning). Across those stages, the data requirements differ dramatically. Pre-training requires billions or trillions of tokens and massive computational resources, while fine-tuning operates on smaller, specific datasets.

This asymmetry has a practical implication: you do not need to train a model from scratch to build a useful product. You need to understand what the foundational model already knows, identify the gap between that and what your application requires, and close that gap through fine-tuning.

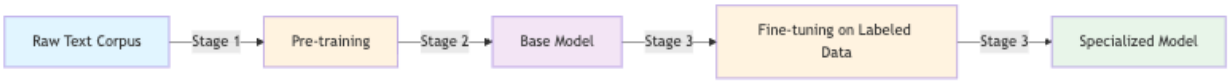


Figure 2.4: Three-stage pipeline for building large language models: starting from raw text corpus through pre-training to a base model, then fine-tuning on labeled data to produce a specialized model.

2.7 When to Pre-Train vs. When to Fine-Tune

Given everything above, a practical question emerges: when should you do each?

Pre-training is rarely the right choice for individuals or small teams. It requires a huge amount of data and significant computational power, making it inaccessible to most individuals. Fine-tuning, by contrast, becomes the right choice in several situations:

- **Domain-specific knowledge is required.** Legal case history, proprietary financial data, and specialized technical documentation are all examples.
- **The model must follow a specific format or instruction style.** Instruction fine-tuning can teach the model to follow specific formats.
- **The target language or subdomain is underrepresented in the pre-training data.** SK Telecom's Korean telecom use case illustrates this.
- **Data privacy demands that sensitive information stay in-house.** Fine-tuning on proprietary data, deployed internally, addresses this.

If none of these factors apply—if you are a student, a researcher, or a developer building a general-purpose tool—then a pre-trained model used directly may be entirely sufficient.

2.8 The Broader Landscape: Transformers, LLMs, and Their Relationship

The terms "transformers" and "large language models" should not be used interchangeably, as they are different concepts. Not all transformers are large language models, and not all large language

models are based on transformer architecture.

On the transformer side, the architecture extends well beyond language. Transformers can be used for computer vision tasks in addition to language tasks. Vision Transformers (ViT) are transformer models applied to computer vision tasks such as image recognition and image classification, and they achieve comparable or better results than Convolutional Neural Networks (CNNs) while requiring substantially fewer computational resources for pre-training. Transformers can also be used for image segmentation.

On the LLM side, earlier architectures predate the transformer entirely. Recurrent Neural Networks (RNNs) maintain a feedback loop to incorporate memory, and Long Short-Term Memory (LSTM) networks incorporate two separate paths: one for short-term memories and one for long-term memories. Both RNNs and LSTMs could perform sequence modeling and text completion tasks—tasks now dominated by transformer-based LLMs, but achievable through other architectural means.

2.9 Summary

- **Pre-training** trains a model on a massive corpus of raw, unlabeled text using next-word prediction (or masked-word prediction, in the case of BERT). The result is a foundational model with broad, general-purpose capabilities.
- **The emergent capabilities** of pre-trained models are remarkable: a model trained only on next-word prediction can perform translation, summarization, sentiment analysis, and question answering without task-specific training.
- **The limitation** of pre-trained models is that they lack domain-specific knowledge and may produce generic responses when applied to specialized tasks.
- **Fine-tuning** adapts a foundational model to a specific task or domain using labeled data. It is needed for production applications in specialized domains.
- **Real-world examples**—SK Telecom, Harvey, and JPMorgan Chase—demonstrate that fine-tuning produces measurable improvements in domain-specific performance.
- **Instruction fine-tuning** and **classification fine-tuning** are the two main approaches, suited to different output types.
- **Transformers and LLMs are related but distinct concepts**. Not every transformer is an LLM, and not every LLM is a transformer.

In the next chapter, we will move from concepts to mechanics, beginning with the data preparation pipeline: how raw text is tokenized, how tokens are assigned IDs, and how those IDs are converted into the vector embeddings that an LLM actually processes.

Chapter 3

Tokenization: From Words to Byte Pair Encoding

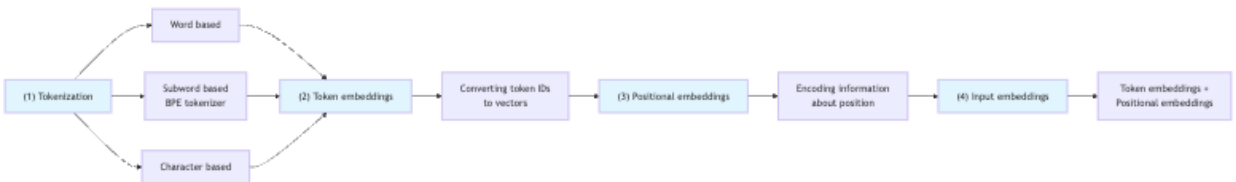
Before a large language model can process text, that text must be converted into a form the model can actually work with: numbers. This chapter traces that conversion from first principles—splitting raw text into tokens—all the way to the production-ready byte pair encoding used by GPT-2.

Figure 3.1: LLM data preprocessing pipeline: raw text is tokenized into token IDs, which are then converted to vector embeddings for model input.

3.1 The Role of Tokenization in the LLM Pipeline

Data preprocessing is the procedure of processing text data—sentences, paragraphs, documents—before it is fed as input to a large language model. Within that preprocessing pipeline, tokenization is the very first step: it converts raw text into tokens. Those tokens are then mapped to integer IDs, and those IDs are later converted into vector representations that the model actually trains on.

Concretely, the pipeline involves three stages: tokenizing input text into words, converting those words into token IDs, and converting token IDs into vector representations. This chapter focuses on the first two stages; the third—embedding—is covered in the next chapter.



tokens, (2) tokens → [4312, 995] IDs, (3) IDs → embedding vectors]

3.2 Three Families of Tokenization

There are three main types of tokenization algorithms: word-based tokenizers, sub-word-based tokenizers, and character-based tokenizers. Each makes a different trade-off between vocabulary size, handling of rare words, and semantic expressiveness.

Word-Based Tokenization

Word-based tokenization is the most intuitive approach: it converts an entire dataset into chunks of words, treating each word as an individual token. In earlier tokenization schemes, every word was treated as a unique token, along with special characters such as commas, full stops, and exclamation marks.

To see this concretely, consider splitting on whitespace:

```
# Source:
import re
text = "Hello world. This is a test."
tokens = re.split(r'\s+', text)
```

This yields one token per whitespace-separated chunk. Because word-based tokenization treats each unique word as a single token, a large corpus will produce an enormous vocabulary. Worse, any word not seen during training becomes an **out-of-vocabulary (OOV) word**—a word absent from the tokenizer's vocabulary—and must be handled with special context tokens.

Character-Based Tokenization

At the opposite extreme, character-based tokenization uses individual characters as tokens instead of words. For example, "dinosaur" is split into eight separate character tokens (d, i, n, o, s, a, u, r), whereas word-based tokenization would treat it as a single token.

The upside is a tiny vocabulary: character-based tokenization requires only approximately 256 characters, far fewer than word-based tokenization. It also eliminates the OOV problem entirely, since every possible string can be represented as a sequence of characters. The trade-off is that the model must learn to compose meaning from individual characters at every level, producing very long sequences with little per-token semantics.

Subword-Based Tokenization

Subword-based tokenization sits between the two extremes, using tokens that are neither full words nor individual characters. It combines features of both approaches by selectively splitting words based on frequency, which retains semantic similarity by sharing root-word tokens across related words like "tokens" and "tokenizing". Byte pair encoding (BPE) is the most widely used algorithm for implementing this approach and is the algorithm used by GPT models.

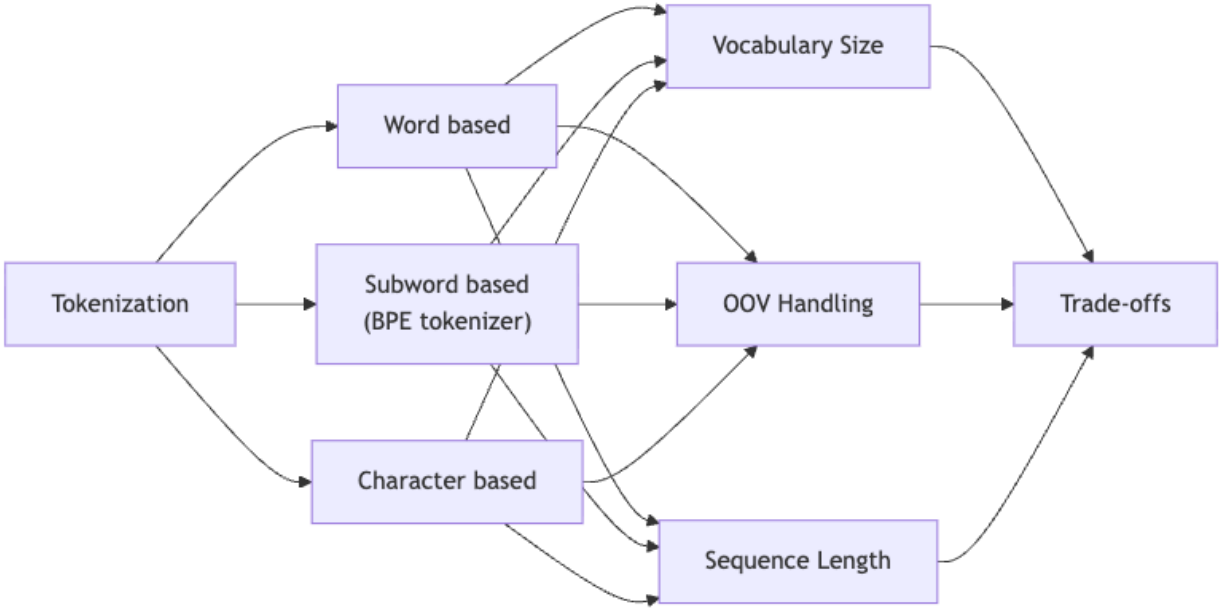


Figure 3.2: Comparison of three tokenization approaches (word-based, subword-based BPE, and character-based) showing their trade-offs across vocabulary size, out-of-vocabulary handling, and sequence length.

3.3 Building a Word-Based Tokenizer from Scratch

Although word-based tokenization is not what GPT uses, building one from scratch is the best way to understand the machinery that all tokenizers share: a vocabulary, an encode method, and a decode method. We will use a real text file as our corpus.

Loading the Raw Text

```
# Source:
with open('the-verdict.txt', 'r') as f:
    raw_text = f.read()
```

The variable raw_text stores the entire text content read from The Verdict file.

Splitting into Tokens

A naive whitespace split misses punctuation. A more robust approach splits on both punctuation and whitespace:

```
# Source:
tokens = re.split(r'[,\.;:\?!"\-\(\)/]|\s+', text)
tokens = [token for token in tokens if token.strip()]
```

This splits on commas, periods, semicolons, colons, question marks, exclamation marks, quotation marks, hyphens, parentheses, and forward slashes, as well as any whitespace. The list comprehension filters out empty strings that result from consecutive delimiters. An alternative regex pattern that achieves similar splitting is:

```
# Source:
result = re.split(r'[,.;:?_!()"\|\-\s]', text)
```

followed by:

```
# Source:
[item for item in result if item.strip()]
```

Using item.strip() filters out whitespace tokens from the result. Applied to the book example, the raw text yields 4,690 tokens after the regex-based tokenization scheme is applied.

Building the Vocabulary

A vocabulary is a dictionary where every unique token is mapped to a unique integer called a token ID. To build one, collect all unique tokens, sort them for reproducibility, and enumerate them:

```
# Source:
vocab = sorted(set(preprocessed))
vocab_dict = {token: idx for idx, token in enumerate(vocab)}
```

You can also capture the vocabulary size at the same time:

```
# Source:
vocab = sorted(set(preprocessed))
vocab_size = len(vocab)
```

Encoding and Decoding

Encoding is the process of converting tokens into token IDs. Decoding is the reverse: mapping token IDs back to tokens, which is needed to convert the model's numerical output back into readable text.

A tokenizer class is a Python class containing an encode method to convert text into token IDs and a decode method to convert token IDs back into text. Its ___init___ method takes a vocabulary parameter—a mapping from tokens to token IDs—and stores two dictionaries internally: str2int, which maps tokens (strings) to token IDs (integers), and int2str, the reverse mapping from token IDs back to tokens.

```
# Source:
class SimpleTokenizer:
    def __init__(self, vocab):
        self.string_to_int = vocab
        self.int_to_string = {idx: token for token, idx in vocab.items()}

    def encode(self, text):
        tokens = re.split(r'[,\.;:\?!"\-\(\)/]|\s+', text)
        tokens = [token for token in tokens if token.strip()]
        return [self.string_to_int[token] for token in tokens]

    def decode(self, ids):
        text = '␣'.join([self.int_to_string[idx] for idx in ids])
        return re.sub(r'\s+([,\.;:\?!"\-\(\)/])', r'\1', text)
```

The tokenizer class uses re.split and item.strip to convert text into tokens. Here is what each method does:

- **encode**: Splits input text on punctuation and whitespace to produce individual tokens, then converts each token to its ID using the str2int dictionary. It takes text as input and outputs token IDs. In the context of a transformer architecture, the encode method corresponds to the encoder block, converting text to token IDs for training data.
- **decode**: Converts token IDs back to individual tokens using the int2str dictionary, then joins them into a single string. It takes token IDs as input and outputs text. A post-processing regex removes spurious spaces before punctuation (e.g., "Hello ," becomes "Hello,").

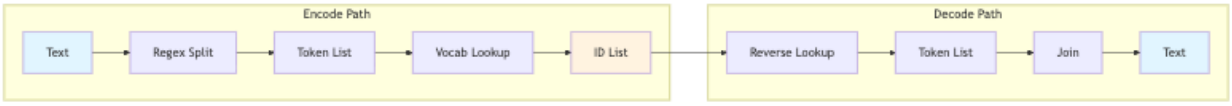


Figure 3.3: Tokenization encode and decode pipeline: the encode path converts text to token IDs via regex splitting and vocabulary lookup, while the decode path reverses the process to reconstruct text.

3.4 Handling Unknown Words with Special Tokens

The SimpleTokenizer above will raise a KeyError if it encounters a word not in its vocabulary. The standard solution is to introduce **special tokens**—reserved tokens that handle unknown words and mark boundaries between text sources.

The **unknown token** (<|unk|>) is added to the vocabulary to represent any word absent from it. The **end-of-text token** (<|endoftext|>) is inserted between unrelated text sources to signal document boundaries and prevent the model from mixing separate documents. Together, these special tokens handle words not present in the vocabulary and separate multiple text sources.

Adding Special Tokens to the Vocabulary

```
# Source:
# Adding special tokens to vocabulary
pre_processed.extend(['<|endoftext|>', '<|unk|>'])
vocab = {token: idx for idx, token in enumerate(sorted(pre_processed))}
```

Updating the Encoder to Use the Unknown Token

```
# Source:
# In SimpleTokenizer v2 encode method: if a word is not in vocabulary, replace it
if item not in self.str2int:
    tokens.append(self.str2int['<|unk|>'])
else:
    tokens.append(self.str2int[item])
```

Other Special Tokens

Some researchers use additional special tokens beyond <|unk|> and <|endoftext|>:

- **BOS (Beginning of Sequence)**: marks the start of a text and signals to the LLM where a piece of context begins.
- **EOS (End of Sequence)**: positioned at the end of a text, useful for separating unrelated passages.
- **Padding token**: extends shorter texts in a batch to match the length of the longest text, enabling parallel processing of variable-length sequences.

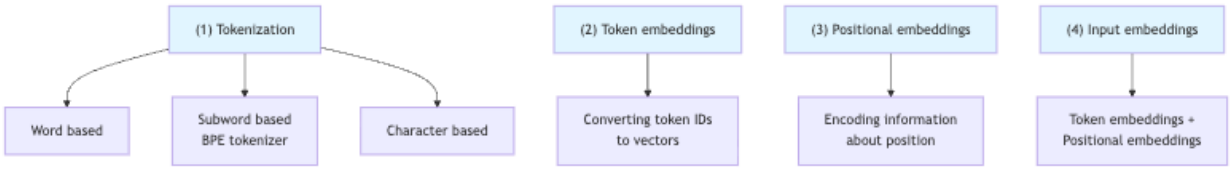


Figure 3.4: LLM Data Preprocessing: A four-step pipeline showing tokenization methods (word-based, subword-based with BPE, character-based), token embeddings, positional embeddings, and their combination into input embeddings.

3.5 The Limits of Word-Based Tokenization

Consider the words “old”, “older”, “finest”, and “lowest”. Word-based tokenization treats each as a distinct token, yielding four tokens for this small dataset—even though “old” and “older” clearly share a root. Subword tokenization solves this by retaining root words and breaking down rare words into subword units.

3.6 Byte Pair Encoding: The Algorithm

Byte pair encoding (BPE) is a subword-based tokenization method used by GPT models and distinct from simple regex-based splitting. Compared to word-level tokenization, BPE reduces vocabulary size while still handling rare and novel words gracefully.

Step 1: Initialize with Characters

The first step is to split all words into character-level tokens. A special end-of-word token (</w>) is appended to each word to mark word boundaries, enabling the algorithm to distinguish between subwords at word endings and the same subwords appearing within words. For example, “low” becomes l o w </w> and “lower” becomes l o w e r </w>.

Step 2: Count and Merge Pairs

The algorithm then identifies the most frequently occurring adjacent byte pair and replaces it with a new token not already present in the data. If “l o” appears very frequently, merging it into “lo”

reduces the total number of tokens in the corpus. This process repeats iteratively, building up a merge table of learned subword units.

Step 3: Apply to New Text

Once the merge table is learned, the BPE tokenizer breaks down words not in its predefined vocabulary into smaller subword units or individual characters. It scans text from left to right, decomposing each word based on what is present in the vocabulary and assigning each piece a unique token ID. The version of BPE used for large language models is a tweaked variant of the standard algorithm, adapted to convert entire sentences into subwords.

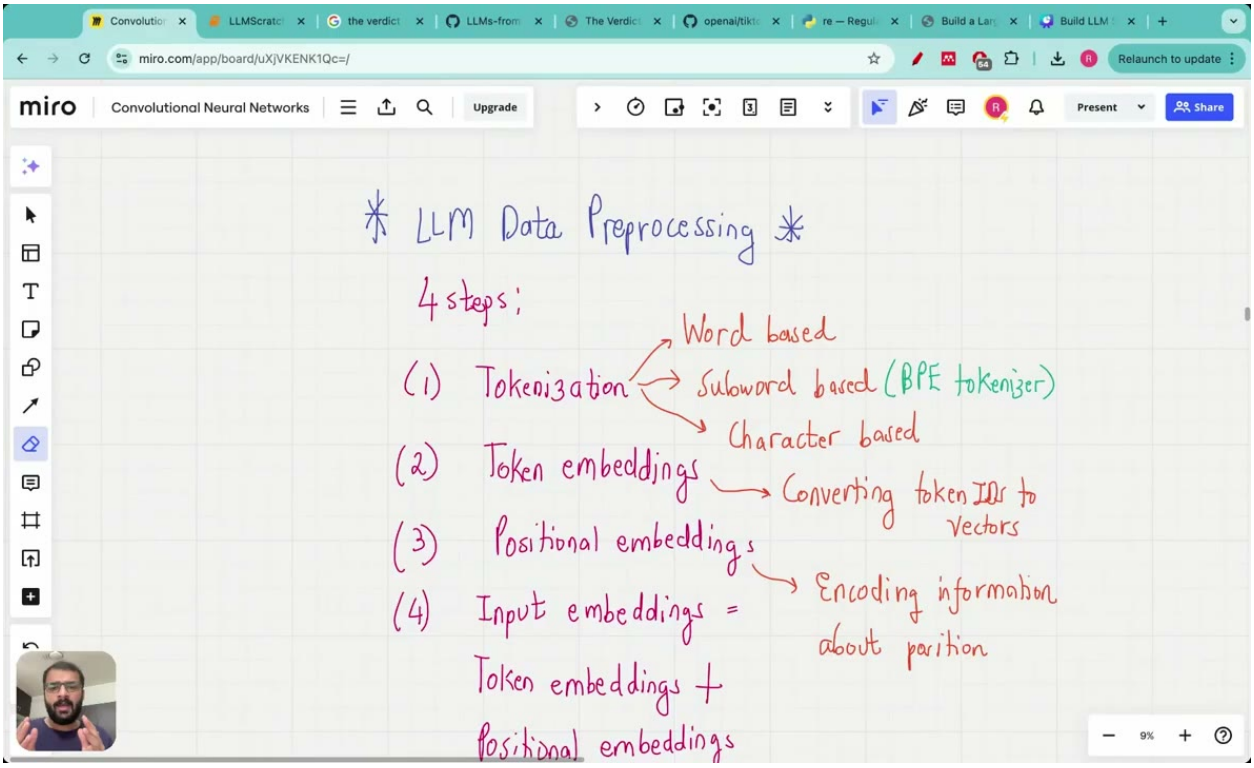


Figure 3.5: Step-by-step BPE merge example showing three rounds of merging the most frequent pair in a character-token corpus, with merge operations tracked in a table.

A Worked Example

After adding end-of-word markers and splitting into characters, the initial token inventory for “old”, “older”, “finest”, and “lowest” is: o, l, d, </w>, e, r, f, i, n, s, t, w. The algorithm counts all adjacent pairs across all occurrences, finds the most frequent, merges it into a new token, and repeats—ultimately retaining root words and decomposing rare words into subword units.

3.7 Using tiktoken for GPT-2 Compatible BPE

Implementing BPE from scratch is relatively complicated, so in practice we use the tiktoken Python library instead. The tiktoken library provides tokenizers using the same encodings as GPT-2.

Installation and Basic Usage

```
# Source:
import tiktoken
tokenizer = tiktoken.get_encoding('gpt2')
```

The tiktoken BPE tokenizer has an encode method that converts text into token IDs:

```
token_ids = tokenizer.encode(text)
print(token_ids)
# e.g. [15496, 11, 466, 345, 588, 8887, 30]
```

How the BPE Tokenizer Handles Unknown Words

Rather than mapping unfamiliar words to a generic unknown token, the BPE tokenizer breaks them down into smaller subword units or individual characters. It scans text from left to right, decomposing each word based on what is present in the vocabulary and assigning each piece a unique token ID. This means that even a completely novel word—a proper noun or a technical term—will be represented as a sequence of recognizable subword pieces.

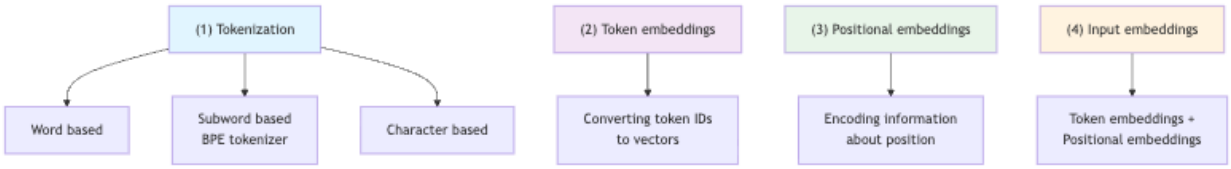


Figure 3.6: LLM Data Preprocessing: Four-step pipeline showing tokenization methods (word-based, subword-based BPE, character-based), token embeddings, positional embeddings, and final input embeddings.

3.8 Putting It All Together: The Full Tokenization Pipeline

Tokenization prepares input text for LLM training in two steps: splitting text into tokens and converting those tokens into token IDs. The encode method of a tokenizer class handles the forward direction—converting sample text into tokens and then into token IDs—while the decode method handles the reverse, converting token IDs back into text.

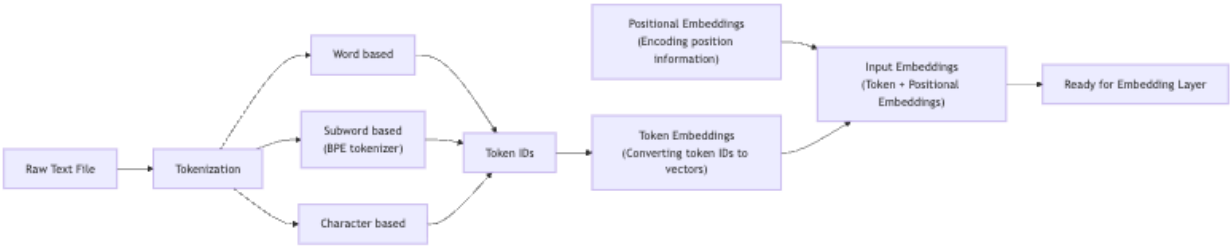


Figure 3.7: LLM data preprocessing pipeline: from raw text through tokenization (word, subword, or character-based), token embeddings, and positional embeddings to final input embeddings.

3.9 Summary

This chapter covered the full tokenization pipeline, from first principles to production tooling.

- **Three families of tokenization** exist—word-based, character-based, and subword-based—each making different trade-offs on vocabulary size, OOV handling, and sequence length.
- **Word-based tokenization** is easy to implement but suffers from large vocabularies and OOV words. We built a SimpleTokenizer class with encode and decode methods and extended it with special tokens (<|unk|> and <|endoftext|>).
- **Character-based tokenization** eliminates OOV with a tiny vocabulary but produces long sequences with little per-token semantics.
- **Subword-based tokenization**, and BPE in particular, offers the best of both worlds: it retains root words, handles rare words gracefully, and keeps vocabulary sizes manageable.
- **BPE** works by iteratively merging the most frequent adjacent byte pair, starting from a character-level initialization and using end-of-word markers to preserve word boundaries.
- The **tiktoken library** provides a production-ready BPE tokenizer compatible with GPT-2, accessible with just two lines of code.

Chapter 4

Data Preprocessing: Input-Target Pairs, Token Embeddings, and Positional Embeddings

Before a large language model can learn anything, raw text must be transformed into a form the model can actually consume. This chapter walks through the complete data preprocessing pipeline: creating input-target pairs from raw text, batching those pairs efficiently with a DataLoader, converting token IDs into dense vector embeddings, and finally injecting positional information so the model knows where each token sits in a sequence.

The pipeline has four distinct stages:

4.1 Creating Input-Target Pairs

Why LLMs Need a Special Pairing Strategy

Standard supervised learning tasks — classification, regression — come with explicit labels. Large language models, by contrast, use a specific technique for creating input-target pairs that differs from those conventional approaches.

Input-target pairs in LLMs are structured so that the input is a sequence of words up to a certain point and the target is the next word that follows. More precisely, there are two variables, x and y, where x contains input tokens and y contains targets that are the inputs shifted by one position. Tokens on the left of the arrow represent the input to the model, and the token on the right represents the target token ID the model is supposed to predict.

For example, given the sequence ["The", "cat", "sat"], the model should predict "on"; given ["The", "cat", "sat", "on"], it should predict "the". The same corpus of text generates thousands of such pairs automatically — no human labeling required.

Input-target pairs must be created before vector embeddings are fed to the LLM training process, and a dataset for language modeling must consist of these input-output pairs, not just raw tokens.

Context Size and the Sliding Window

A context size of 4 means the model is trained to look at a sequence of 4 tokens to predict the next one. Intuitively, context size represents how many words the model should pay attention to at one time.

Input-output pairs are constructed based on context size, where each pair contains multiple prediction tasks corresponding to predicting the next word at each position:

- Input [t0] → target t1
- Input [t0, t1] → target t2
- Input [t0, t1, t2] → target t3
- Input [t0, t1, t2, t3] → target t4

Input-output pairs are constructed by taking a sequence of tokens as input and the next token as the target output. The following code snippet illustrates this loop:

```
# Source:
context = encoded_data[:i]
desired = encoded_data[i]
```

A data loader fetches input-target pairs using a sliding window approach, advancing the window by one token position each time to produce the next pair.

Stride controls how far the window moves between consecutive pairs — specifically, how many tokens to skip when sliding the context window.

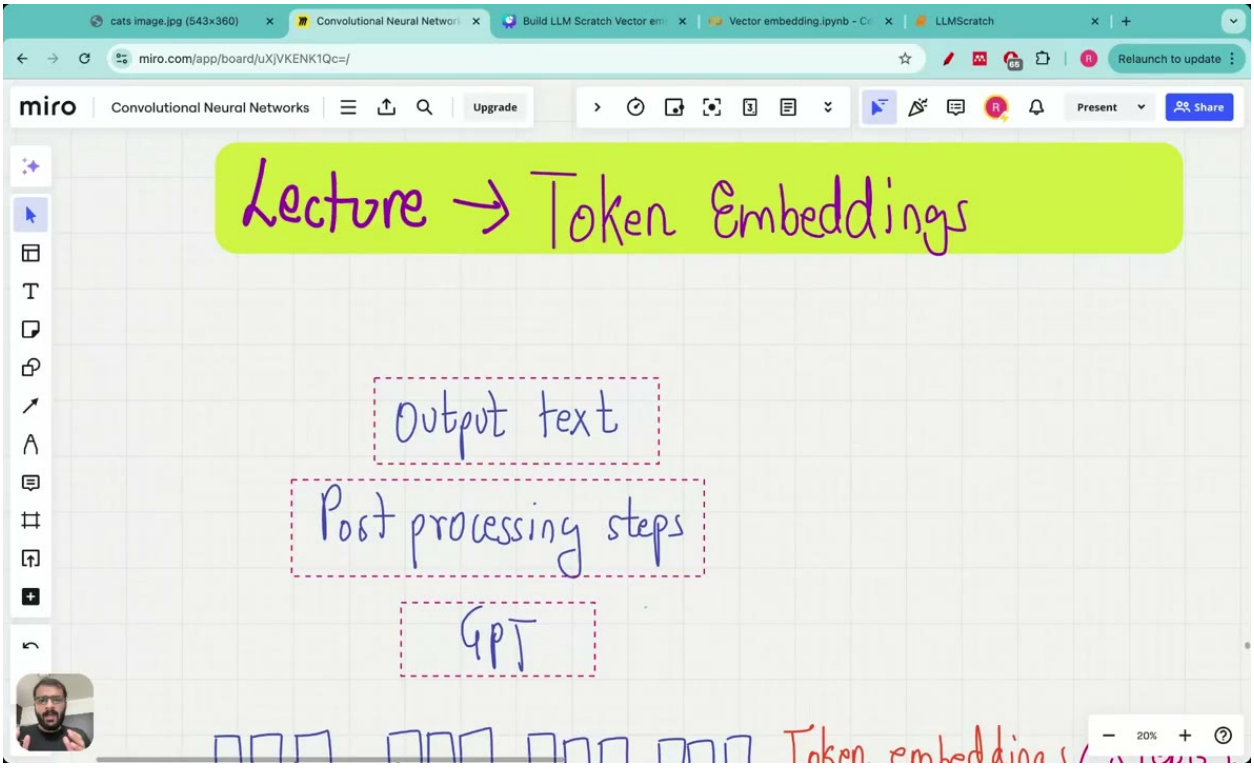


Figure 4.1: Token sequence with sliding window (context_size=4, stride=1) showing how successive input-target pairs are extracted from a token stream.

Loading the Raw Text

Before building pairs, we need encoded tokens. The following code reads a text file and encodes it with a tokenizer:

```
# Source:
raw_text = f.read()

enc_text = tokenizer.encode(raw_text)
print(len(enc_text))
```

This produces a flat list of integer token IDs ready for windowing.

4.2 The Dataset and DataLoader Abstraction

Why Use a DataLoader?

A data loader is a PyTorch tool that processes data by creating input-output target pairs using a sliding window approach, then iterating over those pairs and returning them as PyTorch tensors. A tensor can be thought of as a multi-dimensional array. Batch size specifies how many input-output pairs are processed before updating model parameters.

Dataset Structure

In a data loader implementation, the input tensor X contains rows representing input contexts, and the target tensor Y contains the corresponding prediction targets — that is, the inputs shifted by one word. The ___getitem___ method returns the input tensor row and target tensor row at a given index.

The create_data_loader function initializes a tokenizer, creates a dataset instance, and wraps it in a DataLoader for batch processing:

```
# Source:
data_loader = DataLoader(dataset, batch_size=8)
```

The DataLoader extracts input-output pairs in batches, which are then converted into vector embeddings before being fed into the model.

Workers and Parallel Loading

The num_workers parameter enables parallel computing by distributing data loading across multiple CPU threads, which can significantly speed up data ingestion for large datasets.

Verifying the Output

A DataLoader with batch size of 1 and context size of 4 produces input-output pairs where the output tensor is the input tensor shifted by one position — confirming that input-target pairs are sequences where the target is the input shifted by one position.

[Diagram: Input-target tensor pairs (X, Y) for language modeling, where Y is X shifted right by one token position.]

4.3 Token Embeddings: From IDs to Vectors

What Is a Token Embedding?

Token embeddings map words into a vector space where each word is represented as a vector with a fixed number of dimensions, and they serve as the input to training large language models such as GPT.

The terminology can be confusing: token embeddings are also called vector embeddings or word embeddings, though "word embeddings" is not entirely accurate. The reason is that "token" is a broader term than "word" — a token might be a subword, a punctuation mark, or a special symbol — which is why "token embeddings" is the preferred terminology.

Token embeddings are created by converting token IDs into embedding vectors using an embedding weight matrix, making them the third step in the overall LLM workflow: (1) tokenize input text into tokens, (2) convert tokens into token IDs, and (3) convert token IDs into token embeddings.

The Embedding Weight Matrix

An embedding layer weight matrix is a lookup table that stores vector embeddings for each token ID in the vocabulary, with dimensions of (vocabulary_size, embedding_dimension). Constructing one requires two quantities: the vector dimension (the size each token is projected to) and the vocabulary size (the number of unique token IDs).

The weight matrix serves a dual role: it is both a matrix of trainable weights and a lookup table for retrieving vector representations by token ID, mapping vocabulary IDs to dense vectors of a specified embedding dimension.

Word2Vec: A Historical Perspective

The following code loads a classic Word2Vec model trained on Google News:

```
# Source:
import gensim.downloader as api
model = api.load("word2vec-google-news-300")
word_vectors = model
print(word_vectors['computer'])
```

Modern LLMs do not use pre-trained Word2Vec vectors; instead, they learn their own embeddings from scratch as part of end-to-end training. Creating token embeddings for large vocabularies is computationally expensive, which is one reason why training large language models like GPT takes a huge amount of time.

`torch.nn.Embedding`: The PyTorch Lookup Table

PyTorch provides a dedicated module for this purpose. torch.nn.Embedding is a lookup table that stores embeddings of a fixed dictionary and size, used to store word embeddings and retrieve them

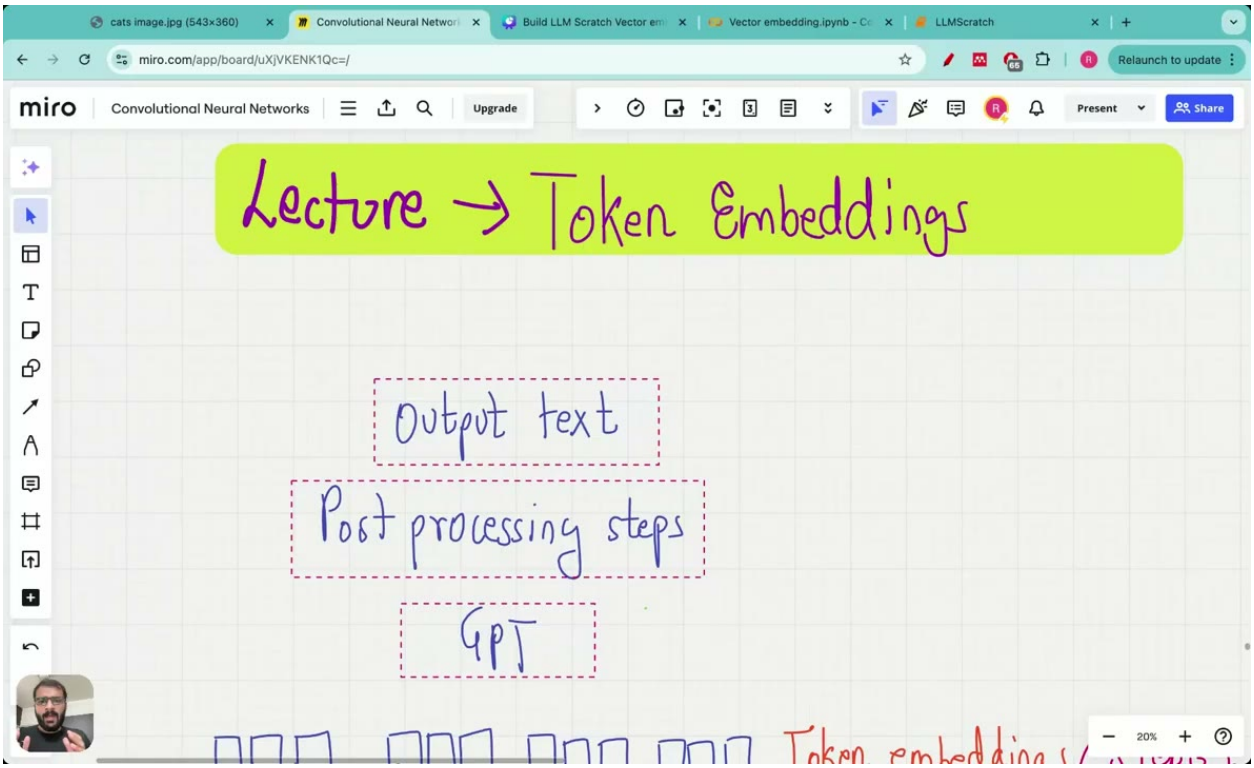


Figure 4.2: A rectangular embedding matrix of shape (vocab_size, embedding_dim) where each row corresponds to a token ID and contains its learned embedding vector; row lookup retrieves the embedding for a given token.

using indices:

```
# Source:
torch.nn.Embedding(num_embeddings, embedding_dim)
# Creates an embedding layer with num_embeddings rows and embedding_dim columns
# Stores embeddings of a fixed dictionary and size

# Source:
embedding_layer = torch.nn.Embedding(num_embeddings=6, embedding_dim=3)
```

torch.nn.Embedding(num_embeddings, embedding_dim) creates an embedding layer that functions as a lookup table, retrieving embeddings using token indices. To inspect the underlying weight matrix:

```
# Source:
embedding_layer.weight
```

The lookup operation retrieves only the embedding vectors corresponding to the input IDs, without computing embeddings for unused vocabulary IDs. Just as convolutional neural networks exploit spatial features of an image before training, token embeddings extract a structured representation of each token before it enters the model.

Building the Token Embedding Layer for GPT

```
# Source:
vocab_size = 50257
embedding_dim = 256
token_embedding_layer = torch.nn.Embedding(vocab_size, embedding_dim)
# Creates embedding matrix with 50257 rows and 256 columns
```

A simpler illustration of the same call:

```
# Source:
embedding = torch.nn.Embedding(vocab_size, embedding_dim)
# Example: torch.nn.Embedding(4, 5) creates a 4x5 embedding matrix
```

With a batch size of 8 and a context size of 4, the token embedding layer produces an output tensor of shape $8 \times 4 \times 256$.

4.4 The Problem with Token Embeddings Alone

Token embeddings capture semantic meaning — words with similar meanings end up close together in the vector space. However, token embedding does not take into account the position of words in a sequence, so identical words at different positions receive the same embedding vector. In other words, token embeddings alone carry no information about where a word appears in a sentence, and without that information the model cannot distinguish sentences that differ only in word order.

It is therefore important to inject additional position information into the model alongside the semantic content captured by token embeddings. Without token embeddings, the semantic relationship between related words like "dog" and "puppy" or "cat" and "kitten" is lost — but without positional embeddings, the *order* of those words is lost.

4.5 Positional Embeddings

What Is Positional Encoding?

Positional encoding is a technique that injects additional positional information into large language models to complement token embeddings. It is the final step in LLM data pre-processing, encoding information about the position of each token in a sequence. The terms "positional encoding" and "positional embedding" are used interchangeably to refer to vectors in a higher-dimensional space.

Both absolute and relative positional embeddings enable large language models to understand the order and relationship between tokens, leading to more accurate and context-aware predictions.

Absolute vs. Relative Positional Embeddings

There are two main approaches to encoding position information: absolute positional embeddings and relative positional embeddings.

In absolute positional embedding, the same token receives different final embeddings when it appears at different positions in the sequence, because a different positional embedding is added to the same token embedding at each position. This approach is well-suited for tasks where the fixed order of tokens is crucial, such as sequence generation. GPT models (GPT-3, GPT-4) use absolute positional embeddings that are optimized during training.

Relative positional encoding takes a different approach, emphasizing the relative distance between tokens rather than their absolute positions in the sequence. Intuitively, relative encodings let the model reason about "how far apart" two tokens are, rather than "what slot each token occupies."

The Sinusoidal Formula (Original Transformer)

The original Transformer paper proposed a fixed, formula-based approach to absolute positional encoding using sinusoidal functions. For even dimensions:

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$

And for odd dimensions:

$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

GPT-style models do not use this fixed formula; they use learned positional embeddings instead.

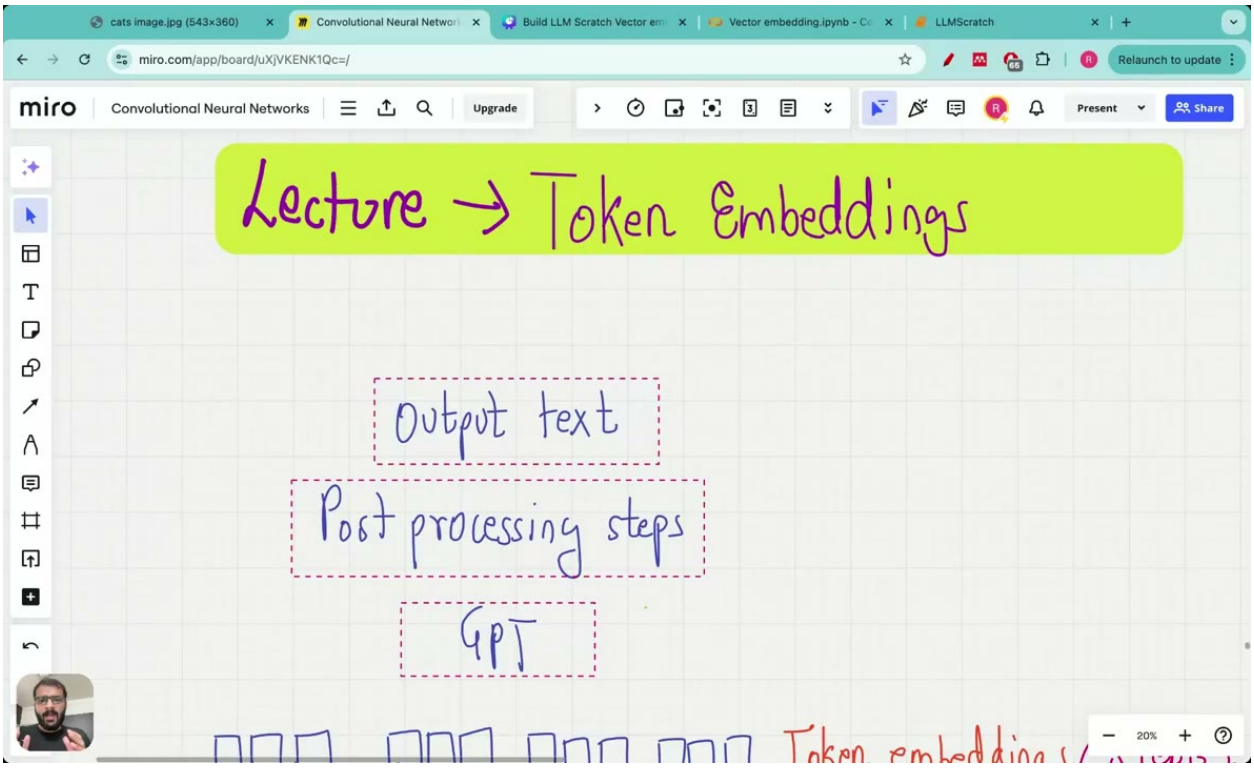


Figure 4.3: Side-by-side comparison of absolute positional embeddings (each position gets a unique fixed vector) versus relative positional embeddings (distances between positions are encoded).

Implementing Learned Positional Embeddings

Positional embedding matrix values are initialized randomly, similar to token embedding initialization, and both must be optimized during training.

To generate positional embeddings for a sequence of length max_length, we first create a sequence of position indices:

```
# Source:
torch.arange(max_length)
# creates a sequence of integers from 0 to max_length-1,
# used to index into the positional embedding lookup table
```

Passing positions 0, 1, 2, and 3 to the positional embedding lookup table generates four 256-dimensional positional embedding vectors. Because positions are identical across all sequences in a batch, the same four positional embedding vectors are reused for every input sequence.

4.6 Combining Token and Positional Embeddings

The Addition Operation

Positional embeddings are added to token embeddings using Python broadcasting, where the same positional embedding vectors are added to each row of the token embedding matrix. Concretely,

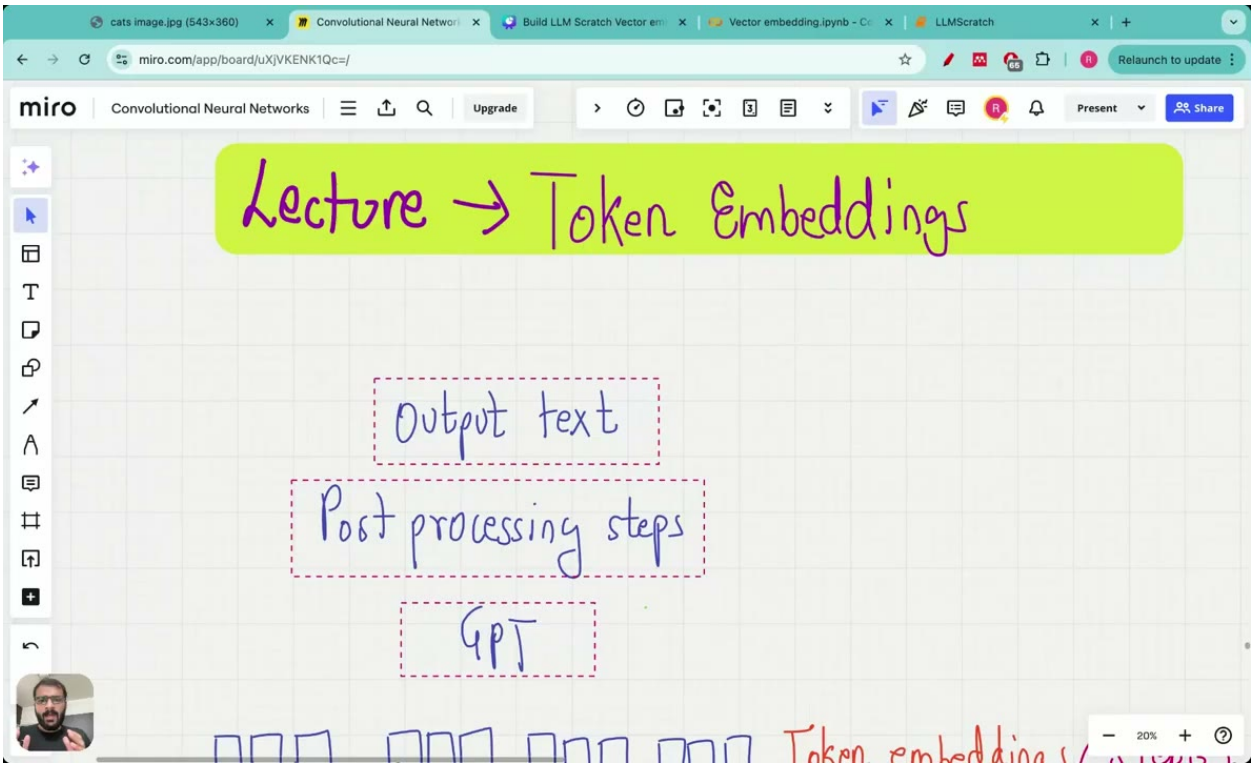


Figure 4.4: Positional embedding lookup table mapping position indices (0, 1, 2, 3) to distinct embedding vectors of shape (context_size, embedding_dim).

broadcasting converts the 4×256 positional embedding tensor into $8 \times 4 \times 256$ by duplicating values eight times — once for each item in the batch. PyTorch handles this expansion automatically, so no manual tiling is required.

The Complete Pipeline

With all four stages in place:

1. **Tokenization**: Raw text → token IDs (covered in earlier chapters)
2. **Token embeddings**: Token IDs → dense vectors via torch.nn.Embedding(vocab_size, embedding_dim)
3. **Positional embeddings**: Position indices → dense vectors via a second torch.nn.Embedding(context_size, e
4. **Input embeddings**: Token embeddings + positional embeddings → final input tensor

Token embeddings have shape $8 \times 4 \times 256$ and positional embeddings have shape 4×256. The resulting input embeddings feed directly into the first layer of the transformer. During inference, the model's own predictions loop back as new inputs — making it an autoregressive model, where the output of one iteration becomes the input of the next.

4.7 Putting It All Together: A Worked Example

Let us trace a concrete example through the entire pipeline.

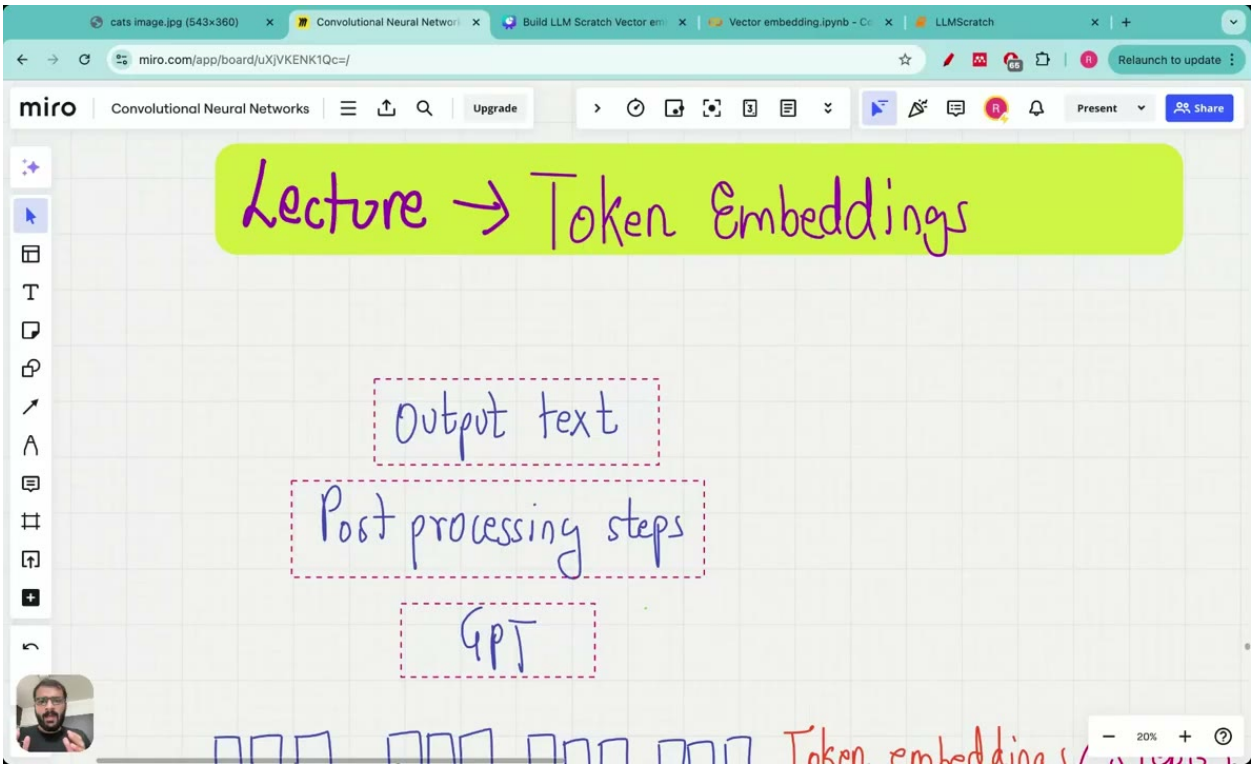


Figure 4.5: Token embedding tensor (8×4×256) plus positional embedding tensor (4×256) via broadcasting to produce input embedding tensor (8×4×256).

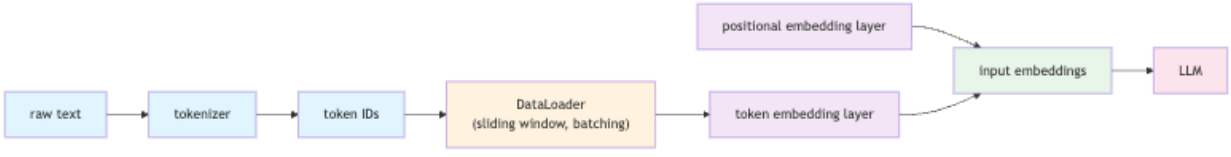


Figure 4.6: End-to-end LLM input pipeline: from raw text through tokenization, batching, and embedding layers to the language model.

Step 1 — Tokenize and encode. We read a text file and encode it into a list of integer token IDs.

Step 2 — Build input-target pairs. With a context size of 4 and a stride of 1, the sliding window produces pairs where each input is 4 tokens and each target is those same 4 tokens shifted right by one. The data loader uses this sliding window approach to fetch input-output target pairs.

Step 3 — Batch. A DataLoader with batch size 8 groups 8 such pairs into a single batch. The input tensor X has shape 8×4 and the target tensor Y has shape 8×4.

Step 4 — Token embeddings. We pass X through torch.nn.Embedding(50257, 256). The lookup retrieves one 256-dimensional vector per token ID, producing an output of shape $8 \times 4 \times 256$.

Step 5 — Positional embeddings. We create position indices $[0,\ 1,\ 2,\ 3]$ with torch.arange(4) and pass them through a second torch.nn.Embedding(4, 256), producing a tensor of shape 4×256.

Step 6 — Combine. Broadcasting expands the positional embedding from 4×256 to $8 \times 4 \times 256$, and the element-wise sum yields the final input embedding tensor of shape $8 \times 4 \times 256$.

4.8 Summary

- **Input-target pairs** are constructed by pairing each sequence of tokens with the next token as the target. The sliding window with a configurable stride generates these pairs efficiently.
- **The DataLoader** wraps a dataset of input-target pairs into batches of PyTorch tensors, with optional parallel loading via multiple worker threads.
- **Token embeddings** are dense vector representations stored in a lookup table (torch.nn.Embedding). Each token ID maps to a row in the embedding weight matrix, and the values are learned during training.
- **Token embeddings alone are insufficient** because they carry no information about token order. Positional embeddings address this gap.
- **Positional embeddings** come in two flavors — absolute and relative. GPT-style models use learned absolute positional embeddings, initialized randomly and optimized during training.
- **Input embeddings** are the element-wise sum of token and positional embeddings, combined via broadcasting, and form the final input to the transformer's first layer.

Chapter 5

The Attention Mechanism: From Simplified to Multi-Head

Understanding how language models weigh the importance of different words is the central challenge this chapter addresses. We build the attention mechanism from the ground up—starting with the historical motivation for why older sequence models struggled, then constructing a simplified, weight-free version of self-attention to build intuition, before arriving at the full multi-head causal attention used in modern LLMs.

5.1 Why Attention? A Brief History

Recurrent Neural Networks and Their Limits

Recurrent Neural Networks are designed to work with sequential data by maintaining a hidden state that captures information about previous inputs—indeed, the hidden state is the key innovation that enables RNNs to process sequential data at all.

The encoder–decoder architecture built on top of RNNs works roughly as follows: an encoder reads the input sequence token by token, updating its hidden state at each step. The final hidden state from the encoder is called the context vector, and it is handed to the decoder to generate the output sequence. Because the hidden state captures memory of previous inputs, the entire source sequence must be compressed into this single vector—a significant bottleneck.

Loss of context is the problem that arises when an RNN decoder struggles to capture longer dependencies and contextual information because it relies on only that one final hidden state. Long-term dependencies in sentences—complex structures where multiple clauses or phrases are connected—make it especially difficult for language models to identify relationships between distant words.

LSTMs alleviate the vanishing gradient problem by maintaining both a long-term memory route and a short-term memory route, but even LSTMs still funnel everything through a single context vector at the end of the encoder, leaving the bottleneck intact.

The Bahdanau Attention Breakthrough

The key insight that broke this bottleneck came from Bahdanau et al. Rather than forcing the decoder to rely solely on a single summary vector, the Bahdanau attention mechanism allows the decoder to selectively access different parts of the input sequence at each decoding step. Concretely, when decoding a particular part of the output, the decoder has access to all input tokens and decides how much attention to give to each one.

The crucial property that makes this work is *dynamic focus*: the ability of the decoder to selectively choose which inputs to attend to, and how much weight to assign each input, at every decoding step. Instead of one fixed summary, the decoder receives a weighted combination of *all* encoder states, with the weights recomputed fresh at every step.

From Attention to Self-Attention

Traditional attention operates across two sequences—an input and an output—determining which parts of the output are more related to which parts of the input. Self-attention takes this idea one step further: it looks at a single sequence and examines how different positions within that *same* sequence relate to each other.

More precisely, self-attention is a mechanism that allows each position of an input sequence to attend to all positions in the same sequence. The term "self" refers to the attention mechanism's ability to compute attention weights by relating different positions within a single input sequence.

5.2 Simplified Self-Attention (No Trainable Weights)

Before introducing the full machinery of queries, keys, and values, it is instructive to build a simplified version of self-attention with no learnable parameters at all. This is the purest and most basic form of the attention technique, and working through it carefully makes the full version much easier to understand.

Tokens, Embeddings, and the Goal

Throughout this chapter, x_i denotes the vector representation of the i-th token, where x_1 is the first token, x_2 is the second, and so on. An input embedding is a vector representation of a token that encodes semantic meaning but does not carry information about how other words in the sentence relate to it. Vector embeddings capture semantic meaning such that semantically related words like "journey" and "starts" are positioned closer to each other in vector space.

The goal of self-attention is to produce a *context vector* for each token: an enriched embedding that combines contributions from all input embeddings weighted by their corresponding attention weights. A context vector for a query contains information about both the query token itself and all other input elements in the sequence, and is denoted by z.

Attention Scores

The first step is to compute *attention scores*—intermediate values that quantify how much importance should be paid to each input word relative to a query word. A *query* is simply the element or token currently being examined.

For each query token, we compute its attention score against every other token in the sequence. The natural tool for measuring similarity between two vectors is the dot product: $a \cdot b = \sum(a_i \times b_i) = |a||b| \cos\theta$, where θ is the angle between the vectors. A higher dot product means the two vectors point in more similar directions, which we interpret as higher relevance.

Normalizing to Attention Weights

Raw dot products are not yet directly usable—they need to be normalized. Normalization transforms attention scores so that they sum to one, enabling interpretable statements about attention allocation as percentages.

Softmax normalization achieves this by taking the exponent of every element and dividing by the sum of all exponents:

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Naive softmax attention weights are thus computed by exponentiating each attention score and dividing by the sum of exponents. In PyTorch, torch.nn.Softmax(dim=None) applies this function to rescale tensor elements to the range $[0, 1]$ with a sum of 1; the dim parameter specifies the dimension along which normalization is computed.

Attention weights determine how much attention to give to each input token when computing the context vector for any embedding vector. Arranged into a matrix, each row represents the attention weights for one particular query word, and each column represents the attention weight between that query and a key word.

Computing the Context Vector

Once we have attention weights, the context vector is computed as a weighted sum over all input embeddings:

$$z_i = \sum_j \text{attention\_weight}_j \times \text{embedding\_vector}_j$$

5.3 Scaled Dot-Product Self-Attention with Trainable Weights

The simplified version above has no learnable parameters. Self-attention introduces trainable weights, which form the basis of the actual mechanism used in LLMs.

The Query, Key, and Value Matrices

The self-attention mechanism is implemented using three trainable weight matrices: Query (W_Q), Key (W_K), and Value (W_V). Each plays a distinct role:

- **Query** is analogous to a search query in a database and represents the current token the model is focusing on.
- **Key** represents items in the input sequence and is used to match against the query.

- **Value** represents the actual content or representation of the input items themselves.

W_Q, W_K, and W_V are trainable weight matrices whose parameters are learned from data during LLM training. In PyTorch, torch.nn.Parameter is a Tensor subclass that marks tensors as module parameters, automatically adding them to the module's parameter list and exposing them through the parameters() iterator. These weight matrices are initialized as torch.nn.Parameter objects with random values and shape (D_in, D_out).

Computing Queries, Keys, and Values

Given an input x, the three projections are computed as straightforward matrix multiplications:

```
keys = x @ W_key
queries = x @ W_query
values = x @ W_value
```

Or equivalently, using named weight matrices:

```
keys = inputs @ W_k
values = inputs @ W_v
queries = inputs @ W_q
```

Scaling by $\sqrt{d_k}$

Attention scores are computed as queries multiplied by the transpose of keys: $Q \times K^T$. However, as the key dimension d_k grows, dot products grow in magnitude, which can push the softmax into regions with very small gradients. The solution is to scale by $\sqrt{d_k}$ before applying softmax:

```
d_k = keys.shape[-1]  # Extract key embedding dimension from last axis
attention_scores_scaled = attention_scores / (d_k ** 0.5)  # Scale by sqrt(d_k)
attention_weights = softmax(attention_scores_scaled, dim=-1)  # Apply softmax over
```

In compact form:

```
attention_scores = attention_scores / (keys.shape[-1] ** 0.5)
attention_weights = softmax(attention_scores, dim=-1)
```

Computing the Final Context Vectors

Context vectors are computed by multiplying attention weights by the values matrix. An attention weight between two tokens (e.g., between "journey" and "your") indicates how much the model should attend to the second token when processing the first as the query:

```
context_vector = attention_weights @ values
```

Each element in an attention score matrix row encodes how much a particular token relates to the query token—that is, how much importance should be paid to that token when processing the query.

5.4 Causal (Masked) Attention

The self-attention mechanism described so far allows every token to attend to every other token in the sequence, including future ones. For language modeling, where the model must predict the next token given only the tokens seen so far, this look-ahead would be a form of cheating. Causal attention solves this problem.

The Concept of Causal Attention

Causal attention is a special form of self-attention, also called masked attention. It restricts the model to only consider previous and current inputs when processing any given token—more precisely, it is designed to prevent future tokens from influencing past tokens.

The causal attention mask is the set of attention weights above the diagonal that are zeroed out. Intuitively, if we lay out the attention weight matrix with query tokens as rows and key tokens as columns, position (i, j) represents how much token i attends to token j. For causal attention, all positions where $j > i$—everything above the diagonal—must be masked.

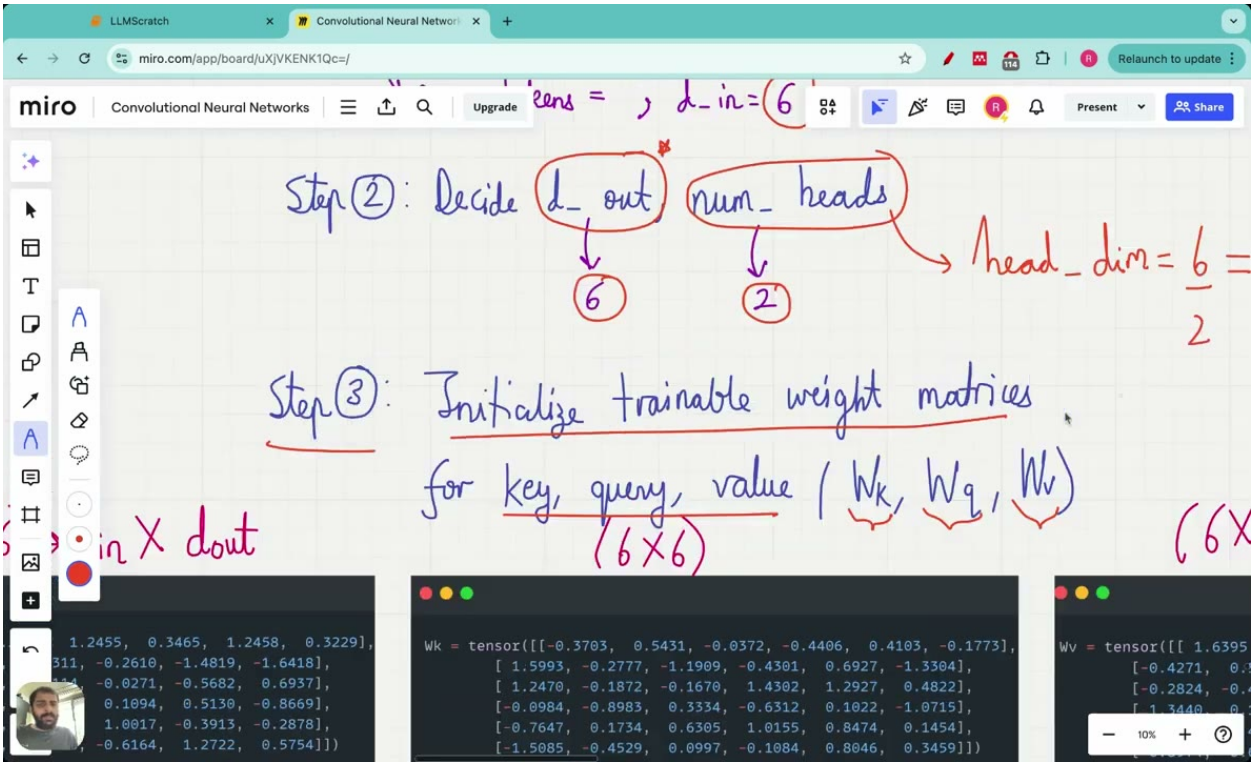


Figure 5.1: Multi-head attention setup showing Step 2 (deciding output dimension and number of heads with head_dim calculation) and Step 3 (initializing trainable weight matrices Wk, Wq, Wv with dimensions 6×6).

Triangular Matrices

To implement the mask, we rely on two types of triangular matrices:

- An **upper triangular matrix** has all elements below the diagonal set to zero.

- A **lower triangular matrix** has all elements above the diagonal set to zero.

For causal masking we want a lower triangular mask—ones on and below the diagonal, zeros above. PyTorch provides torch. tril for exactly this purpose:

```
mask = torch.tril(torch.ones(context_length, context_length))
```

Approach 1: Multiply and Renormalize

One straightforward approach is to multiply the attention weights element-wise by the mask:

```
masked_attn_weights = attn_weights * mask_simple
```

Because the masked rows no longer sum to one, we renormalize:

```
mask_simple_normalized = masked_attn_weights / masked_attn_weights.sum(dim=1, keepd
```

Approach 2: Mask Before Softmax (Preferred)

A cleaner and more numerically stable approach is to mask *before* applying softmax. Masking in transformers sets attention scores for future tokens to large negative values, making their influence in the softmax calculation effectively zero.

First, construct the upper triangular mask:

```
mask = torch.triu(torch.ones(seq_len, seq_len), diagonal=1)
```

Then fill those positions with $-\infty$:

```
attention_scores = attention_scores.masked_fill(mask == 1, float('-inf'))
```

Then apply softmax as usual:

```
attention_weights = torch.softmax(attention_scores / sqrt(d_k), dim=-1)
```

Because $e^{-\infty} = 0$, the masked positions contribute exactly zero weight after softmax—no separate renormalization step is needed.

Dropout in Attention

In practice, dropout is also applied to the attention weights after the softmax. Dropout is a deep learning technique where neurons in different layers are randomly switched to zero during training, preventing the model from over-relying on any particular attention pattern. torch.nn.Dropout(p=0.5) creates a dropout layer that randomly zeroes elements with probability 0.5 and scales the remaining elements by $1/(1 - 0.5) = 2$.

Putting It Together: The CausalAttention Forward Pass

The input x to the causal attention class has shape (batch_size, num_tokens, input_dimensions). A complete forward pass looks like this:

```
attn_scores = queries @ keys.transpose(1, 2)
attn_scores.masked_fill_(self.mask.bool()[:num_tokens, :num_tokens], -torch.inf)
attn_weights = torch.softmax(attn_scores / keys.shape[-1]**0.5, dim=-1)
attn_weights = self.dropout(attn_weights)
context_vec = attn_weights @ values
```

Notice the slice [:num_tokens, :num_tokens] applied to the mask. This ensures the causal mask is created only up to the number of tokens T in the current batch, gracefully handling cases where the batch is smaller than the maximum supported context size.

5.5 Multi-Head Attention

In practice, LLMs use *multi-head attention*, which runs several attention mechanisms in parallel and combines their outputs, allowing the model to capture different types of relationships simultaneously.

What Is Multi-Head Attention?

Multi-head attention refers to dividing the attention mechanism into multiple heads, each operating independently. It is called "multi-head" because it aggregates the outputs of these multiple independent attention heads. More concretely, it involves creating multiple instances of the self-attention mechanism, each with its own weight matrices, and then combining their outputs.

The Wrapper Approach

The simplest implementation wraps several causal attention modules and concatenates their outputs. In the forward method, torch.cat is used with dimension=−1 to concatenate outputs along the columns, combining the context vectors from all heads into a single, wider vector. D_{out} is the output dimension of each individual attention head.

The Efficient Weight-Split Approach

A more computationally efficient alternative avoids running separate modules entirely. All heads' queries, keys, and values can be computed in one batched matrix multiply, then reshaped and transposed to obtain the per-head tensors—achieving the same result with significantly less overhead.

5.6 The Mathematics of Scaling by $\sqrt{d_k}$

As d_k grows, dot products grow in magnitude proportionally to $\sqrt{d_k}$. Dividing by $\sqrt{d_k}$ brings the dot products back to unit variance, keeping the softmax in a well-behaved regime regardless of the embedding dimension. Attention weights are therefore computed by scaling attention scores by the square root of the key embedding dimension before applying softmax.

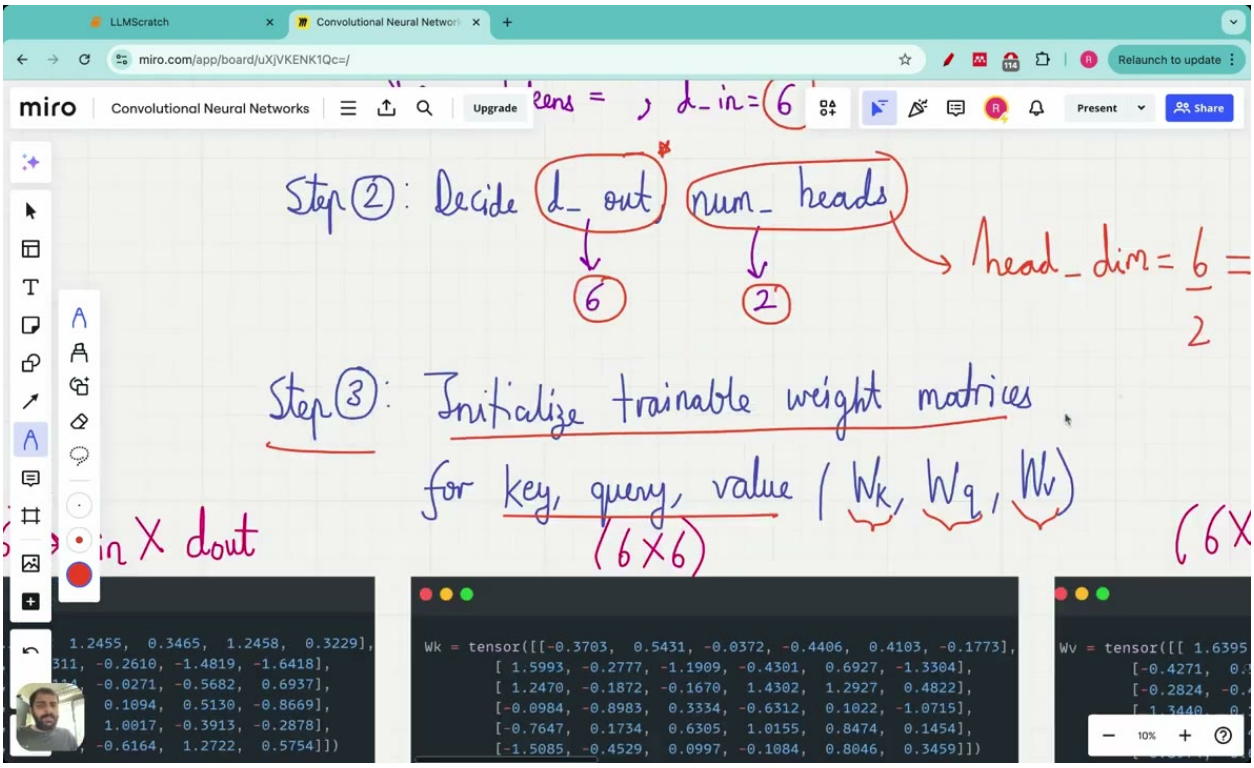


Figure 5.2: Multi-head attention setup showing Step 2 (deciding output dimension d_out=6 and number of heads=2, with head_dim=3) and Step 3 (initializing trainable weight matrices Wk, Wq, Wv with dimensions 6×6), along with tensor code examples.

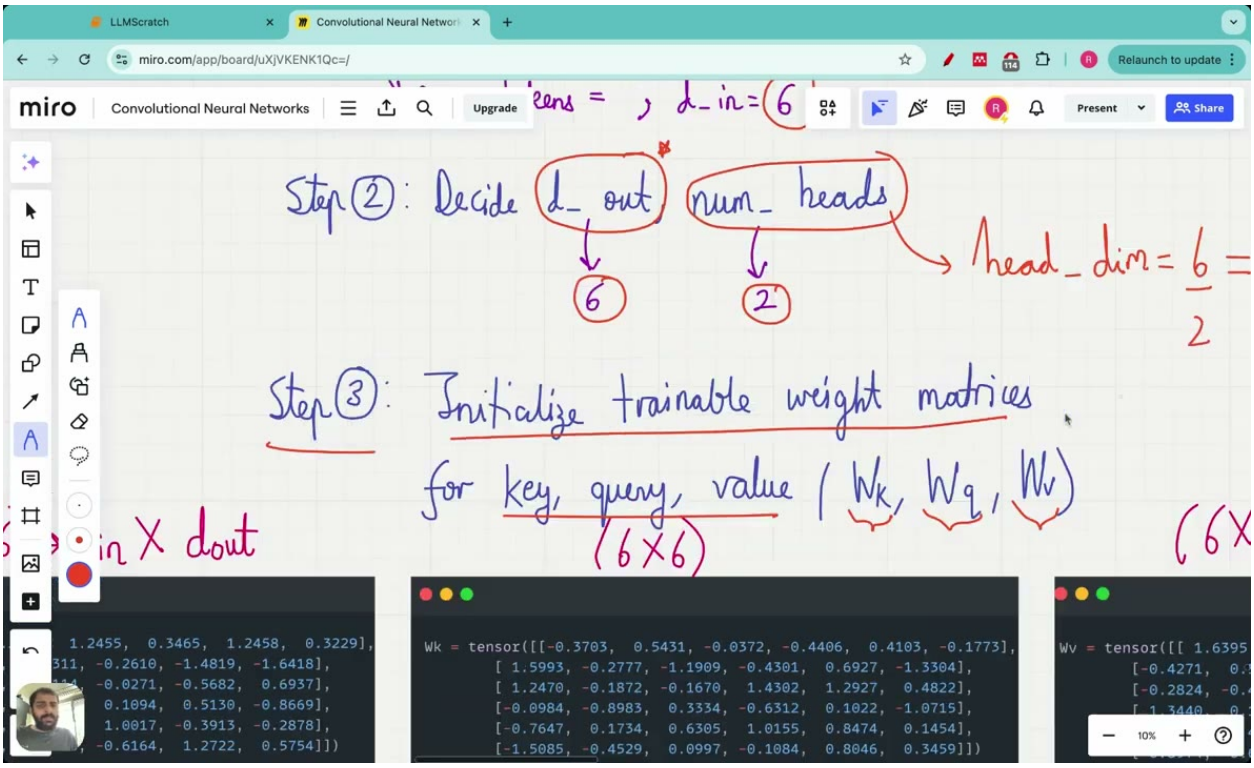


Figure 5.3: Multi-head attention setup: Step 2 decides output dimension (d_out=6) and number of heads (num_heads=2), computing head_dim=3; Step 3 initializes trainable weight matrices (Wk, Wq, Wv) with dimensions 6×6.

5.7 Putting It All Together

Let us trace through the complete flow of a single forward pass through a multi-head causal attention module, from input to output:

1. **Input**: x has shape (batch_size, num_tokens, input_dimensions).
2. **Linear projections**: Each token embedding is projected into query, key, and value spaces using learned weight matrices W_Q, W_K, W_V.
3. **Attention scores**: Queries are multiplied by the transpose of keys: $Q \times K^T$. Each element in the resulting matrix encodes how much a particular token relates to the query token.
4. **Scaling**: Scores are divided by $\sqrt{d_k}$ to prevent softmax saturation.
5. **Causal masking**: Future positions are filled with $-\infty$ so that after softmax they contribute zero weight.
6. **Softmax**: Scores are converted to attention weights that sum to one.
7. **Dropout**: Attention weights are randomly zeroed during training for regularization.
8. **Weighted sum**: Attention weights are multiplied by the values matrix to produce context vectors.
9. **Multi-head combination**: Context vectors from all heads are concatenated and projected to the output dimension.

5.8 Summary

- **Motivation**: RNNs compress entire sequences into a single context vector, losing information about long-range dependencies. The Bahdanau attention mechanism solved this by giving the decoder dynamic access to all encoder states at every decoding step.
- **Self-attention**: Rather than attending across encoder and decoder, self-attention allows each position in a single sequence to attend to all other positions in that same sequence.
- **Simplified self-attention**: Without trainable weights, attention scores are raw dot products between embeddings, normalized by softmax to produce attention weights, which are then used to compute weighted sums of the input embeddings.
- **Scaled dot-product attention**: Adding trainable W_Q, W_K, W_V matrices and scaling by $\sqrt{d_k}$ gives the model the ability to learn what to attend to.
- **Causal masking**: Setting future attention scores to $-\infty$ before softmax ensures the model cannot look ahead, making it suitable for autoregressive language modeling.
- **Dropout**: Randomly zeroing attention weights during training provides regularization against over-reliance on specific attention patterns.
- **Multi-head attention**: Running multiple independent attention heads and concatenating their outputs allows the model to attend to different types of relationships simultaneously.

Chapter 6

Building the Transformer Block: Layer Norm, GELU, Feed-Forward, and Shortcut Connections

At the heart of every GPT-style model lies the transformer block—a carefully orchestrated sequence of normalization, attention, and transformation operations that, when stacked, enables language models to build rich, abstract representations of text. The transformer block is the fundamental building block of GPT and other large language model architectures. Before assembling a complete transformer block, we need to understand and implement each of its sub-components individually.

The five components of a transformer block are: masked multi-head attention, layer normalization, dropout, a feed-forward neural network, and the GELU activation function. This chapter focuses on the normalization and transformation machinery—the pieces that make deep transformer networks trainable—building on earlier coverage of layer normalization, GELU activation, feed-forward networks, and shortcut connections.

6.1 Layer Normalization

Why Normalize at All?

One reason normalization is necessary is a phenomenon called **internal covariate shift**: the input distribution to each layer changes across training iterations, making weight updates difficult and delaying convergence. When the distribution of activations shifts unpredictably from one batch to the next, downstream layers must constantly adapt to a moving target. Layer normalization addresses this by normalizing the output of every layer based on the mean and standard deviation of that layer's own activations.

Layer Norm vs. Batch Norm

- **Batch normalization** performs normalization for an entire batch, computing the mean and variance across the batch dimension. This means the normalization statistics depend on which other samples happen to be in the same mini-batch.

- **Layer normalization** normalizes along the feature dimension (columns) instead, making each sample's statistics independent of the rest of the batch.

The Math

Layer normalization normalizes each sample in a batch independently by subtracting the mean and dividing by the square root of the variance computed across the features for that sample.

Concretely, given a sample with feature values X_1, X_2, X_3, X_4, the mean is:

$$\mu = (X_1 + X_2 + X_3 + X_4)/4 = 2.15$$

The variance is:

$$\sigma^2 = \frac{1}{4}\left[(X_1 - \mu)^2 + (X_2 - \mu)^2 + (X_3 - \mu)^2 + (X_4 - \mu)^2\right]$$

And the normalized value for each feature is:

$$\hat{X} = \frac{X - \mu}{\sqrt{\sigma^2}}$$

In formula form, layer normalization applies:

$$y_{\text{normalized}} = \frac{y - \mu}{\sqrt{\text{variance}}}$$

independently to each sample in the batch. After normalization, the result has mean 0 and variance 1.

Scale and Shift Parameters

Raw normalization to zero mean and unit variance may not always be optimal—some layers may benefit from a different scale or offset. Scale and shift are trainable parameters in layer normalization that have the same dimension as the input and are applied to the normalized output. They allow the model to undo the normalization if that turns out to be optimal for a given layer, while still benefiting from the numerical stability that normalization provides during early training.

A Note on Bessel's Correction

When computing variance in PyTorch, you will encounter the unbiased flag. Bessel's correction is the use of $n - 1$ instead of n in the formula for sample variance and sample standard deviation, where n is the number of observations in a sample. In layer normalization we typically use the biased estimator (dividing by n), so unbiased=False is the correct setting.

Computing Layer Norm Step by Step in PyTorch

Let us walk through the computation manually before looking at the full class. First, create a small batch and pass it through a linear layer:

```
# Source:
torch.manual_seed(123)
batch_example = torch.randn(2, 5)
layer = nn.Sequential(nn.Linear(5, 6), nn.ReLU())
out = layer(batch_example)
print(out)
```

Now compute the mean across the feature dimension:

```
# Source:
mean = output.mean(dim=-1, keepdim=True)
```

Then compute the variance:

```
# Source:
variance = output.var(dim=-1, keepdim=True)
```

And finally normalize:

```
# Source:
normalized_output = (output - mean) / torch.sqrt(variance)
```

The dim=−1 argument is critical: it tells PyTorch to compute statistics along the last dimension (the feature dimension), which is exactly what layer normalization requires.

The LayerNorm Class

Layer normalization is implemented as a class that takes the output of a layer and applies normalization to it. Its forward method takes input with a certain number of rows and embedding-dimension columns.

```
# Source:
class LayerNorm(nn.Module):
    def __init__(self, emb_dim):
        super().__init__()
        self.eps = 1e-5
        self.scale = nn.Parameter(torch.ones(emb_dim))
        self.shift = nn.Parameter(torch.zeros(emb_dim))

    def forward(self, x):
        mean = x.mean(dim=-1, keepdim=True)
        var = x.var(dim=-1, keepdim=True, unbiased=False)
        norm_x = (x - mean) / torch.sqrt(var + self.eps)
        return self.scale * norm_x + self.shift
```

A few things to notice: self.eps = 1e−5 is a small epsilon value added to prevent division by zero, and scale and shift are initialized to ones and zeros respectively so that the layer initially performs pure normalization before learning any rescaling.

6.2 The GELU Activation Function

The Problem with ReLU

ReLU returns the input value for $x > 0$ and returns zero for $x < 0$. This hard cutoff creates the **dead neuron problem**: when a neuron's output becomes negative, ReLU sets it to zero and the neuron stops contributing to the learning process. Because the gradient of ReLU is exactly zero for all negative inputs, a neuron that consistently receives negative pre-activations will never receive a gradient signal and will never update its weights. In a deep network, this can silently eliminate large fractions of the model's capacity.

Introducing GELU

GELU (Gaussian Error Linear Unit) is a smooth activation function that is differentiable at $x = 0$, unlike ReLU. The exact definition is:

$$\text{GELU}(x) = x \cdot \Phi(x)$$

where $\Phi(x)$ is the cumulative distribution function of the standard Gaussian distribution.

The Approximation Used in GPT-2

In practice, GPT-2 uses a polynomial approximation:

$$\text{GELU}(x) \approx 0.5 \cdot x \cdot \left(1 + \tanh\left(\sqrt{\tfrac{2}{\pi}} \cdot (x + 0.044715 \cdot x^3)\right)\right)$$

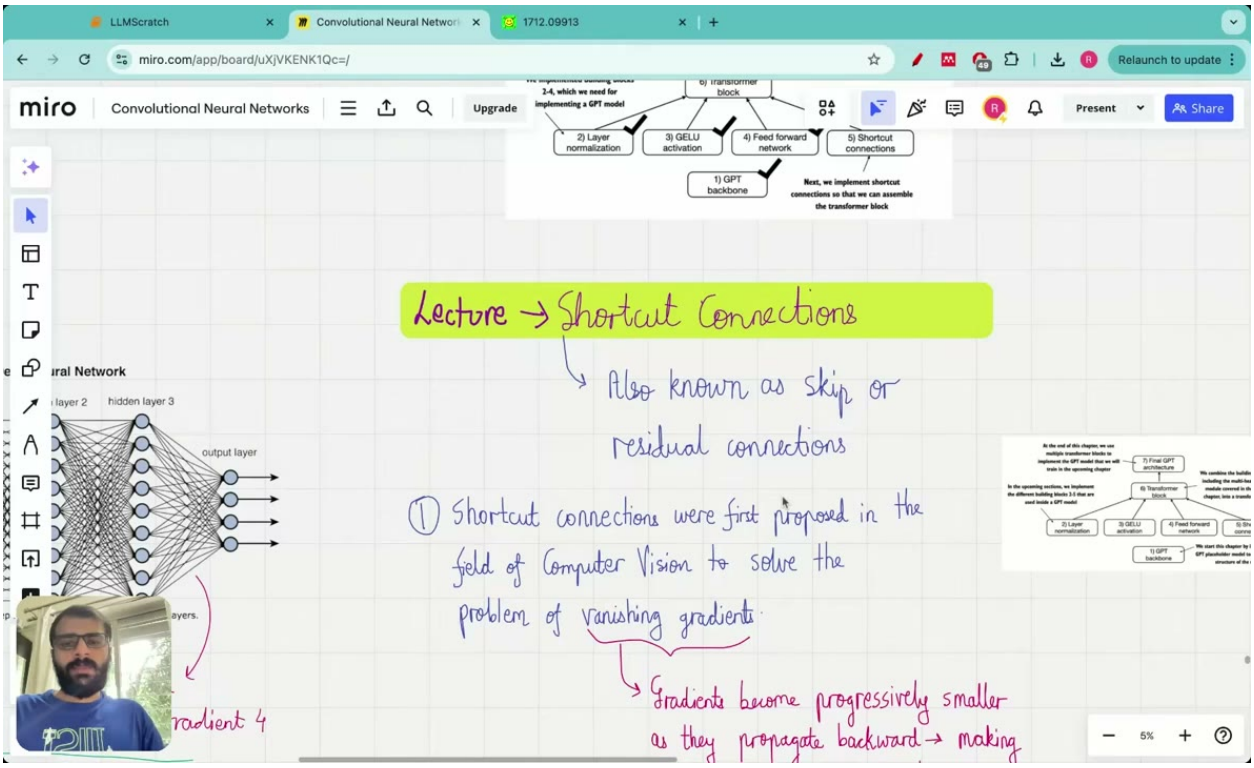
This is the approximation used in GPT-2 and is what we implement here.

Implementing GELU

```
# Source:
class GELU(nn.Module):
    def __init__(self):
        super().__init__()

    def forward(self, x):
        return 0.5 * x * (1 + torch.tanh(
            torch.sqrt(torch.tensor(2.0 / torch.pi)) *
            (x + 0.044715 * torch.pow(x, 3))
        ))
```

The structure mirrors the approximation formula exactly.



, showing ReLU's hard zero cutoff vs. GELU's smooth curve that allows small negative outputs near zero]

6.3 The Feed-Forward Network

Purpose and Position

The goal of the feed-forward sub-module is to serve as a component of the LLM transformer block. Within that block, the feed-forward network processes each element of the input sequence separately, without considering relationships with other elements. This contrasts with the self-attention block, which analyzes relationships between input elements and assigns attention scores based on how one element relates to others.

The Expansion–Contraction Pattern

The feed-forward neural network in a transformer block has an expansion layer followed by a compression layer, preserving input and output dimensions. Specifically, it consists of three stages:

1. An **expansion layer** that increases the dimension to 4 times the embedding dimension.
2. A **GELU activation** applied between the two linear layers.
3. A **contraction layer** that reduces back to the original embedding dimension.

The expansion layer's hidden dimension is four times the embedding dimension. For GPT-2's embedding dimension of 768, this means the network projects from 768 to 3072, applies GELU, and then projects back from 3072 to 768. Intuitively, expanding into a higher-dimensional space gives the network more room to represent complex, non-linear transformations of each token's

representation before compressing back down.

The feedforward module uses GELU between the expansion and contraction layers, and its output dimensions match its input dimensions.

Implementation

nn.Sequential is a PyTorch module that chains neural network layers together by forwarding outputs sequentially to inputs of subsequent modules, making it a natural fit for the linear flow of expansion → activation → contraction. The FF object is an instance of the feed-forward class that takes the embedding dimension from configuration and creates layers with GELU activation, initializing weights randomly.

The architecture is:

```
# Sources:,
nn.Sequential(
    nn.Linear(emb_dim, 4 * emb_dim),
    GELU(),
    nn.Linear(4 * emb_dim, emb_dim)
)
```

6.4 Shortcut Connections and the Vanishing Gradient Problem

The Vanishing Gradient Problem

To understand why shortcut connections matter, we first need to understand the problem they solve. The **vanishing gradient problem** occurs when gradients become extremely small or approach zero during backpropagation through deep layers, preventing weight updates and causing training stagnancy. In a deep network, gradients are computed by multiplying many partial derivatives together; if each factor is less than one, the product shrinks exponentially with depth. Shortcut connections help transformers solve this problem.

What Are Shortcut Connections?

Shortcut connections are also known as skip connections or residual connections. They are achieved by adding the output of one layer to the output of a later layer. In a transformer block diagram, they are represented by plus symbols with associated arrows that bypass the linear flow of data. More precisely, shortcut connections add the output of one layer to the output of the previous layer, creating an alternative path for gradient flow, and they can be added between any layers of the transformer block.

[Diagram: Residual block architecture showing input y_l flowing through both a transformation $f(y_l)$ and a skip connection, with outputs summed to produce y_{l+1}.]

The Residual Block Formula

In a residual block with shortcut connections, the output of layer $l + 1$ is:

$$y_{l+1} = f(y_l) + y_l$$

In code, this is expressed as:

```
x = x + layer_output
```

Why Shortcut Connections Preserve Gradients: The Math

The partial derivative of loss with respect to y_l can be expressed using the chain rule as:

$$\frac{\partial L}{\partial y_l} = \frac{\partial L}{\partial y_{l+1}} \cdot \frac{\partial y_{l+1}}{\partial y_l}$$

When $y_{l+1} = f(y_l) + y_l$, the second factor becomes:

$$\frac{\partial y_{l+1}}{\partial y_l} = \frac{\partial f(y_l)}{\partial y_l} + 1$$

The additive constant of 1 ensures that even if $\frac{\partial f(y_l)}{\partial y_l}$ is very small, the overall gradient is never smaller than $\frac{\partial L}{\partial y_{l+1}}$ itself. As a result, gradient magnitude remains stable across layers, preventing the vanishing gradient problem.

Demonstrating the Effect

To see the vanishing gradient problem in action and verify that shortcut connections fix it, we can build a simple deep network and compare gradients with and without skip connections. A deep neural network class takes layer_sizes as an argument specifying the number of neurons in each layer—for example, [3, 3, 3, 3, 1] means 5 layers with 3 neurons each and a final layer with 1 neuron.

The loss function is defined as the squared difference between the model output and the ground truth target:

$$\text{loss} = (Y - \text{target})^2$$

In code:

```
# Source:
loss = (model(x) - target) ** 2
loss.backward()
```

Without shortcut connections, gradients shrink dramatically in the early layers. With shortcut connections, they remain at a similar magnitude throughout the network.

6.5 Assembling the Transformer Block

Components Review

The GPT architecture has four key components: layer normalization, GELU activation, a feed-forward neural network, and shortcut connections. The full transformer block stacking order is:

layer normalization → multi-head attention → dropout → shortcut connection → layer normalization → feed-forw

Pre-Layer Norm vs. Post-Layer Norm

The original transformer model applied layer normalization *after* the self-attention and feed-forward sub-layers, a pattern called **post-layer norm**. Modern transformer implementations use **pre-layer norm** instead, applying normalization *before* the multi-head attention mechanism and *before* the feed-forward neural network.

Why does this matter? Because the shortcut connection adds the un-normalized input directly to the output, the residual stream accumulates values across many layers. Normalizing *before* each operation keeps the inputs to each sub-layer well-conditioned, improving training stability. Layer normalization is therefore implemented twice within the transformer block: once before multi-head attention and once before the feed-forward network.

The Transformer Block Components in Detail

Each named component of the transformer block class plays a specific role:

- **ATT**: An instance of the multi-head attention class that takes embedding vectors and converts them into context vectors, with input and output dimensions equal to the embedding dimension. It multiplies an input matrix X with trainable query, key, and value matrices to produce context vectors.
- **FF**: An instance of the feed-forward class that takes the embedding dimension from configuration and creates layers with GELU activation, initializing weights randomly.
- **norm_one** and **norm_two**: norm_one is the first layer normalization layer, applied before multi-head attention; norm_two is the second, applied before the feed-forward network.
- **drop_shortcut**: A dropout layer (torch.nn.Dropout) that randomly turns off some layer outputs during training to improve generalization and prevent overfitting.

In a transformer block, each element of an input sequence is represented by a fixed-size vector equal to the embedding dimension.

The Transformer Block Class

A transformer block class in PyTorch includes a multi-head attention mechanism and a feed-forward neural network, and implements shortcut connections that add the input of the block to the output of each component. A dropout layer and residual shortcut connection are applied after the feed-forward network to prevent vanishing gradients.

Based on the architectural description, the forward pass follows this logic:

```
# Sources:,,
# First sub-block: multi-head attention
shortcut = x
x = norm_one(x)
x = ATT(x)
x = drop_shortcut(x)
x = x + shortcut  # residual connection

# Second sub-block: feed-forward
shortcut = x
x = norm_two(x)
x = FF(x)
x = drop_shortcut(x)
x = x + shortcut  # residual connection
```

This structure directly implements the stacking order described in, with pre-layer norm and shortcut connections after each sub-block.

6.6 Putting It All Together: The GPT Architecture

The GPT architecture consists of four main components: the GPT backbone, layer normalization, GELU activation function, and feed-forward neural network. Each transformer block processes the full sequence and passes its output to the next block; stacking many such blocks allows the model to build increasingly abstract representations of the input.

6.7 Summary

This chapter built every sub-component of the transformer block from scratch:

1. **Layer normalization** normalizes each sample independently along the feature dimension, using trainable scale and shift parameters and a small epsilon for numerical stability. It addresses internal covariate shift and is applied twice per transformer block.
2. **GELU activation** provides a smooth, differentiable alternative to ReLU that avoids the dead neuron problem. It is approximated by the formula $0.5 \cdot x \cdot (1 + \tanh(\sqrt{2/\pi} \cdot (x + 0.044715 \cdot x^3)))$.
3. **The feed-forward network** expands the embedding dimension by a factor of 4, applies GELU, and contracts back to the original dimension, processing each sequence position independently.
4. **Shortcut connections** add the input of each sub-block to its output, contributing an additive term of 1 to the gradient computation that prevents gradients from vanishing in deep networks.
5. **The transformer block** assembles these components in pre-layer-norm order—normalize → attend → dropout → add → normalize → feed-forward → dropout → add—and can be stacked repeatedly to form a complete GPT model.

Chapter 7

Assembling the Full GPT Model and Text Generation

We will now assemble all previously built components into a single, coherent GPTModel class, trace a tensor through the entire forward pass, count the model's parameters, and implement the autoregressive text-generation loop that turns raw logits into readable text.

By the end of this chapter, you will have a working 124-million-parameter GPT-2 model that can run inference on a laptop and produce text from a prompt — even with randomly initialized weights. Training comes later; here the focus is on architecture and inference.

7.1 The Big Picture: What the GPT Pipeline Does

Text generation in a large language model is an **autoregressive** process: the model generates one token at a time, and each newly generated token is appended to the input context before the next prediction is made. This continues until a user-specified maximum number of new tokens has been produced.

Given a sequence of token IDs, the model executes the following steps:

1. Converts token IDs into dense token embeddings.
2. Adds positional embeddings to encode each token's position in the sequence.
3. Applies dropout to the combined input embeddings.
4. Passes the result through a stack of transformer blocks.
5. Applies a final layer normalization.
6. Projects the normalized embeddings through an output head to produce logits over the vocabulary.

Concisely: input sentence → token IDs → token embeddings → positional embeddings → input embeddings → dropout → transformer blocks → layer normalization → output head → output tensor. The output is a tensor of logits, which we will later convert into probabilities and then into token IDs.

7.2 The GPT Configuration Object

Rather than hard-coding hyperparameters throughout the model, we collect them in a single dictionary:

```
GPT_CONFIG_124M = {
    "vocab_size": 50257,          # Vocabulary size
    "context_length": 1024,       # Context length
    "emb_dim": 768,               # Embedding dimension
    "n_heads": 12,                # Number of attention heads
    "n_layers": 12,               # Number of layers
    "drop_rate": 0.1,             # Dropout rate
    "qkv_bias": False             # Query-Key-Value bias
}
```

Here is what each field means in the context of the full model:

- **vocab_size: 50257** — Each token ID is an integer in the range [0, 50256].
- **context_length: 1024** — The maximum number of tokens the model can process in a single forward pass; also the size of the positional embedding table.
- **emb_dim: 768** — Every token is represented as a 768-dimensional vector. Each token ID in GPT-2 is converted into a vector of 768 dimensions, and the output dimensions are matched to those of the input token embeddings.
- **n_heads: 12** — The number of separate query, key, and value matrix sets created in multi-head attention. Multi-head attention is a key architectural component that enables coherent and meaningful outputs in modern GPT models.
- **n_layers: 12** — The number of transformer blocks stacked in sequence, chained together using nn.Sequential.
- **drop_rate: 0.1** — The fraction of elements randomly zeroed during dropout.
- **qkv_bias: False** — Whether to include bias terms in the query, key, and value projections of attention.

7.3 A Dummy Model First: Validating the Architecture

Before filling in the details of each sub-module, it is useful to build a skeleton model with dummy internals. This lets us verify that tensor shapes flow correctly through the pipeline before worrying about the internals of each component:

```
class DummyGPTModel(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
        self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
        self.drop_emb = nn.Dropout(cfg["drop_rate"])
        self.trf_blocks = nn.Sequential(
            *[DummyTransformerBlock(cfg) for _ in range(cfg["n_layers"])])
        self.final_norm = DummyLayerNorm(cfg["emb_dim"])
        self.out_head = nn.Linear(
```

```
            cfg["emb_dim"], cfg["vocab_size"], bias=False
        )

    def forward(self, in_idx):
        batch_size, seq_len = in_idx.shape
        tok_embeds = self.tok_emb(in_idx)
        pos_embeds = self.pos_emb(torch.arange(seq_len, device=in_idx.device))
        x = tok_embeds + pos_embeds
        x = self.drop_emb(x)
        x = self.trf_blocks(x)
        x = self.final_norm(x)
        logits = self.out_head(x)
        return logits
```

Even with dummy internals, this code captures the complete forward-pass logic. The dummy model's forward method takes an input and outputs the next-word prediction. Token IDs are converted to token embeddings before being passed through the model, and those token embeddings are then added to positional embeddings. The transformer block is the most important part of the large language model architecture, consisting of multiple aspects linked together, and it is the key component of the entire GPT model.

7.4 Building the Real GPT Model

With the structural skeleton validated, we now describe the real GPTModel that replaces each dummy sub-module with its full implementation.

Token and Positional Embeddings

Token embedding: An nn.Embedding layer maps each token ID to a dense vector. In GPT-2, this embedding layer is trained to learn token representations that capture semantic meaning.

Positional embedding: A second nn.Embedding layer encodes position information. A positional embedding is a vector representation of the position of a token within a sequence:

```
self.pos_emb = nn.Embedding(context_length, embedding_dimension)
```

During the forward pass, position indices are generated to look up in this table:

```
pos = torch.arange(sequence_length)
```

This creates a tensor [0, 1, 2, ..., sequence_length−1] to index into the positional embedding matrix.

Combining the two: The input embedding for each token is the sum of its token embedding and its positional embedding — token embeddings and positional embeddings are summed to produce input embeddings for each token.

Dropout on Input Embeddings

Immediately after combining the embeddings, a dropout layer is applied. Dropout randomly sets some elements of every input embedding to zero, and the combined embeddings pass through this layer before entering the transformer stack. This regularization step helps prevent overfitting during training.

The Transformer Block Stack

The preprocessing pipeline before the transformer block consists of tokenization, token embedding, positional embedding, input-embedding computation, and dropout. After that preprocessing, the tensor enters the transformer block stack.

The GPT model forward pass chains 12 transformer blocks using nn.Sequential, then applies layer normalization, and finally outputs logits. Inside each transformer block, the feed-forward neural network uses GELU activation. Masked multi-head attention converts embedding vectors into context vectors that capture both semantic meaning and attention relationships between tokens; a context vector is an embedding that encodes both the semantic meaning of a token and how much attention should be given to every other token in the sequence. Shortcut connections (residual connections) add the output of each layer back to its input, providing an alternative gradient-flow path and preventing the vanishing gradient problem.

Layer Normalization

Layer normalization normalizes token embeddings by setting the mean to zero and the variance to one for each token independently. This normalization is applied both within each transformer block and once more after the final block.

The Output Head

The output head is a neural network at the final stage of the GPT model that transforms token embeddings into logits for vocabulary prediction. It is the only stage in the model where the tensor dimensions change — from (batch_size, num_tokens, 768) to (batch_size, num_tokens, 50257). The resulting logits matrix is the output produced by passing token embeddings through this linear layer. Logits are the final output matrices from a transformer model that contain unnormalized scores for each token in the vocabulary at each position in the sequence.

7.5 The Forward Pass in Detail

Let's trace a concrete tensor through the model to make the shapes tangible.

The GPT model receives an input tensor of shape (batch_size, sequence_length) containing token IDs. For a batch of 2 sequences each of length 4:

1. **Token embedding lookup:** Shape becomes (2, 4, 768).
2. **Positional embedding lookup:** Shape is (4, 768), broadcast-added to the token embeddings.
3. **Sum:** Shape remains (2, 4, 768).
4. **Dropout:** Shape unchanged, (2, 4, 768).

5. **12 transformer blocks:** Shape unchanged throughout, (2, 4, 768).
6. **Layer normalization:** Shape unchanged, (2, 4, 768).
7. **Output head (linear):** Shape becomes (2, 4, 50257).

The output of the GPT model is therefore logits with shape (batch_size, sequence_length, vocabulary_size). The forward method takes a batch of input tokens, computes their embeddings, applies positional embeddings, passes the sequence through transformer blocks, normalizes the final output, and computes logits representing the next token's unnormalized probabilities. Notably, the core GPT model implementation can be expressed in approximately 8 lines of code — a testament to how cleanly PyTorch's module system composes the sub-components built in earlier chapters.

7.6 Parameter Counting and Weight Tying

Counting Parameters

The standard idiom for counting parameters is:

```
sum(p.numel() for p in model.parameters())
```

Iterating over model.parameters() and summing p.numel() for each tensor gives the total parameter count.

The Token Embedding and Output Layer

Both the token embedding layer and the output layer have shape (50257, 768), meaning they contain the same number of parameters.

Weight Tying

The original GPT-2 architecture exploits this symmetry through **weight tying**: it reuses the weights from the token embedding layer in the output layer. This reduces the memory footprint from 163 million to 124 million parameters and lowers computational complexity — when the output layer parameters are excluded from the count, the total becomes exactly 124 million, matching the original GPT-2 model size.

It is worth noting, however, that using separate token embedding and output layers yields better training and model performance than weight tying. The implementation described in this book therefore uses separate layers for clarity and performance; the 124 M figure reflects the original GPT-2 design choice.

7.7 Instantiating the Model

A GPT model instance is created by passing a configuration object to the constructor:

```
model = GPTModel(GPT_CONFIG_124M)
```

The full GPT architecture has been implemented and the model instance is initialized with random weights. Even at this stage, the model can accept an input, run inference through the 124-million-parameter architecture, and produce output on a laptop. The main task of the model is next-token prediction: it outputs logits that can be decoded to predict the next token, which will subsequently be converted into tokens and text.

7.8 From Logits to Text: The Generation Pipeline

The goal is to convert the model's output tensor into predictions of the next word. This section implements the full autoregressive generation loop.

The Five-Step Decoding Algorithm

The text generation algorithm consists of five sequential steps:

1. Examine the output tensor.
2. Extract the last position's vector (logits for the next token).
3. Convert logits to probabilities via softmax.
4. Identify the index of the largest probability value.
5. Append the resulting token ID to the previous input, then repeat until max_new_tokens is reached.

Figure 7.1: Autoregressive generation loop: at each step, the model extracts last-position logits, applies softmax to obtain probabilities, uses argmax to select the token ID, appends it to the sequence, and feeds the updated sequence back into the model for the next iteration.

Implementing `generate_text_simple`

The generate_text_simple function takes a model instance, input token indices (idx), a maximum number of new tokens to generate, and a context size as arguments. The key steps are as follows.

First, crop the running sequence to the last context_size tokens so it never exceeds the positional embedding table:

```
idx_cond = idx[:, -context_size:]
```

Then run the forward pass with gradients disabled to save memory and computation:

```
with torch.no_grad():
    logits = model(idx_cond)
```

The model produces logits for every position, but only the last position matters for the next-token prediction:

```
logits = logits[:, -1, :]
```

Apply softmax to obtain a probability distribution:

```
probas = torch.softmax(logits, dim=-1)
```

Select the token with the highest probability (greedy decoding):

```
idx_next = torch.argmax(probas, dim=-1, keepdim=True)
```

Finally, append the new token to the running sequence:

```
idx = torch.cat((idx, idx_next), dim=1)
```

The Complete `generate_text_simple` Function

Placing these steps inside a loop gives the full generation function. Text generation proceeds by repeating the token-prediction process until the user-specified maximum number of new tokens is reached:

```
def generate_text_simple(model, idx, max_new_tokens, context_size):
    for _ in range(max_new_tokens):
        # Step 1: crop context to fit positional embedding table
        idx_cond = idx[:, -context_size:]

        # Step 2: forward pass, no gradients needed
        with torch.no_grad():
            logits = model(idx_cond)

        # Step 3: focus on the last time step
        logits = logits[:, -1, :]          # (batch, vocab_size)

        # Step 4: logits → probabilities
        probas = torch.softmax(logits, dim=-1)

        # Step 5: greedy selection
        idx_next = torch.argmax(probas, dim=-1, keepdim=True)

        # Step 6: append to running sequence
        idx = torch.cat((idx, idx_next), dim=1)

    return idx
```

Running the Generator

The generate_text_simple function is called with parameters: model, inputs, max_new_tokens=6, and context_size=1024. Starting from the input 'hello i am' with max_new_tokens=6, the GPT model generates the complete output 'hello i am a model ready to help'.

7.9 Greedy Decoding and Its Limitations

The current implementation always selects the single most probable token at each step — a strategy known as greedy decoding. While simple and deterministic, it tends to produce repetitive, bland text because it never explores lower-probability but potentially more interesting continuations. In GPT training, sampling techniques modify the softmax output so the model does not always select the most likely token, introducing variability and creativity in generated text.

7.10 Putting It All Together: A Complete Walkthrough

One final end-to-end walkthrough consolidates everything covered in this chapter.

1. **Configuration.** Define GPT_CONFIG_124M with vocab_size=50257, context_length=1024, emb_dim=768, n_heads=12, n_layers=12, drop_rate=0.1, qkv_bias=False.
2. **Model instantiation.** model = GPTModel(GPT_CONFIG_124M) creates a model with randomly initialized weights.
3. **Input preparation.** A tokenizer converts the input string into token IDs. The model receives a tensor of shape (batch_size, sequence_length).
4. **Forward pass.** The model maps the input through token embeddings, positional embeddings, dropout, 12 transformer blocks, layer normalization, and the output head, producing an output of shape (batch_size, sequence_length, 50257).
5. **Generation loop.** generate_text_simple runs the forward pass repeatedly, each time extracting the last-position logits, applying softmax, selecting the argmax token, and appending it to the sequence.

The implementation has reached the stage where, given a text input, the model can predict output tokens. All sub-modules of the GPT architecture were implemented and coded from scratch.

7.11 What Comes Next

The model currently outputs logits that will be converted into tokens and text in subsequent steps. The next stage of the series focuses on training the GPT-2 model with 124 million parameters. Once trained, the same generate_text_simple function — or a more sophisticated sampling variant — will produce coherent, meaningful text rather than random token sequences.

Chapter 8

Pre-Training: Loss Functions, Training Loop, and Evaluation

Knowing whether a language model is actually learning requires more than watching it generate text — it requires measuring loss precisely and watching that number fall. This chapter builds that measurement infrastructure from the ground up: we will calculate training loss and validation loss on an actual dataset, observe how those numbers evolve over time, and learn to recognize the warning signs of overfitting. By the end you will have a complete, working pre-training loop of roughly 15 to 20 lines of Python.

8.1 The Language Modeling Objective

Autoregressive Prediction

Large language models are autoregressive models where input-output pairs are constructed from the text itself without pre-labeling. Context size is the maximum number of tokens an LLM can see before it predicts the next token. Given a sequence of tokens, the model receives a window of up to that many tokens as input and must predict what comes next. Targets are the input token sequence shifted by one position, creating input-output pairs for training.

More formally, X represents the input tensor and Y represents the target tensor (the actual desired output values) in LLM input-output pair construction. Input-target pairs are created from training data by pairing inputs with their corresponding target outputs, and the target output is the expected output sequence that corresponds to a given input sequence, which the LLM should learn to predict during training.

From Raw Text to Batches

A dataloader loops over the entire dataset and creates input-output pairs based on a specified context size and stride. Two parameters govern this process:

- **max_length** specifies the context size — the number of tokens to consider at once.
- **stride** specifies how many steps to advance before creating the next input-output pair.

Stride is the step size used to move through the dataset when constructing consecutive input-output pairs. The PyTorch DataLoader is useful for processing data in batches and makes batch processing more convenient. Two additional parameters control how batches are assembled:

- **shuffle** shuffles the dataset order when batches are created, which is sometimes useful for generalization.
- **drop_last** drops the last batch if its size is smaller than the specified batch size.

Loss computation is then applied to the entire dataset, split into training and validation portions.

8.2 A Taxonomy of Loss Functions

Regression Losses

Let y_i denote the true value and $f(x_i)$ the model's prediction for the i-th example.

Mean Bias Error simply averages the signed errors:

$$\mathcal{L}_{MBE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - f(x_i))$$

Mean Absolute Error removes the cancellation problem by taking absolute values:

$$\mathcal{L}_{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - f(x_i)|$$

Mean Squared Error penalizes large errors more heavily by squaring the residuals:

$$\mathcal{L}_{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - f(x_i))^2$$

Root Mean Squared Error brings the loss back to the same units as the target:

$$\mathcal{L}_{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - f(x_i))^2}$$

Huber Loss is a hybrid that behaves like MSE for small errors and like MAE for large ones, making it robust to outliers:

$$\mathcal{L}_{Huber} = \begin{cases} \frac{(y_i - f(x_i))^2}{2} & \text{if } |y_i - f(x_i)| \leq \delta \\ \delta|y_i - f(x_i)| - \frac{\delta}{2} & \text{otherwise} \end{cases}$$

Log-Cosh Loss is another smooth approximation to MAE:

$$\mathcal{L}_{LogCosh} = \frac{1}{N}\sum_{i=1}^{N}\log(\cosh(f(x_i) - y_i))$$

Classification Losses

Binary Cross-Entropy is used for two-class problems:

$$\mathcal{L}_{BCE} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(p_i) + (1 - y_i)\log(1 - p_i)]$$

Hinge Loss is the classic support-vector-machine objective:

$$\mathcal{L}_{Hinge} = \max(0, 1 - (f(x) \cdot y))$$

Kullback-Leibler Divergence measures how one probability distribution differs from another:

$$\mathcal{L}_{KL} = -\sum_{i=1}^{N} y_i \cdot \log\left(\frac{y_i}{f(x_i)}\right)$$

Cross-Entropy Loss: The Language Modeling Standard

For multi-class classification — which is exactly what next-token prediction is — the standard choice is **Cross-Entropy Loss**:

$$\mathcal{L}_{CE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{M} y_{i,c}\log(f(x_i)_c)$$

Cross-entropy loss measures the difference between two probability distributions, specifically between predicted logits and target indices in language model training. The goal of LLM training is to minimize this loss so that predicted probabilities for target tokens approach one.

8.3 Cross-Entropy in Depth

From Logits to Probabilities

The transformer block is the main engine of the GPT architecture, transforming input embedding vectors into context vectors. Context vectors are richer representations than input embedding vectors because they encode both the semantic meaning of a token and information about how that token relates to other tokens in the sequence.

After the final transformer block, a linear projection maps each context vector to a vector of raw scores over the vocabulary. These raw scores are called logits — the raw output values produced by the GPT model architecture before normalization into probabilities. A logit tensor is then converted into a softmax tensor, yielding a probability tensor.

Negative Log-Likelihood

Cross-entropy loss is computed by taking the logarithm of the target probabilities, summing them, taking the mean, and then negating the result. This is equivalent to **negative log-likelihood (NLL)** — the negative of the logarithm of the target probabilities. More precisely, it equals the negative mean of log probabilities at the target indices:

$$-\frac{1}{N}\sum \log(p_i)$$

where p_i are the probabilities at the target indices.

The goal of training an LLM is to minimize this negative log-likelihood toward zero. Cross-entropy loss (negative log-likelihood) is therefore the natural measure of the difference between predicted probabilities and target token indices.

Computing Loss in PyTorch

PyTorch's torch.nn.functional.cross_entropy expects logits (not probabilities) as the first argument and integer class indices as the second:

```
torch.nn.functional.cross_entropy(logit_tensor, target_tensor)
```

When working with a batch of sequences, both tensors must be flattened before the call. For example, given logits of shape (2, 3, 50257) and targets of shape (2, 3):

```
logits_flat = logits.view(-1, 50257)  # Flatten first two dimensions from (2, 3, 5
targets_flat = targets.view(-1)       # Flatten from (2, 3) to (6,)
loss = torch.nn.functional.cross_entropy(logits_flat, targets_flat)
```

The same pattern appears in a more compact form when working with a full training batch:

```
torch.nn.functional.cross_entropy(logits_flat, targets_flat)
```

And in the context of a complete forward pass over a batch:

```
nn.functional.cross_entropy(flattened_logits, flattened_target_batch)
```

To find the predicted token at each position — useful for quick accuracy checks — take the argmax over the vocabulary dimension. torch.argmax(input, dim=−1) returns the indices of the maximum value along the last dimension (columns).

8.4 Perplexity: An Interpretable Loss Metric

Raw cross-entropy values are hard to interpret intuitively. Perplexity addresses this by measuring how well the probability distribution predicted by the model matches the actual distribution of words in the dataset. It is calculated as e raised to the loss value:

$$\text{Perplexity} = \exp(\text{loss})$$

A well-trained language model on English text typically achieves perplexity in the tens to low hundreds, depending on the domain and model size.

8.5 Utility Functions: Text Token IDs

Two helper functions bridge raw text and the integer representations the model operates on. The text_to_token_ids function converts text input into token IDs using the tiktoken tokenizer; tiktoken provides a byte-pair encoder that operates at the character and sub-word level. The reverse operation, token_ids_to_text, converts token IDs back into text.

8.6 Model Parameter Accounting

Understanding how many parameters a model contains helps set expectations about memory requirements and training time. The main contributors are:

- **Token embedding parameters** = vocabulary size × embedding dimension
- **Positional embedding parameters** = context size × embedding dimension
- **Final layer parameters** = embedding dimension × vocabulary size

Embedding dimension is the size of the vector space into which token embeddings are projected to capture semantic meaning. The number of attention heads specifies how many self-attention mechanism blocks exist within one transformer block.

8.7 The Pre-Training Algorithm

With the loss function defined, we can describe the pre-training algorithm at a high level.

LLM pre-training involves minimizing the loss function to make model outputs as close as possible to target value tensors. Each iteration over a batch consists of three steps:

1. **Forward pass** — compute logits and then cross-entropy loss for the entire batch.
2. **Backward pass** — call loss.backward() to compute gradients. This is the most important step because it calculates the gradients of the loss with respect to every parameter. Back propagation is used to calculate these loss gradients.
3. **Parameter update** — apply the gradient descent rule:

$$p_{\text{new}} = p_{\text{old}} - \text{step\_size} \times \text{loss\_gradient}$$

The pre-training workflow is fully differentiable, allowing back propagation to compute partial derivatives of the loss with respect to all model parameters.

Epochs and Batches

One epoch is a single pass through the entire training set. The training set is divided into batches, and the pre-training loop processes one batch per iteration. After exhausting all batches, the loop repeats for multiple epochs. Structurally, the pretraining loop has two nested loops: an outer loop iterating over epochs and an inner loop iterating over batches in the training dataset.

8.8 Implementing the Training Loop

The training pipeline for LLMs begins with text generation and text evaluation. In this context, "target" refers to the true values that the model should predict. The core of the inner loop is a single call:

```
loss.backward()
```

This triggers PyTorch's autograd engine to walk backward through the computation graph and accumulate gradients in each parameter's .grad attribute. The optimizer then reads those gradients and updates the parameters.

The resulting training loop is approximately 15 to 20 lines of Python, with loss.backward() and optimizer.step() as the core gradient update mechanism. A training loop has been implemented where the loss function is minimized and the large language model learns.

An evaluation step is performed during training to monitor training loss and validation loss as the model trains, and an LLM training function will be defined to implement backpropagation and minimize both losses.

8.9 Evaluating the Model During Training

The `evaluate_model` Function

The evaluate_model function calculates loss for both the training loader and validation loader over the entire dataset. During evaluation, the model is set to evaluation mode with gradient tracking disabled and dropout disabled, so loss is computed without any gradient updates. This matters for two reasons: evaluation mode disables stochastic components such as dropout, giving deterministic outputs; and wrapping the evaluation in torch.no_grad() tells PyTorch not to build a computation graph, saving memory and speeding up the forward pass.

The function returns both training loss and validation loss as scalar outputs. Having these two values in hand enables backpropagation in the next step.

Recognizing Overfitting

Divergence between training and validation loss — with validation loss growing substantially larger than training loss — indicates that the model is overfitting to the training data.

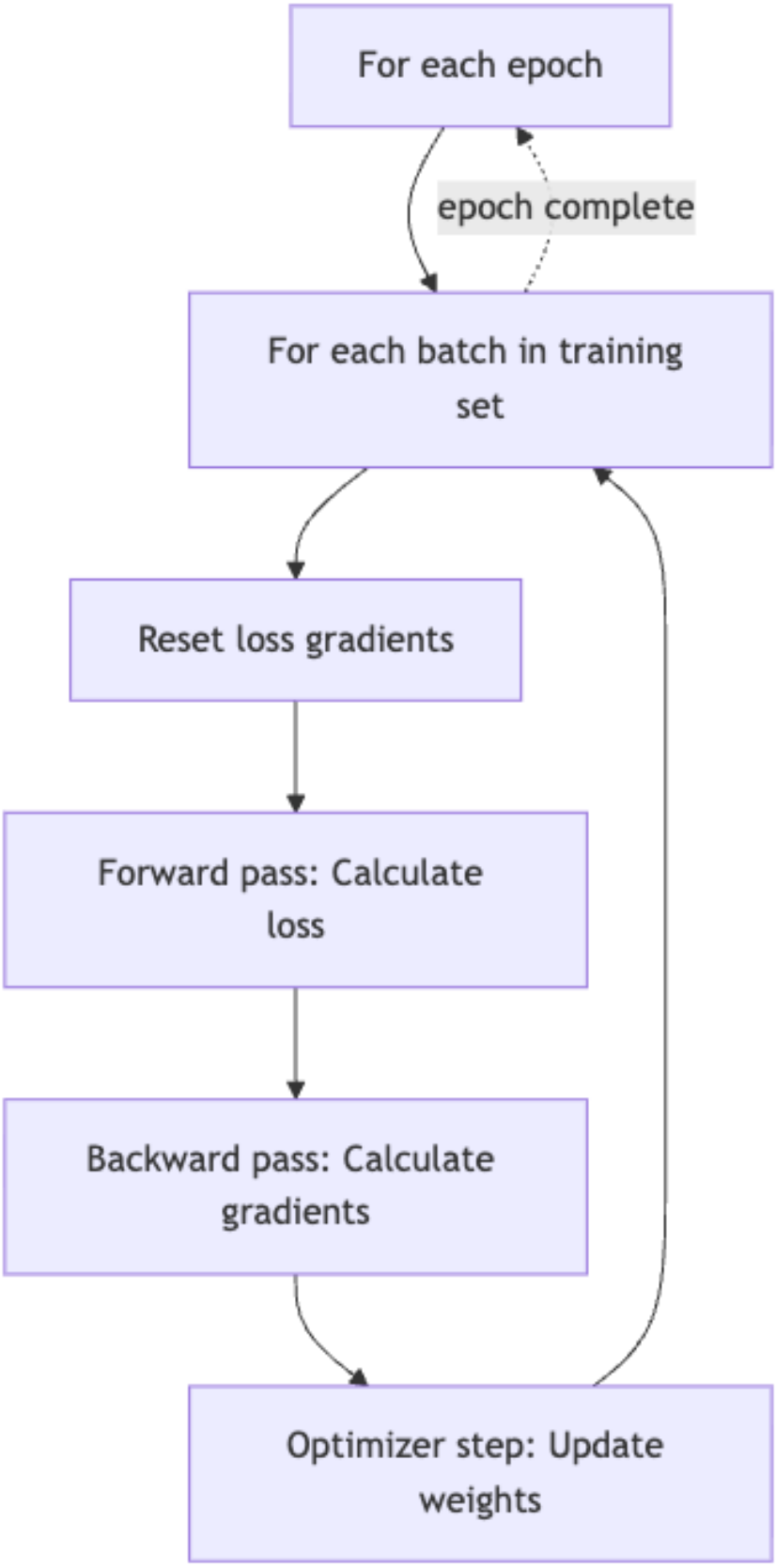


Figure 8.1: Training loop structure showing nested epoch and batch loops with forward pass, loss calculation, backward pass, and optimizer step.

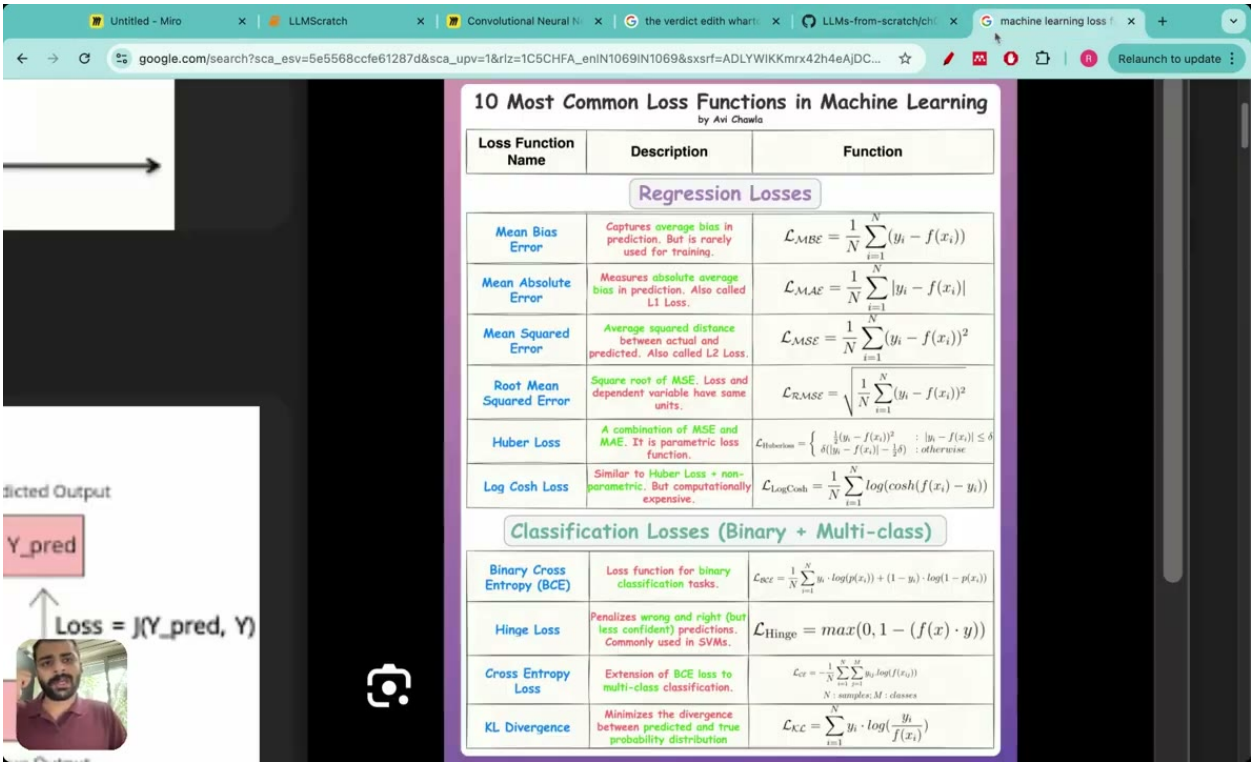


Figure 8.2: Overview of 10 common loss functions in machine learning, categorized into regression losses (Mean Bias Error, Mean Absolute Error, Mean Squared Error, Root Mean Squared Error, Huber Loss, Log Cosh Loss) and classification losses (Binary Cross Entropy, Hinge Loss, Cross Entropy Loss, KL Divergence), with mathematical formulas and descriptions.

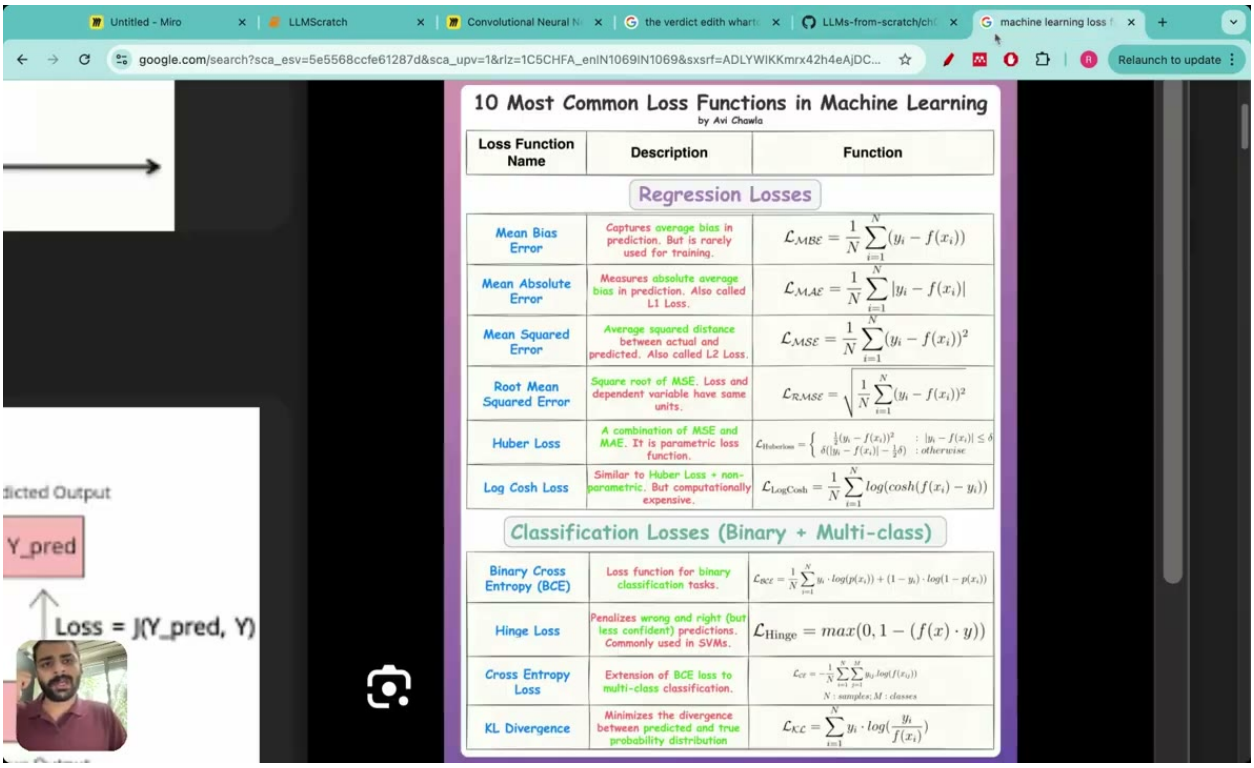


Figure 8.3: Training and validation loss curves over training steps, illustrating overfitting as validation loss diverges upward from training loss.

8.10 Temperature Scaling and Text Generation During Training

Even before the loss numbers tell a clear story, you can often see the model's outputs shift from random noise to recognizable English. Temperature scaling is a decoding strategy used to control the randomness of those outputs.

8.11 Putting It All Together: The Training Pipeline

The code developed in this chapter is generalizable and can be applied to custom datasets for pre-training. The complete pipeline proceeds as follows:

1. **Tokenize the corpus.** Use tiktoken's byte-pair encoder to convert raw text into integer token IDs.
2. **Build the DataLoader.** Wrap the token IDs in a dataset class that produces (input, target) pairs using a sliding window of size max_length and step size stride. Split the data into training and validation portions.
3. **Run the training loop.** For each epoch, iterate over all batches:
   - Compute logits via the forward pass.
   - Flatten logits and targets, then compute cross-entropy loss.
   - Call loss.backward() to compute gradients.

- Call optimizer.step() to update parameters using $p_{\text{new}} = p_{\text{old}} - \text{step\_size} \times \text{loss\_gradient}$.

4. **Evaluate periodically.** Call evaluate_model to compute training and validation loss over the full dataset. Log both values so you can plot them later.

Figure 8.4: End-to-end machine learning pipeline: from raw text through tokenization, data loading, and an iterative training loop (forward pass, loss computation, backpropagation, weight updates) to evaluation and sample generation.

8.12 Worked Example: Loss Computation Step by Step

To make the mechanics concrete, consider tracing through a single forward pass and loss computation for a small batch. After the forward pass produces logits of shape (2, 3, 50257) and targets of shape (2, 3), both tensors are flattened before being passed to PyTorch's cross-entropy function:

```
logits_flat = logits.view(-1, 50257)  # Flatten first two dimensions from (2, 3, 5
targets_flat = targets.view(-1)       # Flatten from (2, 3) to (6,)
loss = torch.nn.functional.cross_entropy(logits_flat, targets_flat)
```

To convert the resulting loss to perplexity:

$$\text{Perplexity} = \exp(\text{loss})$$

As training progresses, both numbers should fall substantially.

8.13 What Comes Next

The next lecture will cover LLM pre-training in full. The code developed here is generalizable and can be applied to custom datasets, and once training and validation loss are in hand, backpropagation can proceed in the next step. Fine-tuning, instruction tuning, and reinforcement learning from human feedback all build on the same basic machinery: a differentiable loss function, a backward pass, and a parameter update rule.

8.14 Summary

This chapter covered the full arc from raw text to a working training loop:

- **Loss functions**: We surveyed regression and classification losses, then focused on cross-entropy as the standard for language modeling.
- **Cross-entropy mechanics**: Logits are produced by the model, converted to probabilities via softmax, and the negative log-likelihood of the target tokens is minimized.

- **PyTorch implementation**: Flatten the logits and targets, then call torch.nn.functional.cross_entropy.
- **Perplexity**: An interpretable metric computed as exp(loss).
- **Training loop structure**: Two nested loops (epochs and batches), with forward pass, loss.backward(), and optimizer step inside the inner loop.
- **Evaluation**: The evaluate_model function computes loss on both training and validation sets, with the model in eval mode and gradients disabled.
- **Overfitting**: Divergence between training and validation loss is the key diagnostic signal.

Chapter 9

Decoding Strategies, Model Persistence, and Loading Pre-Trained GPT-2 Weights

By this point in the book, you have a working GPT model that can generate text—but the output is probably not very good. It may repeat itself, sound robotic, or be outright incoherent. This chapter addresses three interconnected topics that move the model from "technically functional" to "practically useful."

First, we examine *decoding strategies*: the algorithms that decide which token to pick at each generation step. The naive approach has real problems, and we will replace it with temperature scaling and top-k sampling. Second, we cover *model persistence*: how to save and reload both model weights and optimizer state so that long training runs are never lost. Third, we put everything together by loading OpenAI's publicly released GPT-2 weights into our custom model class and verifying that the result produces coherent text.

9.1 Decoding Strategies: Controlling Randomness in Generation

The simplest possible decoding strategy is *greedy decoding*: at every step, pick the token with the highest probability. In PyTorch terms, this is a call to `torch.argmax`. Greedy decoding is deterministic—given the same prompt, you always get the same output—and it tends to produce repetitive, low-diversity text. More importantly for training, a model that always picks the maximum-probability token can effectively memorize passages from its training data.

More generally, LLM decoding strategies are techniques used to control randomness and improve quality when generating the next token in sequence. The naive strategy selects the token corresponding to the largest probability score among all tokens in the vocabulary. The two strategies we introduce here—temperature scaling and top-k sampling—address these shortcomings directly.

Temperature Scaling

Temperature scaling is a technique used in large language models to control randomness and diversity in generated text. Concretely, it means dividing the logits by a temperature parameter T before applying softmax. Temperature controls the entropy of the probability distribution over tokens.

Intuitively, dividing by a small T (say, 0.1) makes the largest logit relatively much larger than the others, sharpening the distribution toward a near-deterministic choice. Dividing by a large T (say, 5.0) compresses the differences between logits, flattening the distribution so that many tokens have similar probability. Temperature scaling therefore adjusts the sharpness of the distribution to either sharpen or flatten it, helping prevent overfitting through multinomial sampling.

Low-temperature scaling (sharper distribution):

```
# Source:
next_token_logits_2 = next_token_logits / 0.1
probabilities = softmax(next_token_logits_2)
```

High-temperature scaling (flatter distribution):

```
# Source:
next_token_logits_3 = next_token_logits / 5
probabilities = softmax(next_token_logits_3)
```

A temperature of 0.1 produces a very sharp distribution—almost greedy—while a temperature of 5 produces a very flat one where even low-probability tokens have a reasonable chance of being selected.

Multinomial Sampling

Once we have a probability distribution, we need a way to sample from it that is not simply "take the maximum." The probability distribution used for sampling the next token is the multinomial probability distribution. Sampling from a distribution means drawing values where the outcome is not predetermined; for a multinomial distribution, samples are drawn according to the probability scores of mutually exclusive outcomes. The multinomial distribution is used for k mutually exclusive outcomes, each with corresponding probabilities, where n independent trials are conducted to predict outcomes.

In practice, the multinomial function samples the next token proportional to its probability score. PyTorch's torch.multinomial implements this directly:

```
# Source:
weights = torch.tensor([0, 10, 3, 0], dtype=torch.float)
torch.multinomial(weights, 2)
# Returns: tensor([1, 2])

# Source:
torch.multinomial(weights, 5)  # ERROR without replacement
# RuntimeError: cannot sample n_sample > prob_dist.size(-1) samples without replace
```

```
# Source:
torch.multinomial(weights, 4, replacement=True)
# Returns: tensor([2, 1, 1, 1])
```

The first example draws 2 samples without replacement from a weight vector where index 1 has weight 10 and index 2 has weight 3—so index 1 is much more likely to appear first. The second example shows that you cannot draw more samples than there are elements without enabling replacement. The third enables replacement, allowing the same index to be drawn multiple times.

Top-K Sampling

Even with temperature scaling and multinomial sampling, very low-probability tokens can occasionally be selected, producing nonsensical output. Top-K sampling addresses this by restricting the sampled tokens to the top K most likely tokens and excluding all others. Put differently, it selects only the top k tokens with the highest logits for the next token prediction, preventing random or low-probability tokens from ever becoming the next token.

The mechanism is straightforward: find the K highest logits, then replace every other logit with negative infinity. After softmax, negative-infinity logits become zero probability and can never be sampled.

PyTorch provides torch.topk for finding the top values:

```
torch.topk(logits, k=3)  # returns the top 3 values and their indices
```

And torch.where for replacing the rest:

```
torch.where(condition, logits, -inf)  # replaces logits that don't meet the conditi
```

Putting It All Together: The Decoding Workflow

The complete decoding pipeline combines both strategies in the following order:

1. Apply top-k sampling: keep only the top-k logits.
2. Replace all non-top-k logits with negative infinity.
3. Apply temperature scaling: divide the remaining logits by the temperature T.
4. Apply softmax to convert logits to probabilities.
5. Sample from the multinomial distribution to predict the next token.

Figure 9.1: Five-step decoding pipeline for sampling the next token from GPT logits using top-K sampling with temperature scaling.

Together, temperature scaling and top-k sampling reduce overfitting in text generation. This is also why ChatGPT produces different output each time: it uses these decoding strategies rather than memorizing user input.

Here is an example call to the generate function using both strategies:

```
# Source:
generate(model, input_token_ids=['every', 'effort', 'moves', 'you'], max_new_tokens
```

Top-k sampling was introduced as a second decoding strategy specifically to reduce loss and overfitting.

9.2 Model Persistence: Saving and Loading Weights

Loading and saving model weights is especially important when working with large models like the one built in this series. It saves both memory and time, and it is also a prerequisite for working with pre-trained weights from OpenAI.

The Model State Dictionary

The two main PyTorch functions for saving and loading model parameters are torch.save() and model.load_state_dict(). model.state_dict() returns a dictionary mapping each layer of a PyTorch model to its parameter tensors, and all learnable parameters of any torch.nn.Module are accessible via model.parameters().

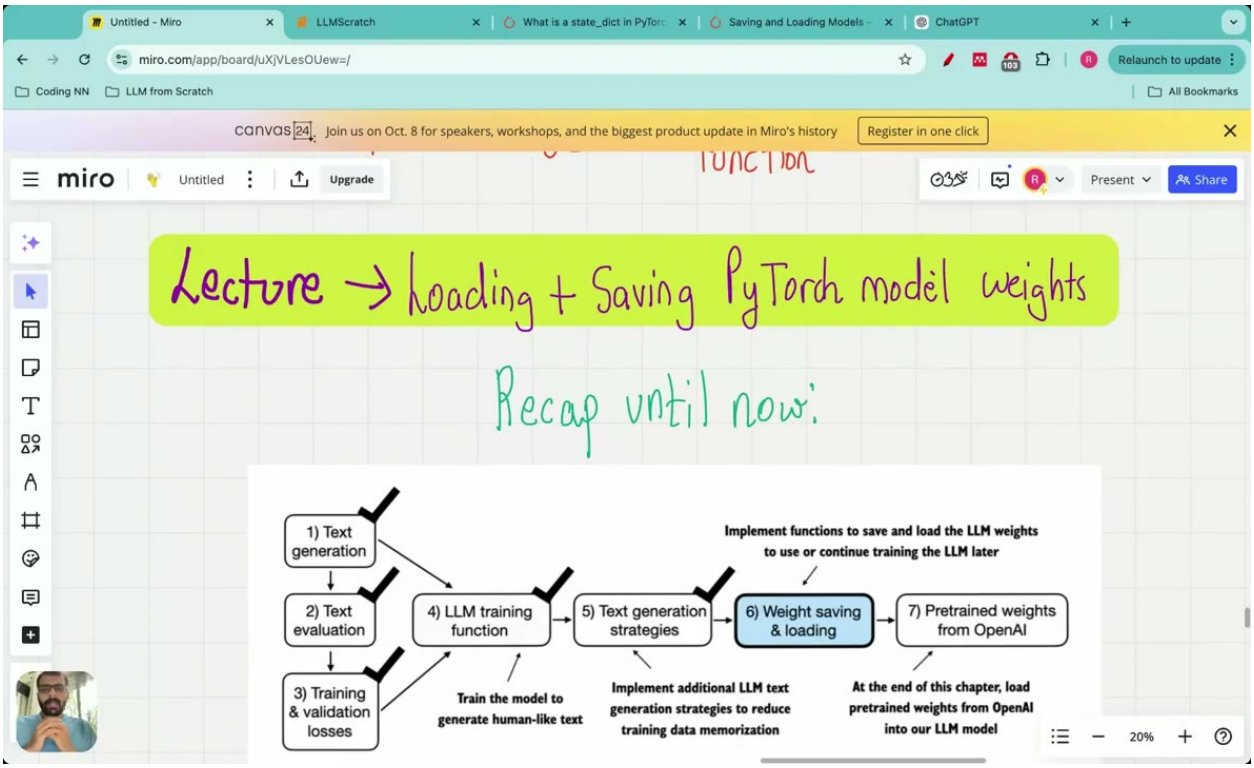


Figure 9.2: Lecture roadmap showing the progression from text generation through weight saving and loading, with checkmarks indicating completed steps and annotations describing implementation goals for LLM development.

The Optimizer State Dictionary

When resuming training—not just inference—you also need to save the optimizer state. The optimizer state dictionary stores both hyperparameters (such as learning rate and weight decay) and

historical data used by the optimizer (such as past gradient values and squared gradient values). Specifically, optimizer.state_dict() returns a dictionary with two entries: 'state' (a dict holding current optimization state per parameter) and 'param_groups' (a list containing all parameter groups).

Saving and reloading both the model and optimizer states is essential for training large language models without losing progress and having to restart from scratch.

Loading a Checkpoint

To resume from a saved checkpoint, you reconstruct both the model and the optimizer, then load their respective state dictionaries:

```
# Source:
checkpoint = torch.load('model_and_optimizer.pth')
model = GPTModel()
model.load_state_dict(checkpoint['model'])
optimizer = torch.optim.AdamW(model.parameters())
optimizer.load_state_dict(checkpoint['optimizer'])
model.train()
```

The key insight is that GPTModel() is first instantiated with its architecture—so PyTorch knows the shape of every tensor—and then the saved weights are injected via load_state_dict. The optimizer is similarly reconstructed and then populated with its historical state, so momentum and adaptive learning-rate estimates are preserved exactly where training left off.

9.3 Loading Pre-Trained GPT-2 Weights

With a working model class, decoding strategies, and weight-loading infrastructure in place, we can now load OpenAI's publicly released GPT-2 weights into our custom model and verify that the result generates coherent text. The lecture series builds a GPT model from scratch using a custom-defined model class, and the GPT-2 weights will be loaded directly into that class for text generation.

GPT-2 Model Sizes

GPT-2 is available in four sizes—124M, 355M, 774M, and 1.5B parameters—differing in embedding dimension, number of layers, and number of attention heads:

```
# Source:
model_configs = {
    "gpt2-small␣(124M)": {"emb_dim": 768, "n_layers": 12, "n_heads": 12},
    "gpt2-medium␣(355M)": {"emb_dim": 1024, "n_layers": 24, "n_heads": 16},
    "gpt2-large␣(774M)": {"emb_dim": 1280, "n_layers": 36, "n_heads": 20},
    "gpt2-xl␣(1558M)": {"emb_dim": 1600, "n_layers": 48, "n_heads": 25},
}

model_name = "gpt2-small␣(124M)"
```

```
NEW_CONFIG = GPT_CONFIG_124M.copy()
NEW_CONFIG.update(model_configs[model_name])
```

Here, n_heads is the number of attention heads in each transformer block, and n_layers is the total number of transformer blocks in the model.

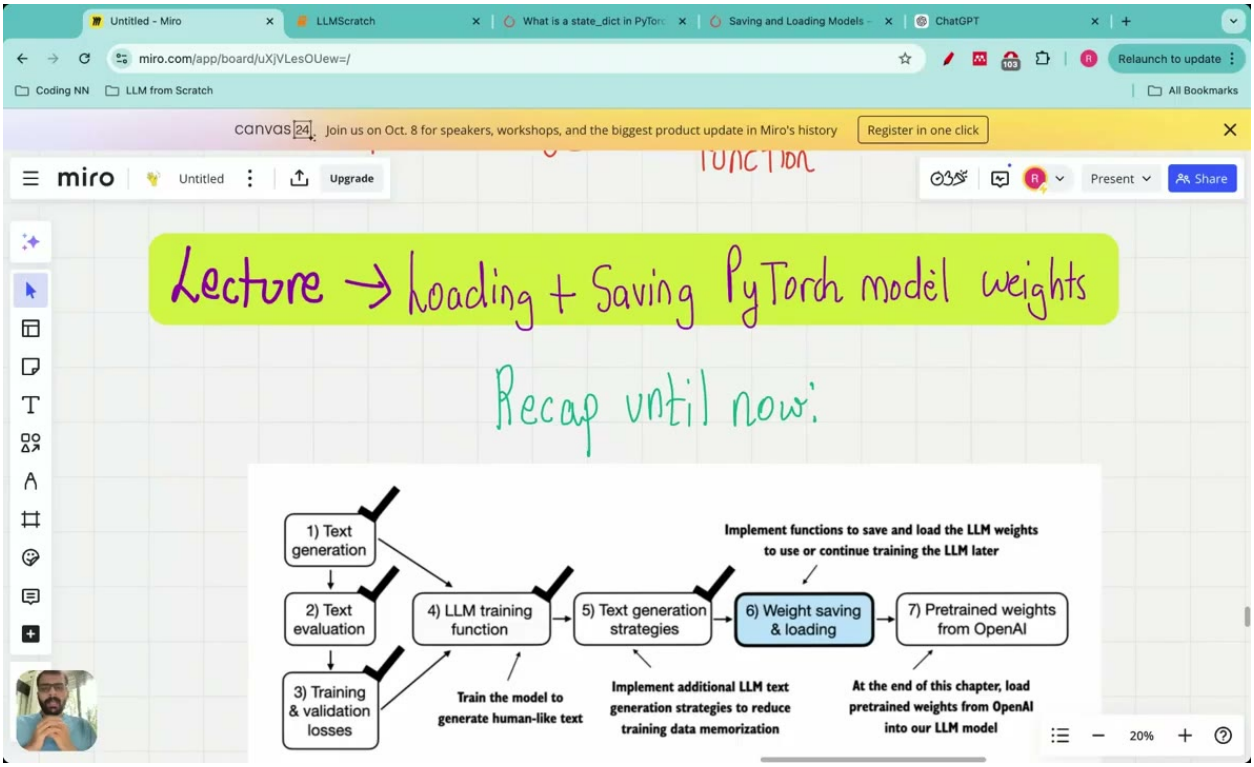


Figure 9.3: Lecture roadmap showing the progression of LLM development steps from text generation through weight saving and loading, with checkmarks indicating completed milestones and annotations describing implementation goals.

Downloading the GPT-2 Files

OpenAI originally saved GPT-2 weights using TensorFlow, while our code uses PyTorch, so some conversion work is required. The following function handles downloading all necessary files:

```
# Source:
def download_and_load_gpt2(model_size, models_dir):
    allowed_sizes = ("124M", "355M", "774M", "1558M")
    if model_size not in allowed_sizes:
        raise ValueError(f"Model size not in {allowed_sizes}")
    model_dir = os.path.join(models_dir, model_size)
    base_url = "https://openaipublic.blob.core.windows.net/gpt-2/models"
    filenames = ["checkpoint", "encoder.json", "params.json", "model.ckpt.data-0000
    os.makedirs(model_dir, exist_ok=True)
    for filename in filenames:
        file_url = os.path.join(base_url, model_size, filename)
        file_path = os.path.join(model_dir, filename)
```

```
    download_file(file_url, file_path)
```

To invoke it:

```
# Source:
settings, params = download_and_load_gpt2(model_size="124M", models_dir="gpt2")
```

What the Downloaded Files Contain

Each downloaded file serves a specific purpose:

- **checkpoint**: Contains the path where all current parameters and weights of the GPT-2 model are stored.
- **model.ckpt files**: Store all the weights of the GPT-2 model.
- **encoder.json**: A vocabulary mapping tokens to their corresponding token IDs.
- **vocab.bpe**: Contains a list of byte-pair encoded token merges, ordered by merge frequency with the highest-probability merges at the top. Byte-pair encoding is a sub-word tokenization scheme that merges the most frequently occurring pairs of tokens into single tokens.
- **hparams.json**: Contains all the hyperparameter settings and configuration values for the GPT-2 model.

The `settings` and `params` Dictionaries

After calling download_and_load_gpt2, you receive two dictionaries. The settings dictionary contains the same hyperparameters as those in hparams.json: vocabulary size, context length, embedding dimension, number of attention heads, and number of transformer blocks.

The params dictionary has five keys:

Key	Contents
WTE	Token embedding parameters—used to convert input token IDs into token embeddings
WPE	Positional embedding parameters—used to add positional information to token embeddings
blocks	All trainable weights within the transformer blocks, including attention layers, feed-forward networks, output projections, and layer normalization parameters
G	Final layer normalization scale
B	Final layer normalization shift

One important structural difference to be aware of: in GPT-2, the query, key, and value weight matrices in the attention layer are fused into a single large matrix called C_ATTN. Our custom model stores them separately, so this matrix must be split when loading.

Parsing the TensorFlow Checkpoint

Because OpenAI saved the weights in TensorFlow format, we need a helper function to read them and organize them into a Python dictionary that mirrors our model's structure:

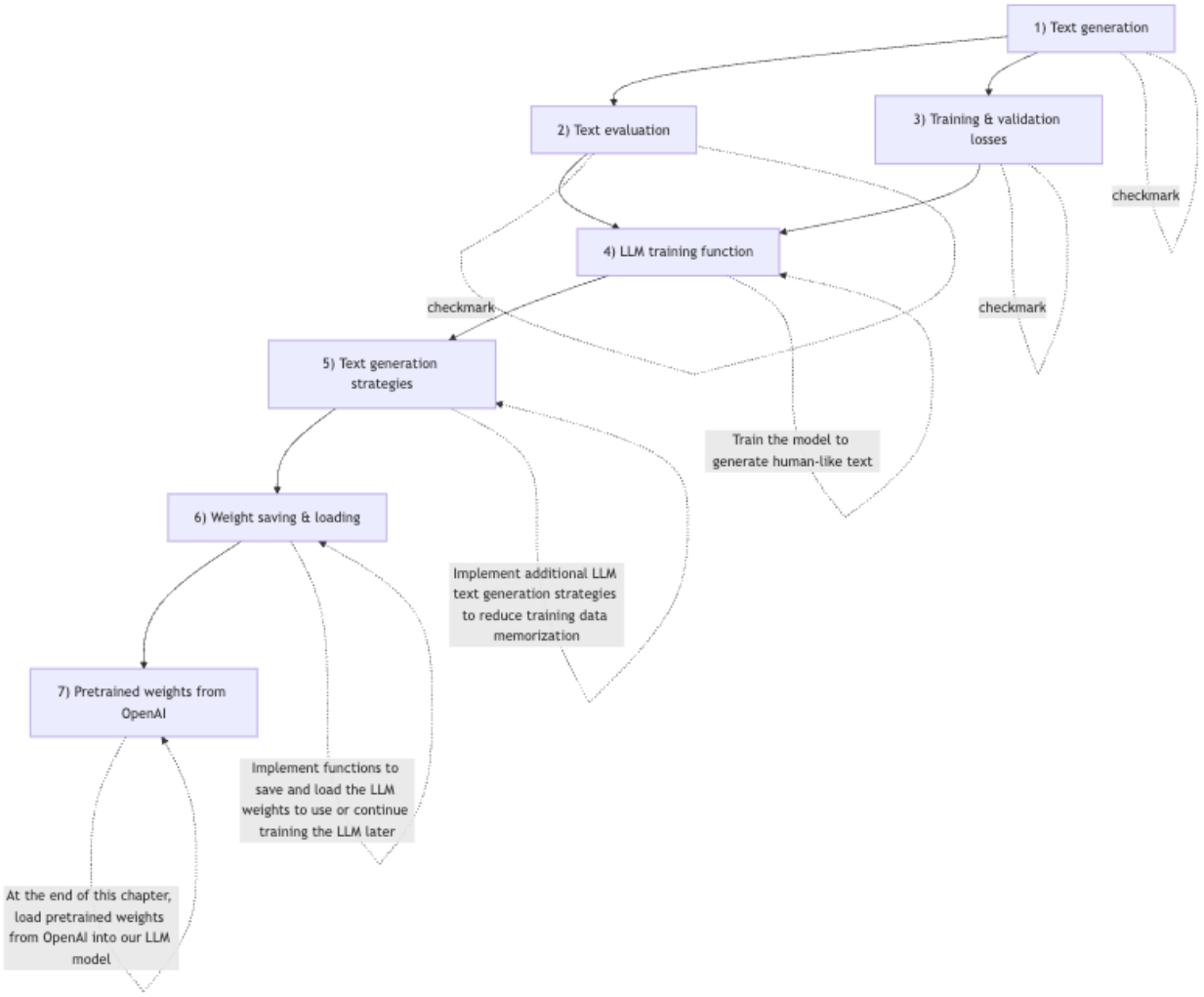


Figure 9.4: LLM development workflow showing the progression from text generation through weight saving and loading, with completed steps marked and implementation goals annotated.

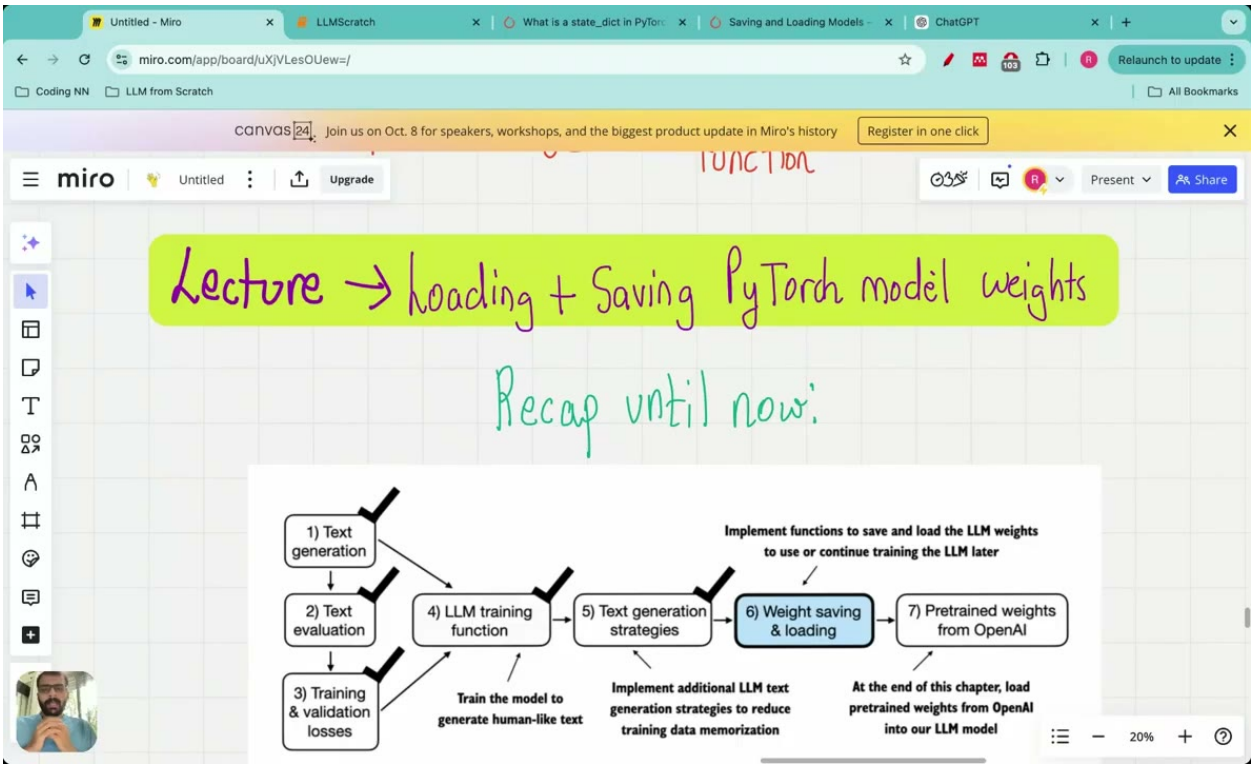


Figure 9.5: Lecture roadmap showing the progression from text generation through weight saving and loading, with checkmarks indicating completed steps and annotations describing implementation goals for the LLM training chapter.

```
# Source:
def load_gpt2_params_from_tf_ckpt(ckpt_path, settings):
    # Initialize parameters dictionary with empty blocks for each layer
    params = {"blocks": [{} for _ in range(settings["n_layer"])]}

    # Iterate over each variable in the checkpoint
    for name, _ in tf.train.list_variables(ckpt_path):
        # Load the variable and remove singleton dimensions
        variable_array = np.squeeze(tf.train.load_variable(ckpt_path, name))

        # Process the variable name to extract relevant parts
        variable_name_parts = name.split("/")[1:]  # Skip the 'model/' prefix

        # Identify the target dictionary for the variable
        target_dict = params
        if variable_name_parts[0].startswith("h"):
            layer_number = int(variable_name_parts[0][1:])
            target_dict = params["blocks"][layer_number]

        # Recursively access or create nested dictionaries
        for key in variable_name_parts[1:-1]:
            target_dict = target_dict.setdefault(key, {})

        # Assign the variable array to the last key
        last_key = variable_name_parts[-1]
        target_dict[last_key] = variable_array
```

This function iterates over every variable in the TensorFlow checkpoint, strips the model/ prefix from the variable name, and uses the remaining path components to build a nested Python dictionary. Variables whose names start with h (e.g., h0, h1, ...) are routed into the corresponding entry of params["blocks"].

Assigning Weights to the Custom Model

Once the params dictionary is populated, its values are assigned to the corresponding tensors in our PyTorch model. A shape-checking helper ensures we do not silently load mismatched weights:

```
# Source:
def assign(left, right):
    if left.shape != right.shape:
        raise ValueError(f"Shape␣mismatch:␣{left.shape}␣vs␣{right.shape}")
    return right
```

The GPT model class initializes token embeddings, positional embeddings, transformer blocks, layer normalization, and an output head layer, and each of these components receives its weights from the corresponding key in params. The output projection layer weights and biases from GPT-2 are assigned to the attention object's output projection, and feed-forward network weights and biases for both the fully connected and projection layers are assigned from the downloaded GPT-2 values.

After all assignments are complete, we confirm the full pipeline with one final call:

```
# Source:
from gpt_download3 import download_and_load_gpt2

settings, params = download_and_load_gpt2(model_size="124M", models_dir="gpt2")
```

Verifying the Loaded Weights

The proof is in the output. Without GPT-2 weights, the model generates incoherent text. With pretrained weights loaded, inference is fast because no training is required, and the output is noticeably more coherent. In short, the model weights are loaded correctly when the model produces coherent text.

Here is an example generation call that exercises both decoding strategies with the loaded model:

```
# Source:
generate(model, input_token_ids=['every', 'effort', 'moves', 'you'], max_new_tokens
```

With temperature=1.5 and top_k=50, the model samples from the top 50 most likely tokens at each step, with a relatively high temperature that encourages diversity. The result should be grammatically coherent English that continues the prompt naturally.

9.4 Why Build a Custom Model Class?

One might ask: if OpenAI already provides GPT-2, why go through the effort of building a custom model class and loading weights into it? The answer is flexibility. The custom-built GPT model allows exploration of hyperparameter tuning and architectural modifications that are not possible with ChatGPT. By owning the model code, you can change the number of layers, swap out the attention mechanism, experiment with different normalization schemes, or fine-tune on domain-specific data—none of which is possible through an API.

9.5 Summary

This chapter covered three major topics:

1. **Decoding strategies.** Greedy decoding is deterministic and prone to repetition. Temperature scaling divides logits by a temperature parameter before softmax, sharpening or flattening the probability distribution. Multinomial sampling draws the next token proportionally to its probability rather than always taking the maximum. Top-k sampling restricts the candidate set to the K most likely tokens, preventing low-probability tokens from ever being selected. The full decoding pipeline applies these in sequence: top-k masking → temperature scaling → softmax → multinomial sample.
2. **Model persistence.** model.state_dict() captures all learnable parameters as a dictionary. optimizer.state_dict() captures both hyperparameters and historical gradient information. Saving and reloading both is essential for resuming long training runs without losing progress.

3. **Loading pre-trained GPT-2 weights.** OpenAI's GPT-2 comes in four sizes (124M, 355M, 774M, 1558M) and was originally saved in TensorFlow format. A download-and-parse pipeline converts these weights into a nested Python dictionary, which is then assigned to the corresponding tensors in our custom PyTorch model using a shape-checking assign helper. The result is a model that generates coherent text immediately, without any additional training.

Chapter 10

Classification Fine-Tuning: Spam Detection from Scratch

Fine-tuning is the process of adapting a pre-trained model to a specific task by training it on additional data. Rather than training from scratch—which demands enormous compute and data—we start from a foundation model that has already learned rich representations of language, then nudge its weights toward a narrower objective.

This chapter builds the entire pipeline deliberately from scratch. By the end, you will have downloaded and balanced a real dataset, written a custom PyTorch Dataset and DataLoader, surgically modified the GPT-2 architecture to output class logits instead of vocabulary logits, frozen the right subset of parameters, implemented accuracy and loss utilities, run a full training loop, and tested the resulting classifier on messages it has never seen.

10.1 The Big Picture: Classification Fine-Tuning

10.1.1 Two Flavors of Fine-Tuning

LLM fine-tuning has two broad categories: instruction fine-tuning and classification fine-tuning. Instruction fine-tuning trains the model to follow natural-language instructions and produce free-form text responses. Classification fine-tuning, by contrast, uses a large language model to classify inputs into predefined categories without explicit instructions.

The practical difference matters. Instruction fine-tuning requires greater computational power than classification fine-tuning because the model must search the entire corpus based on given instructions and perform non-specific actions. For spam detection, we do not need the model to generate a sentence explaining its reasoning—we simply need it to output one of two labels.

In classification fine-tuning for spam detection, no instruction is given to the LLM; only the input text is provided and the model classifies it as spam or not spam. The model architecture is augmented at the output end to support binary classification, with outputs of either spam (1) or no spam (0).

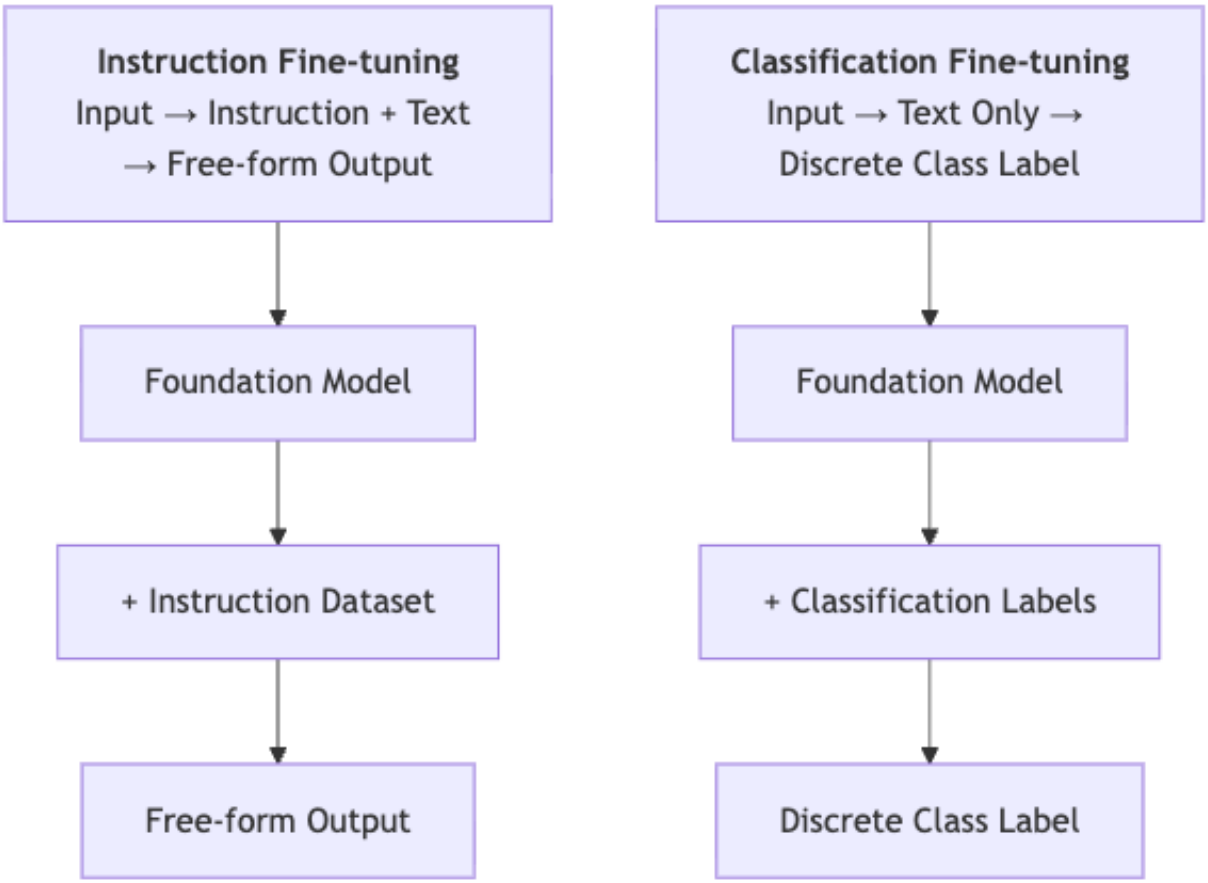


Figure 10.1: Comparison of instruction fine-tuning (left) versus classification fine-tuning (right), showing different input formats and output types.

10.1.2 Parameter-Efficient Alternatives

Parameter-efficient fine-tuning updates only a subset of parameters while freezing the rest, reducing both the number of trainable parameters and memory requirements. LoRA, for instance, fine-tunes two smaller matrices that approximate a larger weight matrix rather than updating all weights directly. QLoRA takes this further by quantizing the weights of LoRA adapters to lower precision. This chapter focuses on selective full fine-tuning—freezing most of the model but fully updating the layers we care about—which strikes a good balance between simplicity and effectiveness.

10.1.3 The Three Stages

The fine-tuning process is divided into three main stages: dataset preparation, model initialization and modification, and fine-tuning with evaluation.

- **Stage 1 – Dataset preparation:** Download the SMS Spam Collection dataset, balance it, tokenize messages, pad sequences, and wrap everything in PyTorch Dataset and DataLoader objects.
- **Stage 2 – Model initialization and modification:** Load pre-trained GPT-2 weights, replace the output head with a two-class linear layer, and freeze all but the last transformer block and final layer normalization.
- **Stage 3 – Fine-tuning and evaluation:** Run the training loop, track loss and accuracy on train and validation splits, and finally test on held-out data and new messages.

10.2 The SMS Spam Collection Dataset

10.2.1 Dataset Overview

The fine-tuning classification project uses the SMS Spam Collection dataset from the UCI Machine Learning Repository. It consists of text messages labeled as either spam or not spam; the label for non-spam messages is called HAM. Among its sources, the dataset contains 425 spam messages manually extracted from the Grumbletext website.

The original dataset is imbalanced: there are 4,825 ham messages and only 747 spam messages. We address this by balancing the dataset before training.

10.2.2 Downloading the Dataset

The following snippet downloads the dataset archive, extracts it, and renames the file to give it a .csv extension:

```
# Create an unverified SSL context
ssl_context = ssl._create_unverified_context()

# Downloading the file
with urllib.request.urlopen(url, context=ssl_context) as response:
    with open(zip_path, "wb") as out_file:
        out_file.write(response.read())

# Unzipping the file
```

```
with zipfile.ZipFile(zip_path, "r") as zip_ref:
    zip_ref.extractall(extracted_path)

# Add .csv file extension
original_file_path = Path(extracted_path) / "SMSSpamCollection"
os.rename(original_file_path, data_file_path)
print(f"File downloaded and saved as {data_file_path}")
```

In production code you would want proper certificate handling.

10.2.3 Balancing and Splitting the Dataset

Because the original dataset has approximately 4,800 ham messages and 747 spam messages, we downsample the majority class. A balanced dataset prevents the model from learning a trivial shortcut—predicting the majority class every time.

After balancing, we split the data into training, validation, and test sets using the following function:

```
def random_split(df, train_frac=0.7, validation_frac=0.2):
    train_df = df.sample(frac=train_frac)
    remaining = df.drop(train_df.index)
    validation_df = remaining.sample(frac=validation_frac/(1-train_frac))
    test_df = remaining.drop(validation_df.index)
    return train_df, validation_df, test_df
```

Applying this split to the balanced dataset yields the following counts:

```
train_df, validation_df, test_df = random_split(balanced_df)
print(len(train_df))        # 1045
print(len(validation_df))   # 149
print(len(test_df))         # 300
```

The training set contains 1,045 examples, the validation set 149, and the test set 300. The validation set is used during training to monitor generalization; the test set is held out until the very end.

10.3 Building the SpamDataset

10.3.1 Dataset vs. DataLoader

A Dataset stores the samples and their corresponding labels. A DataLoader wraps an iterable around the Dataset to enable easy access to those samples. More precisely, a PyTorch dataset specifies how data is loaded and processed before a data loader is instantiated.

10.3.2 Tokenization

The SpamDataset class uses TikToken—a tokenizer library developed by OpenAI that converts sentences into token IDs—to tokenize each message. TikToken employs a byte pair encoder, a sub-word tokenizer that breaks words into smaller units rather than mapping each word to a single token. The resulting vocabulary is a dictionary mapping tokens to their corresponding token IDs.

An end-of-text token is a special token used to mark the end of a document or sentence, and it serves as the padding symbol when sequences are extended to a uniform length.

10.3.3 The SpamDataset Class

The SpamDataset class performs three steps: identifying the longest sequence in the training dataset, converting each text message into token IDs, and padding all sequences to match that longest length. Concretely, it loads data from CSV files, tokenizes text using the GPT-2 tokenizer from TikToken, and pads or truncates sequences to uniform length.

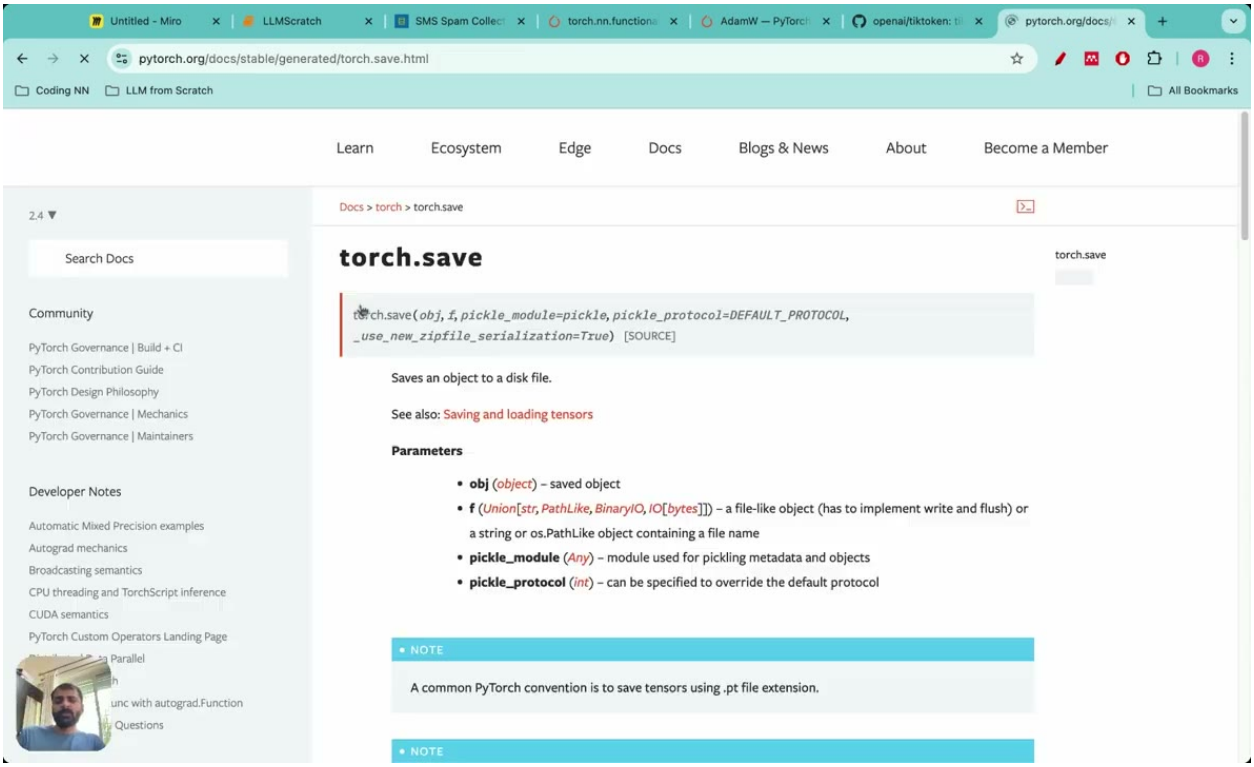


Figure 10.2: Three text messages of varying lengths are tokenized and padded with end-of-text tokens to match the longest sequence length.

The large language model architecture then takes this uniformly-sized input and outputs a classification of whether the message is spam or not spam.

10.3.4 Creating the DataLoaders

With the SpamDataset defined, a DataLoader is instantiated for each split. The training DataLoader shuffles examples each epoch; the validation and test DataLoaders do not. All three handle batching and iteration automatically.

10.4 Modifying the GPT-2 Architecture

10.4.1 Loading Pre-Trained Weights

Fine-tuning begins with the pre-trained GPT-2 weights, which are then updated on the specific classification dataset. The project uses a GPT-based LLM architecture that will be modified for classification fine-tuning. The following function downloads the model files from OpenAI's public storage:

```
def download_and_load_gpt2(model_size, models_dir):
    # Validate model size
    allowed_sizes = ("124M", "355M", "774M", "1558M")
    if model_size not in allowed_sizes:
        raise ValueError(f"Model␣size␣not␣in␣{allowed_sizes}")
    # Downloads GPT-2 model files from OpenAI public blob storage
```

A companion helper then loads the checkpoint parameters into a nested Python dictionary:

```
def load_gpt2_params_from_tf_ckpt(ckpt_path, settings):
    # Recursively loads TensorFlow checkpoint variables into nested params diction
    # Handles token embeddings, positional embeddings, transformer blocks, and laye
```

10.4.2 Why the Pre-Trained Model Cannot Classify Spam Out of the Box

The pre-trained GPT-2 model without fine-tuning struggles to follow instructions and lacks classification capabilities for tasks like spam detection. Even when given explicit instructions, it cannot correctly answer spam classification prompts. We therefore need to modify its architecture before any fine-tuning can take place.

10.4.3 Replacing the Output Head

A classification head is a small linear layer added on top of a pre-trained language model to adapt it for classification tasks by mapping hidden states to class logits. For binary classification we need only two logits, so we replace the original 768→50,257 output layer with a 768→2 linear layer:

```
model.output_head = Linear(in_features=768, out_features=num_classes)
```

This replaces the original output layer—which mapped to vocabulary size—with a smaller layer that maps to the number of classes, taking input of size 768 and producing 2 values for spam or no-spam classification.

10.4.4 Which Token's Output to Use

Rather than aggregating all output positions, we focus on the last output token, since it contains all the information accumulated over the sequence. In PyTorch, extracting that token's output from a batch is simply:

```
outputs[:, -1, :]
```

Extracting the last output token from a 4-token sequence yields a vector with two class logits—for example, -3.5898 and 3.9902 for spam/not-spam classification. The larger logit wins: in this example the model would predict "not spam."

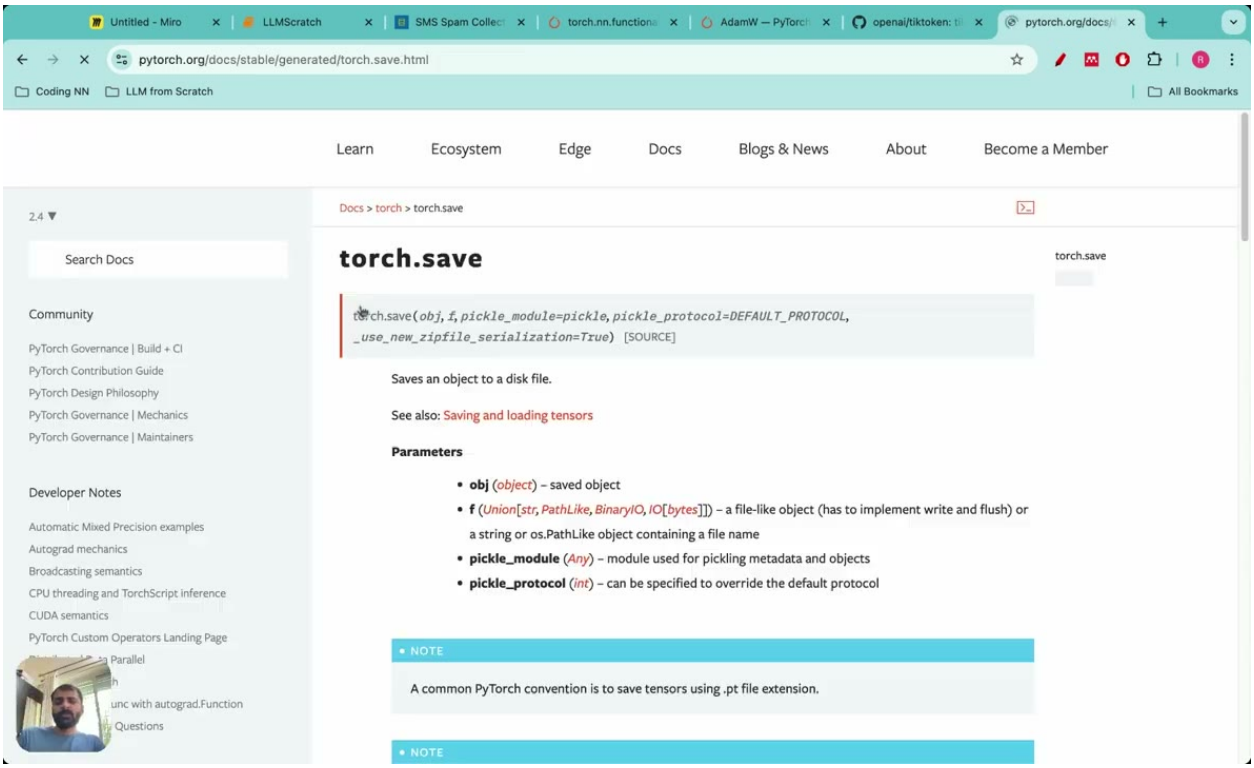


Figure 10.3: Three-stage pipeline for instruction finetuning: Stage 1 builds an LLM from data preparation, attention mechanism, and architecture; Stage 2 pretrains the model; Stage 3 finetunes with a classifier and personal assistant using instruction data.

10.4.5 Deciding Which Parameters to Train

Before training, the parameters in the new output head, final transformer block, and final layer normalization are still random and have not been trained on the classification dataset. We need to update them—but we do not want to train the entire model, which would be slow and risk destroying the pre-trained representations.

Three components will be fine-tuned: the classification head, the final (12th) transformer block, and the final layer normalization module. To unfreeze the final transformer block:

```
for param in model.transformer.h[-1].parameters():
    param.requires_grad = True
```

To unfreeze the final layer normalization:

```
for param in model.ln_f.parameters():
    param.requires_grad = True
```

The model passed to the training function is a GPT model class with a modified architecture that includes the classification head on top. By freezing the bulk of the network and training only the top layers, we preserve the general language understanding learned during pre-training while adapting the model's output behavior to our specific task.

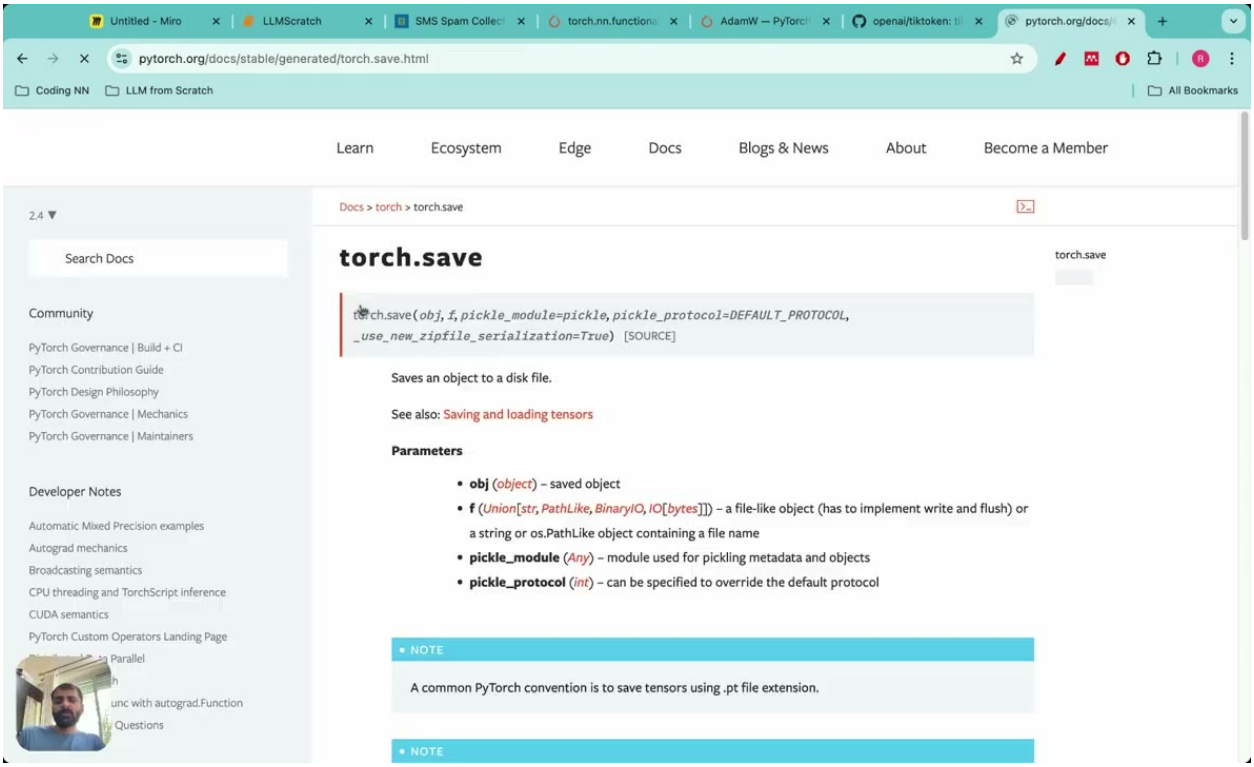


Figure 10.4: GPT-2 layer stack showing frozen layers (grey) and trainable layers including the classification head, final transformer block, and final layer norm.

10.5 Evaluation Utilities: Accuracy and Loss

Before writing the training loop, we need two measurement tools: an accuracy function and a loss function.

10.5.1 Classification Accuracy

Classification accuracy measures the percentage of correct predictions across a dataset:

$$\text{Accuracy} = \frac{\text{correct\_predictions}}{\text{total\_number\_of\_examples}}$$

Predicted labels are the class predictions generated by applying argmax to model logits; output labels are also called target labels, representing the true values in the dataset. The following function computes accuracy over a data loader:

```
def calculate_accuracy(loader, num_batches=None):
    if num_batches is None:
        num_batches = len(loader)
    else:
        num_batches = min(num_batches, len(loader))
    correct_predictions = 0
    num_examples = 0
    for batch in loader:
        logits = model(batch)
        predicted_labels = argmax(logits)
        target_labels = batch.labels
        correct_predictions += sum(predicted_labels == target_labels)
        num_examples += len(batch)
    accuracy = correct_predictions / num_examples
    return accuracy
```

Evaluation frequency specifies how many batches must be processed before printing training and validation loss metrics. Evaluation iteration specifies the number of batches to use for evaluation; using fewer batches (e.g., 5 or 10) saves computation time during training. The num_batches parameter in calculate_accuracy serves exactly this purpose—during training we can pass a small number to get a quick estimate without waiting for a full pass over the data.

10.5.2 Cross-Entropy Loss

Categorical cross-entropy loss is defined as the negative sum over all class labels of $(y_i \cdot \log(p_i))$, where y_i is the true value and p_i is the predicted probability. In PyTorch, this is computed with the built-in function torch.nn.functional.cross_entropy().

10.6 The Fine-Tuning Training Loop

10.6.1 Training Loop Structure

With the dataset, model, and evaluation utilities in place, we can write the training loop. One epoch is one complete pass through the entire dataset. The loop iterates over epochs, and within each epoch iterates over batches from the training DataLoader. The overall process covers implementing the accuracy and loss metrics, a backward pass for training, parameter updates to minimize loss, and testing on unseen data.

At regular intervals controlled by evaluation frequency, the loop computes and prints training and validation loss. After every epoch, it calculates and prints training accuracy and validation accuracy.

10.6.2 Epoch-Level Evaluation

Printing accuracy after each epoch provides a higher-level view of learning progress than per-batch loss alone.

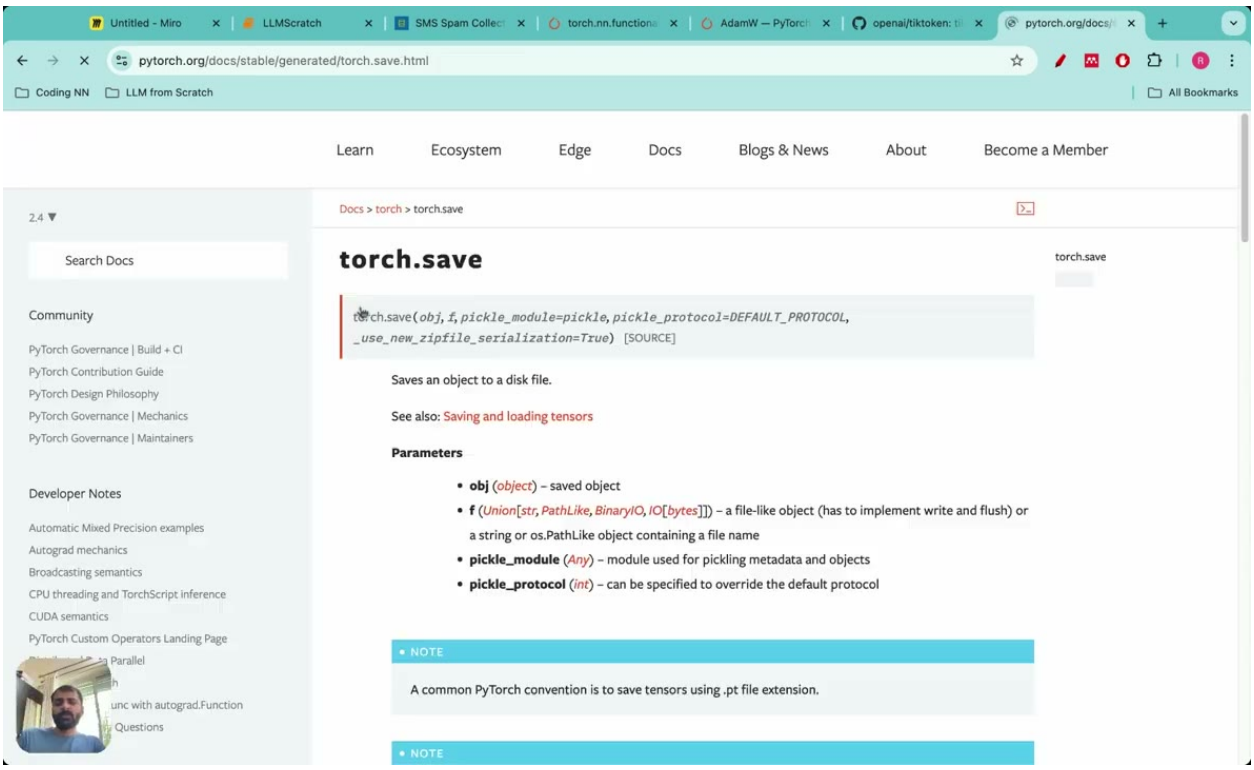


Figure 10.5: Training and validation loss and accuracy curves over epochs, demonstrating model convergence during the training process.

10.6.3 Saving the Model

After training, we save the model weights so they can be reloaded without retraining. PyTorch's torch.save handles this:

```
torch.save(obj, f, pickle_module=pickle, pickle_protocol=DEFAULT_PROTOCOL,
```

```
_use_new_zipfile_serialization=True)
```

Fine-tuning is the process of changing model parameters and retraining on a specific dataset so the model performs well on that particular task; saving the weights preserves the result of that process for future use.

10.7 Evaluating the Fine-Tuned Model

10.7.1 Train, Validation, and Test Accuracy

Training and validation accuracies are monitored throughout training. Test accuracy is the number we care about most, because it reflects performance on data the model has never seen. The email classification project involves training a large language model to classify emails as spam or no spam, and the final stage is testing the fine-tuned model on exactly that held-out data.

10.7.2 Comparing to the Unmodified Model

The contrast with the baseline is stark. The pre-trained GPT-2 model without fine-tuning struggles to follow instructions and lacks classification capabilities for spam detection; even when given explicit instructions, it cannot correctly answer spam classification prompts. Fine-tuning resolves both shortcomings.

10.8 Running Inference on New Messages

After training and evaluation, we want to use the model in practice—feeding it a raw text message and getting back a prediction. The inference procedure mirrors the training forward pass: tokenize the input, pad to the expected length, run the forward pass through the modified GPT-2, extract the last token's output (`outputs[:, -1, :]`), and apply argmax to obtain the predicted class.

Figure 10.6: Inference pipeline for spam/ham classification: raw text flows through tokenization, padding, a GPT-2 model with classification head, and argmax selection to produce a final label.

The classification fine-tuning project involves predicting whether text messages are spam or not spam. With a well-trained model, this pipeline correctly flags messages like "Congratulations! You've won a free prize, call now!" as spam while passing through legitimate messages.

10.9 Putting It All Together

Let us trace the full pipeline from raw data to a working spam classifier.

Stage 1 – Data preparation: - Download the SMS Spam Collection dataset from the UCI repository. - Observe the class imbalance: 4,825 ham vs. 747 spam. - Downsample ham to match the spam count, creating a balanced dataset. - Split into train (1,045), validation (149), and test (300) sets. - Build the SpamDataset class: tokenize with TikToken, pad to the longest sequence in the training set. - Wrap each split in a DataLoader.

Stage 2 – Model initialization and modification: - Download GPT-2 weights using download_and_load_gpt2. - Load TensorFlow checkpoint parameters with load_gpt2_params_from_tf_ckpt. - Replace the 768→50,257 output head with a 768→2 classification head. - Freeze all parameters except the classification head, the final transformer block, and the final layer normalization.

Stage 3 – Fine-tuning and evaluation: - Implement calculate_accuracy and use torch.nn.functional.cross_entropy for loss. - Run the training loop: forward pass, loss computation, backward pass, parameter update. - Print loss every eval_freq batches; print accuracy after every epoch. - After training, evaluate on the test set. - Save the fine-tuned weights with torch.save. - Run inference on new messages.

10.10 Summary and What Comes Next

This chapter built a complete classification fine-tuning pipeline from scratch. Starting from the observation that LLM fine-tuning has two broad categories—instruction fine-tuning and classification fine-tuning—we focused on the latter because it is computationally lighter and well-suited to the spam detection task.

The key architectural insight is the classification head: a small 768→2 linear layer that replaces the original vocabulary projection. By freezing most of the pre-trained model and training only the classification head, the final transformer block, and the final layer normalization, we preserve the general language understanding encoded in GPT-2's weights while teaching the model to produce class labels.

The evaluation utilities—accuracy and cross-entropy loss—provide concrete numbers to track during training. The num_batches parameter in the accuracy function allows a trade-off between evaluation speed and precision during training, while full-dataset evaluation at the end of each epoch gives the complete picture.

LLM fine-tuning involves the additional training of a pre-existing model that has previously acquired patterns and features from an extensive dataset, now applied to a smaller domain-specific dataset. The SMS Spam Collection dataset, with its 1,494 balanced examples after preprocessing, illustrates how a small, well-curated dataset can be enough to adapt a powerful pre-trained model to a new task. The techniques introduced here—selective layer unfreezing, classification head replacement, and the train/val/test evaluation discipline—generalize directly to other classification tasks such as sentiment analysis, topic classification, and intent detection. The next natural step is instruction fine-tuning, where instead of predicting a class label the model learns to follow natural-language instructions and generate free-form responses.

Chapter 11

Instruction Fine-Tuning: Building a Personal Assistant

After pre-training, a language model can complete sentences and generate coherent prose—but it does not yet behave like an assistant. Instruction fine-tuning bridges that gap, transforming a next-token predictor into a model that responds helpfully to user requests.

The pipeline breaks naturally into three stages:

- **Stage 1 – Dataset Preparation:** download the data, format it with the Alpaca prompt template, and build training, validation, and test DataLoaders.
- **Stage 2 – Fine-Tuning:** load the pre-trained GPT-2 medium (355 M) checkpoint and run the supervised training loop.
- **Stage 3 – Evaluation:** extract responses from the fine-tuned model, inspect them qualitatively, and score them automatically using Ollama.

Let's begin.

11.1 What Is Instruction Fine-Tuning?

A **foundation model** is a pre-trained large language model trained on large-scale data that can be fine-tuned for downstream tasks. The pre-training phase teaches the model grammar, facts, and reasoning patterns, but it does not teach the model to *follow instructions*.

Instruction fine-tuning is the process of training a pre-trained LLM on a specific dataset to teach it to correctly follow instructions—more precisely, providing a large set of instruction-response pairs so the model learns to respond appropriately. Because we start from a pre-trained checkpoint rather than random weights, fine-tuning requires significantly less computational time than training from scratch.

Fine-tuning modifies the weights and parameters of the GPT model so that it understands instruction input-output pairs from the target dataset. A few representative examples illustrate the range of tasks a single fine-tuned model can handle:

```
Instruction: convert 45 km to meters
```

```
Response: 45 km is 45000 meters

Instruction: provide a synonym for bright
Response: a synonym for bright is radiant

Instruction: edit the following sentence to remove all passive voice.
             The song was composed by the artist
Response: the artist composed the song
```

Unit conversion, vocabulary lookup, and grammar editing—all from the same model, trained on the same dataset.

11.2 Stage 1: Dataset Preparation

The Instruction Dataset

The instruction dataset used in this chapter consists of 1,100 instruction input-output pairs. For reference, the Stanford Alpaca repository contains 52,000 such pairs; our smaller dataset is self-contained and suitable for experimentation.

Each entry in the dataset contains three keys: instruction, input, and output. Before any training can begin, this raw data must be formatted, tokenized, padded, and loaded in batches. Stage 1 covers all of that: downloading the data, formatting it with the Alpaca prompt template, and creating training, validation, and test DataLoaders.

The Alpaca Prompt Format

Raw instruction-input-output triples cannot be fed directly to a language model. A consistent text template is needed to tell the model where the instruction ends and where the response should begin.

Alpaca prompt style is a formatting convention for converting instruction-input-output pairs into prompts for fine-tuning large language models, maintained by Stanford in the Stanford Alpaca repository. It was originally designed for instruction-following LLaMA models. The template reads:

> *Below is an instruction that describes a task paired with an input that provides further context. Write a response that appropriately completes the request.*

followed by the instruction, input, and output fields. Crucially, the instruction and input fields are kept separate in the template.

A second common convention is **Phi-3 prompt style**, in which the instruction and input are fused into a single user field and the output is placed in an assistant field. There are therefore two main ways to format instruction-input-output datasets: Alpaca style and Phi-3 style.

The full Alpaca template looks like this:

```
Below is an instruction that describes a task paired with an input that
provides further context. Write a response that appropriately completes
the request.
```

```
### Instruction:
<instruction text>

### Input:
<input text>

### Response:
<output text>
```

Implementing `format_input`

The following function converts a dataset entry into a formatted Alpaca prompt:

```
def format_input(entry):
    instruction_text = (
        "Below is an instruction that describes a task paired with an input "
        "that provides further context. Write a response that appropriately "
        "completes the request."
    )
    instruction = entry['instruction']
    input_text = entry.get('input', '')
    if input_text:
        prompt = (
            f"{instruction_text}\n\n"
            f"Instruction:\n{instruction}\n\n"
            f"Input:\n{input_text}"
        )
    else:
        prompt = (
            f"{instruction_text}\n\n"
            f"Instruction:\n{instruction}"
        )
    return prompt
```

The function returns only the *input* side of the prompt—the instruction and optional context—without the response. A companion function formatInput follows the same logic:

```
def formatInput(entry):
    # Converts instruction-input-output pairs into alpaca prompt format
    # Takes entry with instruction, input, and output keys
    # Returns formatted prompt with instruction, input, and response sections
```

Data Batching: Five Steps

Batching the dataset means converting multiple data samples into a batch where each sample is represented as a numerical array (row) with uniform dimensions. For instruction fine-tuning, this process has five steps:

1. **Format** – apply the Alpaca prompt template.

2. **Tokenize** – convert the formatted text to token ID sequences.
3. **Pad** – equalize sequence lengths within each batch.
4. **Create target pairs** – shift the input token IDs right by one position.
5. **Mask padding** – replace padding token IDs in the targets with −100.

Let's examine each step in detail.

Step 1: Format

Apply the Alpaca template to each entry using format_input, as described above.

Step 2: Tokenize

Tokenization converts the formatted text into a numerical representation by mapping sentences to token IDs.

Step 3: Pad

Because sequences in a batch typically have different lengths, padding adjusts them to a uniform length. The end-of-text token—token ID 50256 in GPT-2—serves as the padding symbol.

Step 4: Create Target Pairs

During instruction fine-tuning the model is trained with next-token prediction, so the entire formatted prompt serves as both input and target. Target token IDs are created by shifting the input tokens one position to the right and appending an end-of-text token to maintain equal length:

```
targets = inputs[1:]  # Shift inputs right by one by removing first element
```

Step 5: Replace Padding Tokens with -100

Padding positions in the target tensor should not contribute to the loss. We therefore replace them with the special ignore index −100, which PyTorch's cross-entropy loss automatically skips:

```
def custom_collate_function(batch, padding_token_id, ignore_index):
    # Get inputs and targets
    # Create mask for all padding token indices
    mask = (target_tensor == padding_token_id)
    # Ignore the first occurrence of padding_token_id
    mask[0] = False
    # Replace all remaining padding tokens with ignore_index (-100)
    target_tensor[mask] = ignore_index
    return input_tensor, target_tensor
```

Note that the *first* end-of-text token is deliberately preserved: it marks the boundary between the response and the padding, and the model should learn to generate it.

The custom collate function takes a batch of sequences and requires a padding token ID (50256) and a target device (cpu) as inputs:

```
# Custom collate function for creating batches
# Converts tokens to token IDs, pads sequences to equal length within batch,
# replaces padding tokens (50256) with -100, and creates input and target tensors
```

A Note on Masking the Instruction Tokens

A recent paper titled *Instruction Tuning With Loss Over Instructions* demonstrated that not masking the instruction tokens actually benefits LLM performance during instruction fine-tuning. This chapter follows the simpler approach and does not mask instruction tokens, but the custom collate function can be extended to do so if desired.

Building the DataLoader

Data loaders provide an efficient way to collect batches sequentially and access them iteratively during training:

```
train_dataset = InstructionDataset(training_data)
```

The DataLoader wraps this dataset, calls the custom collate function on each batch, and handles shuffling and multi-process loading automatically:

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

11.3 Stage 2: Loading the Pre-Trained Model and Fine-Tuning

Loading GPT-2 Medium (355 M)

A pre-trained GPT-2 model with 355 million weights serves as the foundation for instruction fine-tuning. The configuration and model-selection code is:

```
from gpt_download3 import download_and_load_gpt2

BASE_CONFIG = {
    "vocab_size": 50257,
    "context_length": 1024,
    "drop_rate": 0.0,
    "qkv_bias": True
}

model_configs = {
    "gpt2-small␣(124M)": {"emb_dim": 768,  "n_layers": 12, "n_heads": 12},
    "gpt2-medium␣(355M)": {"emb_dim": 1024, "n_layers": 24, "n_heads": 16},
    "gpt2-large␣(774M)":  {"emb_dim": 1280, "n_layers": 36, "n_heads": 20},
    "gpt2-xl␣(1558M)":    {"emb_dim": 1600, "n_layers": 48, "n_heads": 25},
}

CHOOSE_MODEL = "gpt2-medium␣(355M)"
```

BASE_CONFIG specifies parameters shared across all GPT-2 variants: a vocabulary of 50,257 tokens, a context window of 1,024 tokens, no dropout during fine-tuning (drop_rate: 0.0), and query-key-value bias enabled (qkv_bias: True). model_configs then provides the variant-specific embedding dimension, layer count, and attention-head count.

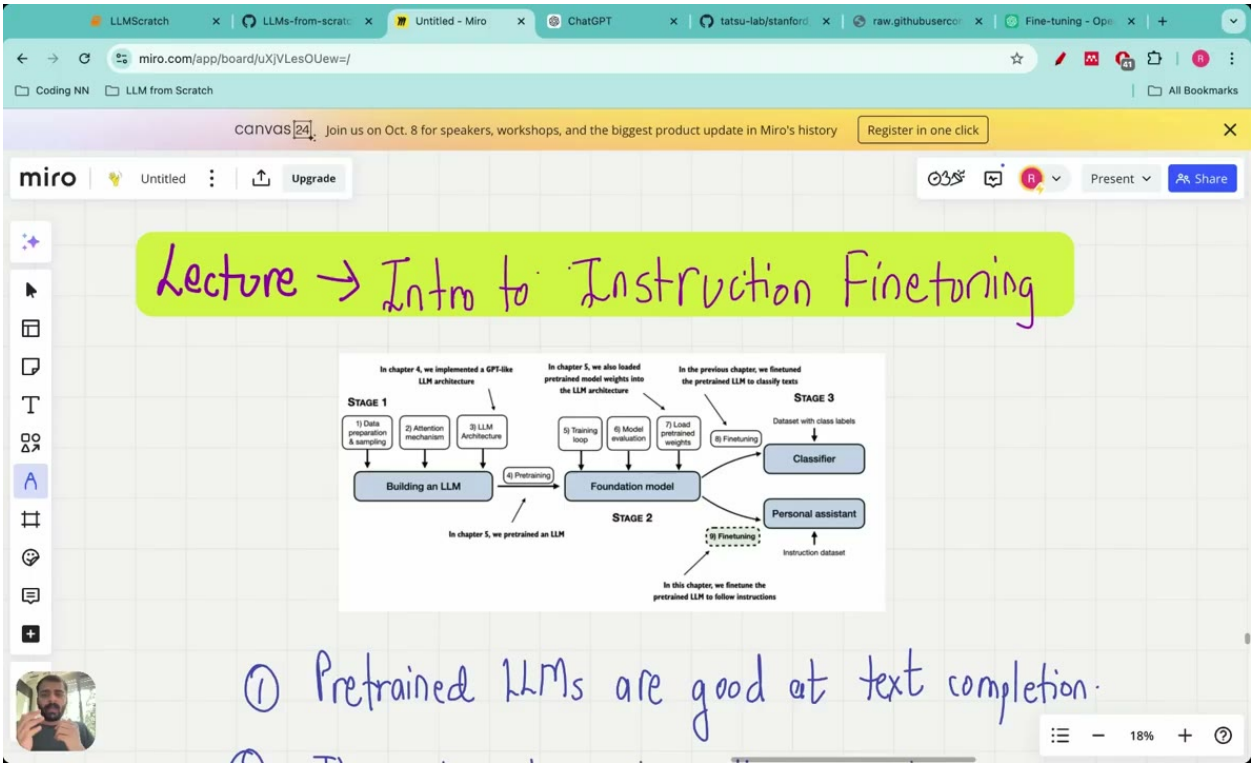


Figure 11.1: Three-stage pipeline for instruction finetuning: Stage 1 builds an LLM from data preparation, attention mechanism, and architecture; Stage 2 pretrains the LLM; Stage 3 finetunes with foundation model, classifier, and personal assistant using instruction dataset.

The Fine-Tuning Training Loop

The dataset processing, batching, and DataLoaders are all in place before the training loop begins, and the loss calculation and training functions from pre-training are reused directly.

Loss function. Cross-entropy loss is used for fine-tuning, the same function used during pre-training. Conceptually it is the negative logarithm of the predicted probability for the correct token, and it is applied to each batch of data.

Optimizer. AdamW—the Adam optimizer with weight decay—is used, implemented as torch.optim.AdamW. The gradient-descent update rule is:

$$w_{\text{new}} = w_{\text{old}} - \alpha \cdot \frac{\partial L}{\partial w}$$

where w_{new} is the updated weight, w_{old} is the previous weight, α is the learning rate, and $\partial L/\partial w$ is the partial derivative of the loss with respect to w.

Backward pass. The backward pass computes the gradient of the loss—the partial derivative with respect to every model parameter—which the optimizer then uses to update the weights.

Evaluation frequency. The evaluation-iterations setting controls after how many training iterations the loss is computed on the validation set.

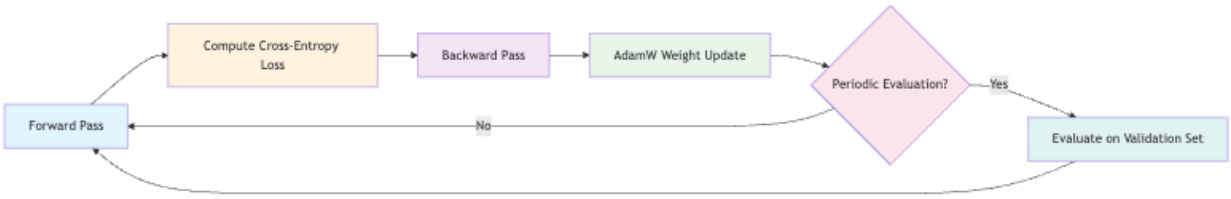


Figure 11.2: Training loop for instruction finetuning: forward pass, loss computation, backward pass, and weight updates via AdamW, with periodic validation set evaluation.

Saving and Loading the Fine-Tuned Model

Once training is complete, save the model weights:

```
torch.save(model.state_dict(), 'gpt2medium355_million_sft.pth')
```

To reload the weights later:

```
model.load_state_dict(torch.load('gpt2medium355_million_sft.pth'))
```

The suffix `sft` stands for *supervised fine-tuning*, a common convention in the field.

11.4 Stage 3: Evaluation

Saving a checkpoint is not enough—we also need to know whether the fine-tuned model actually follows instructions well. The full evaluation pipeline covers response extraction, qualitative inspection, and automated scoring.

Extracting Responses

The evaluation code iterates through all instruction-input pairs in the test set, calls the generate function on each pair, and appends the model response to the instruction_data_with_response file. For a quick sanity check during development, a shorter loop selects the first three test entries, generates output from the fine-tuned model, decodes the token IDs to text, strips the instruction prefix to isolate the response, and prints the instruction, the expected response, and the model's response side by side.

Figure 11.3: Response extraction pipeline: sequential processing from test entry through model generation to final isolated response string.

Qualitative Evaluation

Human preference comparison is an evaluation method in which a human applies their own intuition and understanding to benchmark or compare LLMs. Reading a sample of model outputs by hand is often the fastest way to spot systematic errors or surprising successes.

Automated Evaluation with Ollama and LLaMA-3

Qualitative inspection does not scale to the full test set, so automated scoring provides a complementary signal. Step 7 of the evaluation workflow involves implementing automated response evaluation using a larger pre-trained LLM: the true expected output and the model's response are both sent to a judge model, which assigns a numeric score.

Ollama is an efficient application for running large language models locally. It supports LLM inference for text generation but does not support training or fine-tuning. To pull and run LLaMA-3:

```
ollama run llama3
```

If the server is not already running, start it with:

```
ollama serve
```

The full evaluation loop iterates over every entry in the test set, generates a response from the fine-tuned model, and sends the triple (instruction, expected output, model response) to the Ollama judge for scoring. Aggregating these scores yields a single number summarizing model quality across the entire test set, making it straightforward to compare different checkpoints or hyperparameter settings.

11.5 Putting It All Together

The complete pipeline, as a numbered sequence:

1. **Download the dataset** – 1,100 instruction-input-output triples.
2. **Format with the Alpaca template** – wrap each entry using format_input.
3. **Tokenize** – convert formatted strings to token ID sequences.
4. **Pad** – equalize sequence lengths within each batch using token 50256.
5. **Create target pairs** – shift input right by one and append an end-of-text token.
6. **Mask padding in targets** – replace padding token IDs with −100.
7. **Build DataLoaders** – wrap datasets with the custom collate function.
8. **Load GPT-2 medium (355 M)** – initialize from pre-trained weights.
9. **Fine-tune** – run the training loop with cross-entropy loss and AdamW.
10. **Save checkpoint** – torch.save(model.state_dict(), ...) .
11. **Extract responses** – generate on the test set and strip the instruction prefix.
12. **Qualitative evaluation** – read a sample of outputs by hand.
13. **Automated scoring** – query Ollama/LLaMA-3 for numeric scores.

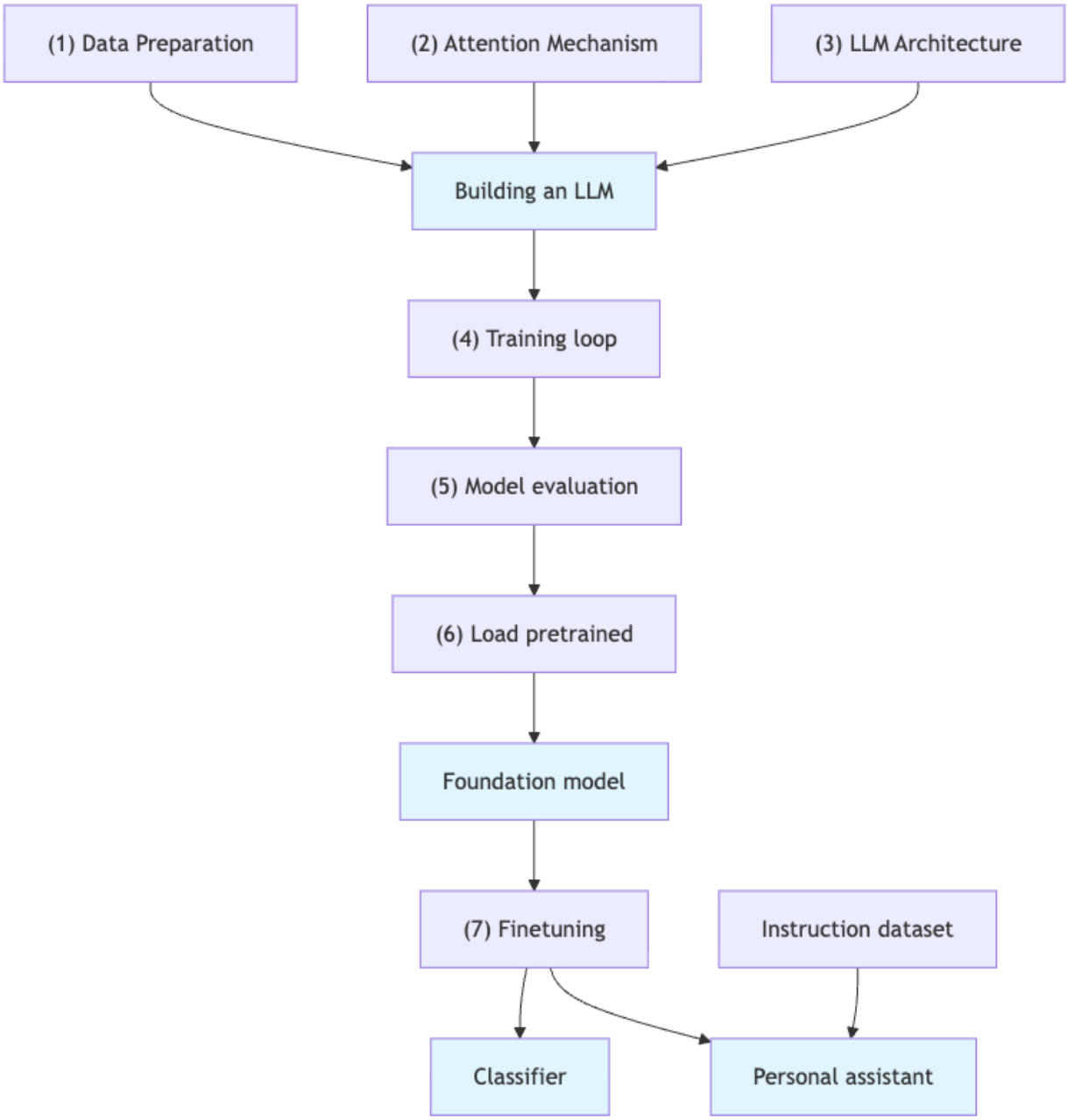


Figure 11.4: Three-stage pipeline for instruction finetuning: Stage 1 builds an LLM from data preparation, attention mechanism, and architecture; Stage 2 pretrains the model; Stage 3 finetunes with a foundation model to create a classifier and personal assistant.

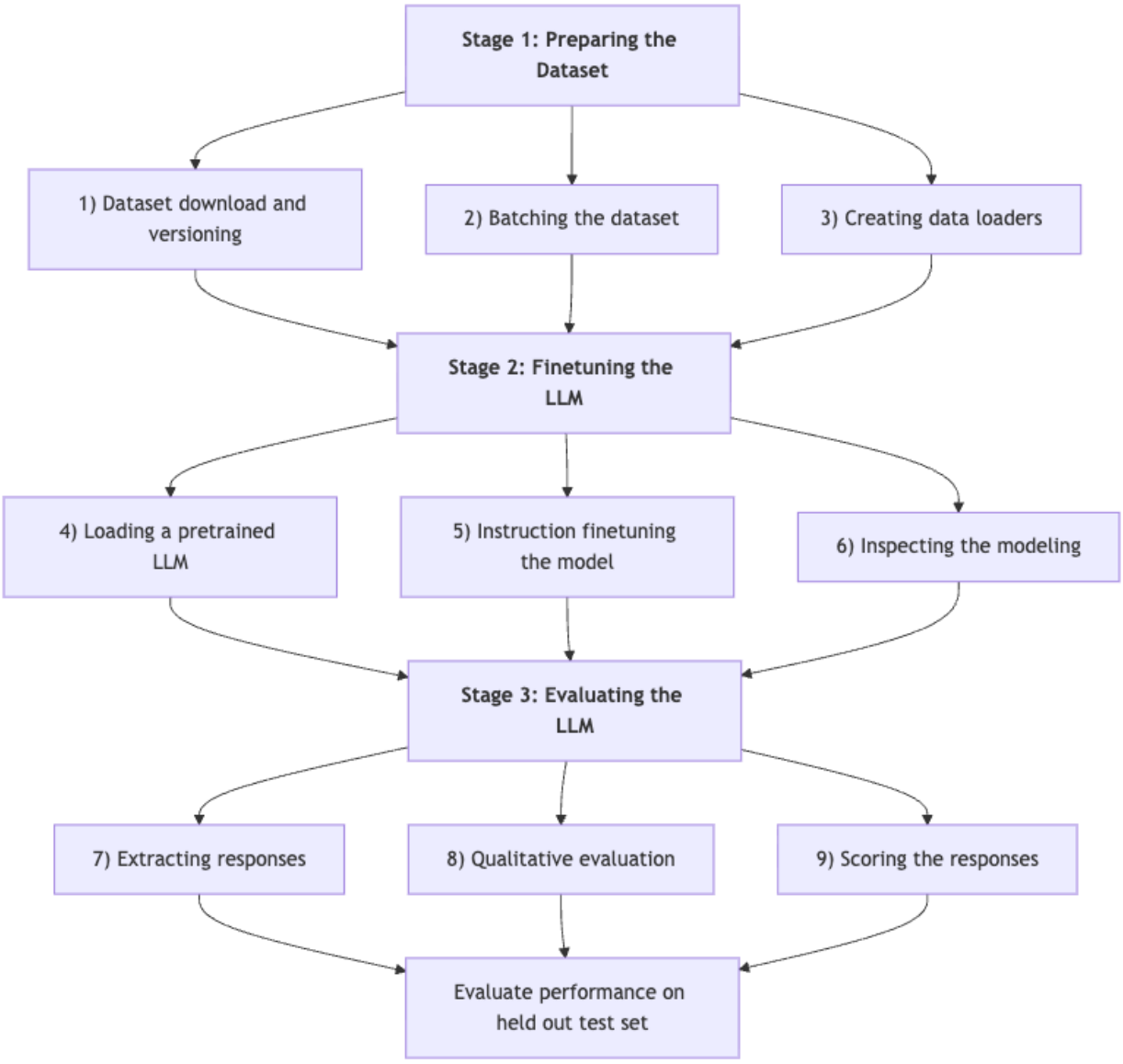


Figure 11.5: Three-stage pipeline for instruction finetuning: Stage 1 prepares the dataset (steps 1–3), Stage 2 finetunes the LLM (steps 4–6), and Stage 3 evaluates the LLM (steps 7–9).

11.6 Improving Fine-Tuning Performance

Use a larger pre-trained model. A larger model may have greater capacity to capture complex patterns and generate more accurate responses. The model_configs dictionary already includes GPT-2 large (774 M) and GPT-2 XL (1,558 M); switching requires only a one-line change to CHOOSE_MODEL.

Experiment with prompt formats. Different prompts or instruction formats can guide model responses more effectively. The Phi-3 style, which fuses instruction and input into a single user field, is one alternative worth trying.

Use a larger dataset. The Stanford Alpaca dataset contains 52,000 instruction-output pairs, compared to the 1,100 pairs used here. Training on more diverse examples generally improves generalization.

11.7 Summary

This chapter walked through the complete instruction fine-tuning pipeline, from raw data to a scored, evaluated model. The key ideas are:

- **Instruction fine-tuning** teaches a pre-trained language model to follow instructions by training on instruction-response pairs with supervised learning.
- **The Alpaca prompt format** provides a consistent text template that separates the instruction, optional input context, and expected response.
- **Data batching** requires five steps: format, tokenize, pad, shift targets, and mask padding with -100. The -100 masking ensures that padding positions do not contribute to the cross-entropy loss.
- **The training loop** reuses the same cross-entropy loss and AdamW optimizer from pre-training; the only change is the dataset.
- **Evaluation** combines qualitative human inspection with automated LLM-based scoring via Ollama, providing both interpretable examples and a scalar metric.

Quality can be pushed further by training longer, using more data, switching to a larger base model, or applying parameter-efficient methods such as LoRA.

Chapter 12

Putting It All Together: Summary and Next Steps

By this point in the series, you have done something that most practitioners never attempt: building a large language model completely from scratch. Not by calling a library function, not by wrapping an API, but by writing every component yourself and understanding why each piece exists. This final chapter consolidates that journey, revisits the key ideas that make LLMs work, and points toward the advanced topics that await you beyond this foundation.

12.1 The Three-Stage Mental Model

Before diving into the recap, it helps to anchor everything in the high-level workflow that structured the entire series. Large language model development follows three sequential stages:

1. **Stage 1 — Foundation and Architecture**: data preparation, the attention mechanism, and assembling the transformer block into a full model.
2. **Stage 2 — Pre-training**: implementing the loss function, running the backward pass, and loading pre-trained weights to accelerate training.
3. **Stage 3 — Fine-tuning**: adapting the pre-trained model to specific tasks such as classification or instruction following.

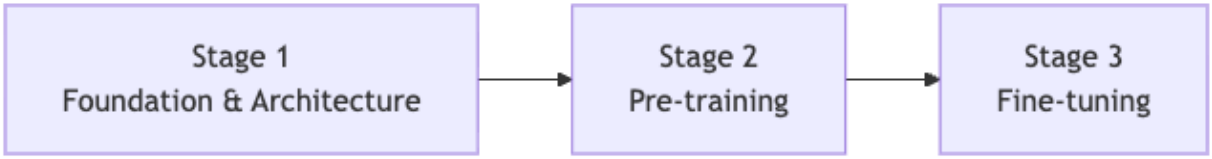


Figure 12.1: Three-stage LLM development pipeline progressing from foundation and architecture through pre-training to fine-tuning.

Each stage builds directly on the previous one: you cannot fine-tune a model you have not pre-trained, and you cannot pre-train a model whose architecture you do not understand. The series

was deliberately designed to teach the nuts and bolts of how large language models are built rather than simply running fancy applications on top of them.

12.2 Stage 1 Recap: Data, Attention, and Architecture

The Data Preprocessing Pipeline

The first thing that distinguishes LLM development from conventional machine learning is the data pipeline. In a regression or classification setting you typically have a fixed feature matrix; in an LLM the goal is to predict the next token, and that changes everything about how you prepare your data.

The pipeline proceeds through a well-defined sequence of transformations:

1. Raw text from training documents is split into **tokens**.
2. Tokens are mapped to integer **token IDs**.
3. Token IDs are projected into a higher-dimensional vector space to capture semantic meaning — these are the **token embeddings**.
4. **Positional embeddings** are added to the token embeddings because token positions matter for predicting the next token.
5. The sum of token embeddings and positional embeddings forms the **input embeddings**, which are the final output of the preprocessing pipeline.

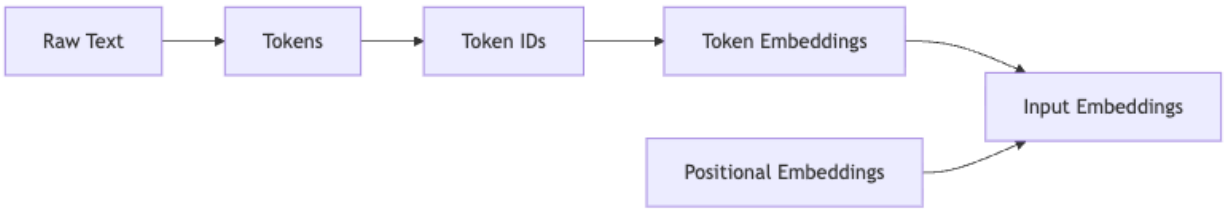


Figure 12.2: Data preprocessing pipeline transforming raw text through tokenization, embedding, and positional encoding to produce input embeddings for the model.

As a concrete reference point, GPT-2 uses an embedding dimension of 768. A higher-dimensional embedding space gives the model more room to encode nuanced semantic distinctions between tokens.

The Attention Mechanism: The Engine of LLMs

The attention mechanism is the single most important component in the entire architecture. To understand why it is necessary, consider what token embeddings alone provide: a representation of the semantic meaning of each individual token, but no information about how tokens relate to one another. When predicting the next token, you need to know how every token in the context relates to every other token.

Attention solves this by assigning an importance score to every other token when processing a given token. The goal is to convert the input embedding vectors into richer **context vectors** that encode these inter-token relationships — richer precisely because they contain information about how a token relates to all other tokens in the sentence.

The mechanics of computing attention follow a precise sequence:

1. Multiply the input embeddings by three trainable weight matrices to produce **queries**, **keys**, and **values**.
2. Compute **attention scores** by multiplying queries with the transpose of the keys.
3. **Scale** the attention scores by the square root of the key dimension.
4. Apply **dropout** after scaling.
5. Apply a **causal mask** to prevent tokens from attending to future positions — this is what makes the attention mechanism suitable for next-token prediction.
6. Pass the masked scores through a **softmax** to obtain **attention weights**.
7. Multiply the attention weights by the values to produce the **context vector matrix**.

Figure 12.3: Single attention head computation pipeline: projections of Query, Key, and Value matrices flow through scaled dot-product attention with causal masking and softmax normalization to produce a context vector.

A single attention head produces one context vector matrix, but real LLMs use **multi-head attention**, running multiple heads in parallel. Different heads can capture different types of dependencies and long-range relationships within large paragraphs, and their context vector matrices are then combined to produce the final output.

The revolution in large language models is, at its core, this workflow: converting input embedding matrices into context vector matrices through attention.

The Full LLM Architecture

With the attention mechanism understood, the full architecture assembles naturally. At the highest level, the model takes input tokens, converts them to embeddings, passes them through a stack of transformer blocks, applies a final layer normalization, and feeds the result through a linear layer that outputs **logits** for next-token prediction.

Figure 12.4: Full LLM architecture pipeline showing the sequential transformation from input tokens through embeddings, transformer blocks, normalization, and linear projection to output logits.

Each **transformer block** contains the following components in sequence:

- A normalization layer
- The masked multi-head attention module, where the Q/K/V projections operate
- A dropout layer
- Shortcut (residual) connections
- A second normalization layer
- A feedforward neural network

- A second dropout layer

The **logits tensor** output by the final linear layer is what the model uses to predict the next token.

GPT-2 stacks 12 transformer blocks sequentially; larger LLMs use more, and each block can contain 12 or 24 attention heads. These numbers give a sense of the scale involved even in a relatively compact model like GPT-2.

12.3 Stage 2 Recap: Pre-training

Loss Function and Optimization

Pre-training teaches the model to predict the next token across a massive corpus of text. The loss function used is **cross-entropy loss** between the predicted next token and the actual next token.

The trainable parameters updated during this process span the entire model:

- Token embeddings
- Positional embeddings
- Layer normalization scale and shift parameters
- Query, key, and value weight matrices in every multi-head attention module
- Feedforward network weights in every transformer block
- Final output layer weights

The vanilla gradient descent update rule is:

$$w_{i+1} = w_i - \alpha \frac{\partial L}{\partial w_i}$$

In practice, more sophisticated optimizers such as Adam or Adam with weight decay are used instead. The weight update with L2 regularization takes the form:

$$W_i^{(l)} = W_i - \lambda \frac{\partial L}{\partial W}$$

where pre-trained weights from GPT-2 can be loaded as initialization.

The Scale of Real Pre-training

Pre-training at production scale is a sobering exercise. Models like GPT-2, GPT-3, and GPT-4 are trained on huge datasets containing millions of news articles, blogs, and books, and the compute costs exceed one million dollars. This is why the series also covers loading pre-trained weights from OpenAI's GPT-2 — doing so accelerates the process enormously and makes the experiments in this course tractable. Pre-training typically involves optimizing more than 100 million or even billions of parameters via the backward pass, which is important context for appreciating why fine-tuning is so valuable: you get to start from a model that already understands language rather than learning from random noise.

12.4 Stage 3 Recap: Fine-tuning

Pre-training produces a model that can predict the next token; fine-tuning is what makes that model useful for specific tasks. The series covered two distinct flavors.

Classification Fine-tuning

Classification fine-tuning trains the model to assign inputs to discrete categories. The hands-on project was an LLM classifier that distinguishes between spam and non-spam emails — in effect, taking a model that already understands language and adding a small classification head that maps its representations to a fixed set of labels.

Instruction Fine-tuning

Instruction fine-tuning trains the model on a dataset of instructions, inputs, and outputs so that it learns to follow natural language directives. A concrete example: given the instruction "convert to passive voice" and the input "the chef cooks the meal every day," the model should produce "the meal is cooked every day by the chef". The corresponding hands-on project was a personal assistant capable of following instructions.

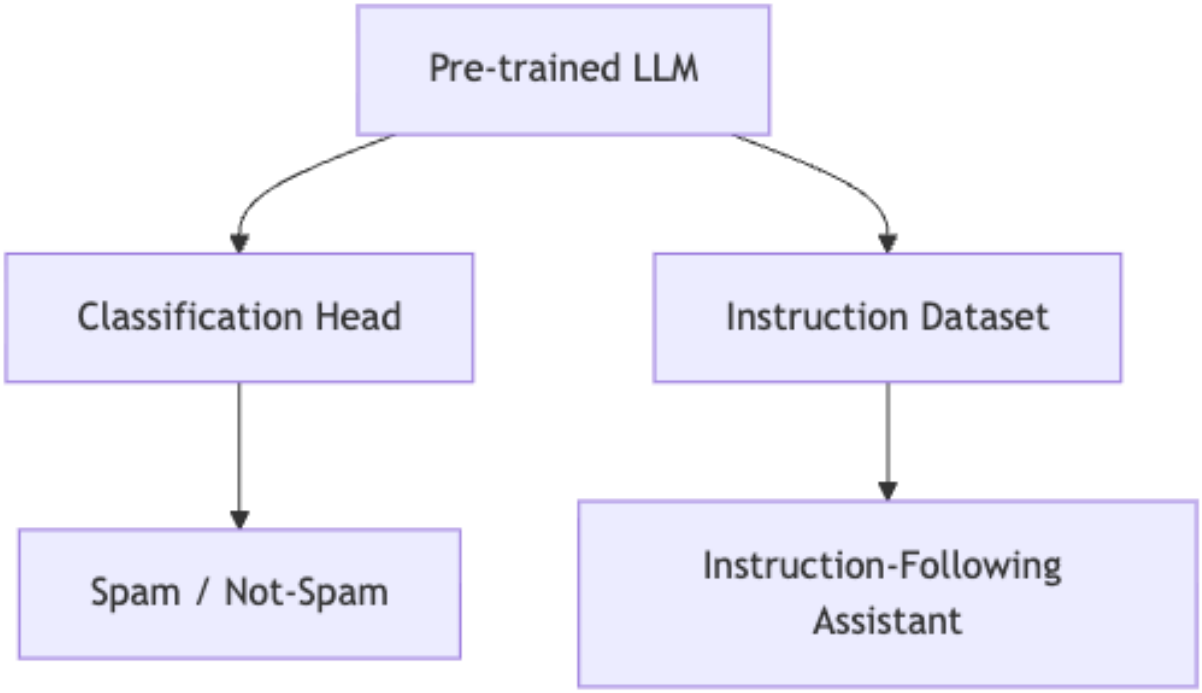


Figure 12.5: Two fine-tuning paths from a pre-trained LLM: one branch adds a classification head for spam detection, while the other uses an instruction dataset to create an instruction-following assistant.

Together, the spam classifier and the personal assistant represent the two most common fine-tuning paradigms you will encounter in practice.

12.5 Evaluating Your LLM

Building a model is only half the work; you also need to know whether it performs well. The series introduced three complementary evaluation approaches.

Benchmark Evaluation: MMLU

MMLU (Measuring Massive Multitask Language Understanding) is a standardized benchmark that uses 57 tests to evaluate LLM performance across diverse domains. It produces a single comparable number that lets you position your model relative to published results, and the breadth of 57 tests is what makes it a useful signal — a model that scores well on MMLU has demonstrated competence across a wide range of knowledge areas.

Human Evaluation

Human evaluation is the most direct method: humans compare and rate the outputs of different LLMs. It is expensive and slow, but it captures qualities — fluency, helpfulness, tone — that automated metrics often miss.

LLM-Based Evaluation

LLM-based evaluation uses a powerful large language model to assess the outputs of another LLM. In practice, the evaluating model compares the true output with the model's response and assigns a score out of 100. The series used **Ollama** to access and run **Llama 3** locally, with Llama 3 8B Instruct — a fine-tuned model with 8 billion parameters — serving as the evaluator.

LLM-based evaluation is increasingly popular because it scales better than human evaluation while capturing more nuance than simple benchmark scores. The tradeoff is that you are trusting one model's judgment to evaluate another, a circularity worth keeping in mind.

12.6 What You Have Actually Built

It is worth pausing to take stock of the concrete artifacts produced across the three stages. The series implements three main components:

1. A **next-token prediction LLM** built from scratch.
2. An **email classification fine-tuned LLM** that distinguishes spam from non-spam.
3. An **instruction fine-tuned LLM** that acts as a personal assistant.

Every code block was explained in detail, and the entire implementation was built from scratch using a custom architecture rather than imported from external sources. A single code file was developed and shared throughout the series, giving you a coherent, runnable artifact rather than a collection of disconnected snippets.

The series was inspired by the book *Build a Large Language Model* by Sebastian Raschka and was designed to teach fundamental research methodology in machine learning — not just how to use LLMs, but how to think about them.

12.7 Where to Go Next

Completing this series puts you in a strong position to engage with the research literature and with production-grade tooling.

Parameter-Efficient Fine-tuning: LoRA and QLoRA

Full fine-tuning updates every parameter in the model, which is prohibitively expensive for large models. **LoRA (Low-Rank Adaptation)** and **QLoRA (Quantized LoRA)** dramatically reduce the number of trainable parameters during fine-tuning by injecting small trainable matrices into the existing weight structure. [GAP: specific LoRA/QLoRA mechanics and citations were not covered in the supplied claims — consult the research literature for details.]

Because you now understand exactly which weight matrices exist in the transformer block and how they are updated, you are well-equipped to understand what LoRA is doing when it decomposes those matrices into low-rank factors.

Scaling to Larger Models

GPT-2, with its 12 transformer blocks and 768-dimensional embeddings, is a tractable starting point, but the same architecture scales directly to much larger models by increasing the number of blocks, the number of attention heads per block, and the embedding dimension. The principles you have learned — attention, transformer blocks, pre-training, fine-tuning — are identical at every scale.

Advanced Evaluation

The three evaluation methods covered in the series — MMLU, human evaluation, and LLM-based scoring — are a solid foundation, but the evaluation landscape is rich. Benchmarks like HellaSwag, TruthfulQA, and BIG-Bench extend MMLU's coverage. [GAP: specific details on these benchmarks were not covered in the supplied claims.]

Retrieval-Augmented Generation and Tool Use

Fine-tuning adapts a model's weights. An alternative approach is to leave the weights fixed and instead give the model access to external knowledge through retrieval or tool calls. [GAP: RAG and tool-use mechanics were not covered in the supplied claims.]

12.8 Closing Thoughts

The central insight of this entire series is captured in a single sentence: the revolution in large language models is driven by the workflow of converting input embedding matrices into context vector matrices through attention. Everything else — the transformer block, the pre-training objective, the fine-tuning strategies — is scaffolding around that core idea.

You started with raw text, learned to tokenize it, embed it, and encode positional information into it. You built the attention mechanism from first principles, understanding why queries, keys, and values exist and why causal masking is necessary. You assembled transformer blocks and stacked

them into a full GPT-style architecture. You pre-trained the model using cross-entropy loss and gradient-based optimization, then fine-tuned it for both classification and instruction following. Finally, you evaluated it using benchmarks, human judgment, and LLM-based scoring.

That is the complete picture. The field will continue to evolve — new architectures, new training recipes, new evaluation frameworks — but the foundation you have built here is durable. The nuts and bolts do not change as fast as the headlines do.

Appendix A

Environment Setup and Dependencies

Before writing a single line of model code, you need a working environment. This chapter walks through every library you will install, explains why each one is needed, and gives you concrete guidance on which hardware to expect reasonable runtimes from. Get this right once and the rest of the book runs smoothly.

A.1 Python Libraries Overview

The examples in this book rely on a small, focused set of libraries. Each one earns its place:

- **PyTorch** — the primary deep-learning framework used throughout for model definition, training, and inference.
- **tiktoken** — OpenAI's fast tokenizer library. Implementing Byte-Pair Encoding (BPE) from scratch is relatively complicated, so the tutorials use tiktoken instead. At the time of writing, tiktoken has approximately 11,000 stars and 780 forks on its repository, reflecting broad community adoption.
- **TensorFlow** — needed only for one specific task: loading the original GPT-2 weights. OpenAI originally saved those weights in TensorFlow format, while the lecture-series code uses PyTorch, so TensorFlow must be installed to bridge the two.
- **tqdm** — a lightweight progress-bar library installed to track the download progress of the GPT-2 weights.
- **pandas** — used for data handling in later fine-tuning chapters.
- **Ollama** — a separate application (not a Python package) for running pre-trained LLMs locally; covered in its own section below.

A.2 Installing the Core Libraries

Version Requirements

Two libraries have explicit minimum version requirements:

- TensorFlow 2.15
- tqdm 4.66

For PyTorch, follow the official installation matrix at pytorch.org and select the build that matches your operating system and CUDA version (if applicable).

Tokenizer Setup

Once tiktoken is installed, initializing the GPT-2 tokenizer takes two lines:

```
import tiktoken
tokenizer = tiktoken.get_encoding('gpt2')
```

This downloads a small vocabulary file on first run and caches it locally; all subsequent calls are instantaneous.

A.3 Device Selection: CPU, CUDA, and Apple MPS

PyTorch supports three device backends relevant to this book: CPU, CUDA (NVIDIA GPUs), and MPS (Apple silicon). A standard one-liner selects CUDA when available and falls back to CPU otherwise:

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

For MacBooks with Apple silicon, the device can instead be set to torch.device('mps') for faster execution. MPS offloads tensor operations to the GPU cores inside the M-series chip, giving a meaningful speedup over the CPU path without requiring an NVIDIA card.

Why Device Transfer Placement Matters

When building data-loading pipelines, *where* you move tensors to the target device is not merely a style choice — it has a real performance consequence. Performing the device transfer inside the custom collate function (outside the training loop) allows it to happen as a background process, preventing it from blocking the GPU during model training. The custom collate function therefore includes code to move input and target tensors to the appropriate device (CPU, CUDA, or MPS) at data-loading time.

A.4 Hardware Expectations and Realistic Runtimes

One of the most common frustrations when starting out is underestimating how long training takes. The table below summarizes concrete timings from the source material so you can plan accordingly.

Task	Hardware	Time
Pre-training a small GPT model (1 epoch)	MacBook Air 2020, 8 GB RAM, CPU	~2 hours
Pre-training a small GPT model (short run)	MacBook Air 2020, CPU	~8.8 minutes
Pre-training on i5/i7 or comparable MacBook	CPU	7–10 minutes
Fine-tuning GPT-2 medium	Author's PC (GPU)	~4 hours

Task	Hardware	Time
Running the full evaluation function	MacBook Air M3	~1 minute

A few points are worth noting. Training for 2 epochs on a CPU-only machine would take 4 to 5 hours and risk crashing the system — a strong reason to either use a GPU or keep experimental runs short. Apple silicon is approximately 2× faster than Apple CPU, so an M-series MacBook is a meaningful upgrade over an Intel-based one for this workbook. The examples were developed and tested on a MacBook Air 2020 with 8 GB of RAM running on CPU, so the code is deliberately written to be accessible even without dedicated hardware.

A.5 Ollama: Running Large Models Locally

Later chapters explore instruction following and evaluation using larger pre-trained models. For that, Ollama is the recommended tool.

What Ollama Is (and Is Not)

Ollama is an efficient application for running large language models on your laptop. It generates text via LLM inference but does not support training or fine-tuning: when you use Ollama, the model is already pre-trained and you are performing inference only. Keep this distinction clear — Ollama is a runtime, not a training framework.

Supported Models

Ollama supports several well-known open models out of the box, all runnable on a laptop:

- **LLAMA 3**, developed by Meta
- **Phi 3**, developed by Microsoft
- **Mistral**
- **Gemma 2**

Installation

Ollama is available for Mac OS, Linux, and Windows. The Mac OS installer is 177 MB, so the download itself is quick. The larger cost comes from pulling the model weights afterward: the LLAMA 3 model files are 4.7 GB, and downloading the 8-billion-parameter model takes approximately 15 to 20 minutes.

Running Ollama

On Mac, start the model with a single command:

```
ollama run llama3
```

On Windows, run ollama serve first, then ollama run llama3:

```
ollama serve
```

When LLAMA 3 loads successfully, right-hand-side arrows appear in the terminal indicating the model is ready for interaction. One important operational note: Ollama must remain running in the terminal for any code that depends on it — closing that terminal window will stop Ollama.

Interacting via Python

Typing prompts directly into the terminal is fine for quick tests, but the evaluation pipelines in later chapters require programmatic access. Rather than invoking ollama run in the terminal each time, you can interact with the model through a Python API. A short check confirms the session is live by verifying that ollama_running equals true.

A.6 Summary

With the libraries installed and your device configured, you are ready to start building. The key points to carry forward:

1. Install tiktoken, TensorFlow (2.15), tqdm (4.66), and PyTorch before running any notebook.
2. Use torch.device('mps') on Apple silicon for a roughly 2× speedup over CPU; use torch.device('cuda') on NVIDIA hardware.
3. Move tensors to the target device inside the collate function, not inside the training loop, to avoid blocking the GPU.
4. Expect multi-hour runtimes for fine-tuning on CPU; plan experiments accordingly.
5. Ollama handles inference for large pre-trained models locally — it does not train or fine-tune.

Appendix B

GPT-2 Model Configurations and Key Hyperparameters

Before writing a single line of model code, it pays to have a clear map of the territory: what are the exact numbers that define a GPT-2 model, and which training knobs will we be turning throughout this book? Think of this chapter as a reference you can flip back to whenever you need to double-check a dimension, a layer count, or a learning rate.

GPT-2 was released in four sizes. Each size shares the same fundamental architecture—transformer blocks stacked on top of a token embedding layer—but differs in how wide and how deep that stack is. Understanding the relationship between these sizes gives you intuition for how scaling works in practice.

The smallest member of the family is **GPT-2 small**, with 124 million parameters. It uses an embedding dimension of 768, 12 transformer layers, and 12 attention heads. Its context length—the maximum number of tokens the model can attend to at once—is 1024. This is the model we will build from scratch in the early chapters of this book, and the hyperparameter values shown throughout those chapters correspond to this configuration.

Moving up in scale, **GPT-2 medium** has 345 million parameters (sometimes cited as 355 million). It widens and deepens the architecture considerably: the embedding dimension grows to 1024, the number of layers doubles to 24, and the number of attention heads increases to 16. Its context size remains 1024 tokens. As a general pattern, as GPT-2 model complexity increases, the number of transformer blocks also increases.

GPT-2 large reaches 774 million parameters with an embedding dimension of 1280, 36 layers, and 20 attention heads. Finally, **GPT-2 XL** tops out at 1558 million parameters, using an embedding dimension of 1600, 48 layers, and 25 attention heads.

The table below summarizes the full family at a glance.

B.1 The Baseline Configuration Dictionary

Throughout this book we represent a model's configuration as a plain Python dictionary. Here is the canonical configuration for GPT-2 small (124M) that serves as our starting point:

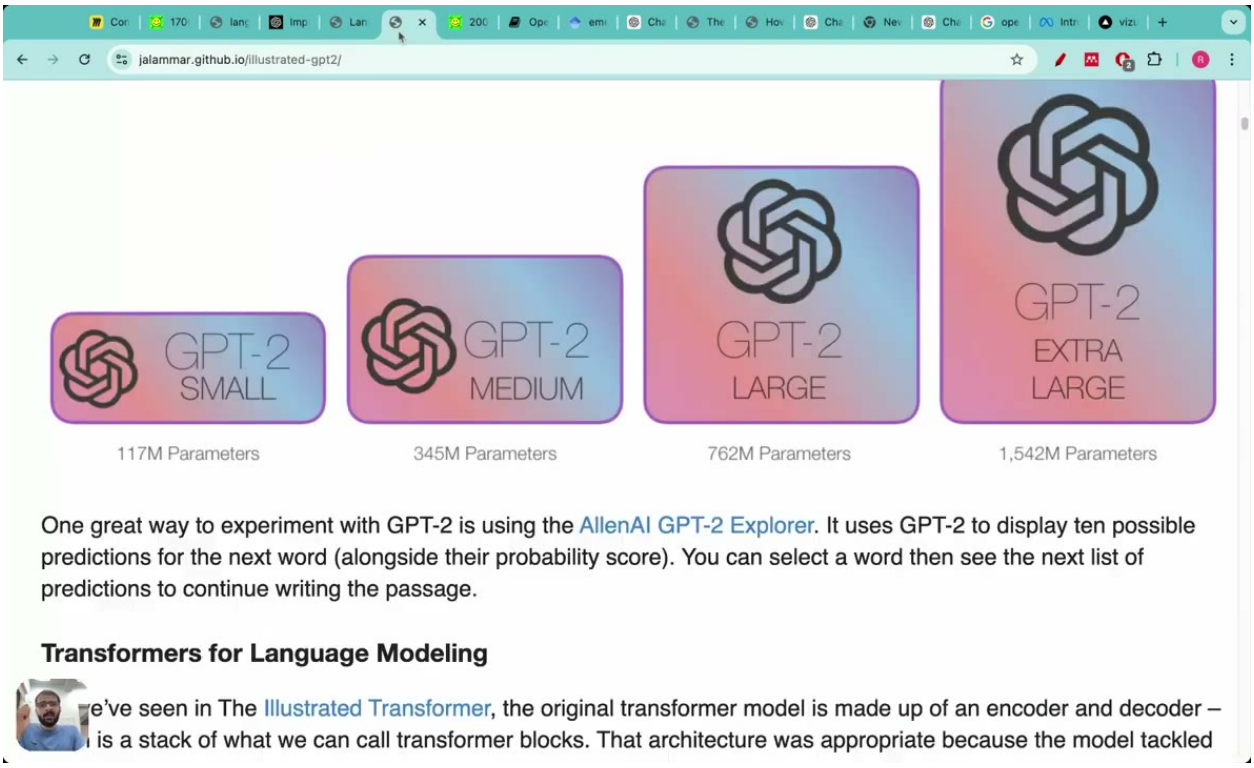


Figure B.1: GPT-2 model variants showing parameter counts: Small (117M), Medium (345M), Large (762M), and Extra Large (1,542M).

```
GPT_CONFIG_124M = {
    "vocab_size": 50257,    # Vocabulary size
    "context_length": 1024, # Context length
    "emb_dim": 768,         # Embedding dimension
    "n_heads": 12,          # Number of attention heads
    "n_layers": 12,         # Number of layers
    "drop_rate": 0.1,       # Dropout rate
    "qkv_bias": False       # Query-Key-Value bias
}
```

Every key in this dictionary corresponds to a meaningful architectural choice, each of which we walk through in turn below.

B.2 Dissecting the Configuration Keys

`vocab_size`

The vocabulary size is 50257 tokens—the number of unique tokens that the model's embedding layer must handle, with one learned vector per token. Intuitively, this is the size of the model's "alphabet," and every input token ID must fall within the range $[0,\ \text{vocab\_size} - 1]$.

`context_length`

The context length is 1024 for the full GPT-2 small model, and must be set explicitly in any GPT model configuration. It defines the maximum sequence length the model can process in a single forward pass; tokens beyond this limit simply cannot be attended to.

For the hands-on tutorials in this book, we reduce the context length to 256 tokens. This reduction is deliberate: it cuts memory and compute requirements enough that the training examples can run on a laptop. Functionally the model is identical; only the maximum sequence length changes.

`emb_dim`

The embedding dimension is 768. This is the size of the vector space into which each token ID is projected—for GPT-2 small, every token is represented as a 768-dimensional vector. Because the same dimensionality flows through every subsequent layer (attention, feed-forward, and layer normalization), this single number has an outsized influence on the model's total parameter count.

`n_heads`

n_heads is the number of attention heads present in each transformer block. For GPT-2 small this is 12. Multi-head attention splits the embedding dimension across heads, so each head operates on a $768/12 = 64$-dimensional subspace, allowing different heads to learn to attend to different kinds of relationships in the input simultaneously.

`n_layers`

n_layers is the number of transformer blocks stacked in the model. GPT-2 small uses 12. Each block contains its own multi-head self-attention sublayer and feed-forward sublayer, so doubling n_layers roughly doubles both the depth and a significant portion of the parameter count of the network.

`drop_rate`

The dropout rate is 0.1 in the standard GPT-2 small configuration. Dropout is a regularization technique; a rate of 0.1 means that during training, 10% of activations are randomly zeroed out at each dropout layer. When experimenting or debugging, it is common to set this to zero, which disables regularization and makes the forward pass fully deterministic.

`qkv_bias`

The query, key, and value weight matrices are initialized without bias terms—qkv_bias is False. This deliberate design decision matches the original GPT-2 implementation: omitting these biases slightly reduces the parameter count and has been found to work well in practice.

B.3 Variations Across Model Sizes

Not all GPT-2 configurations use exactly the same defaults. When loading pretrained GPT-2 weights (as opposed to training from scratch), the base configuration sets the dropout rate to 0.0 and qkv_bias to True. These differences matter when you are trying to match a pretrained checkpoint exactly. For instruction fine-tuning experiments later in the book, GPT-2 medium

(355M) is preferred over the 124M model because the smaller model does not perform well on instruction-following tasks.

B.4 Training Hyperparameters

Beyond the model architecture, several training hyperparameters appear repeatedly in the code examples throughout this book. They are collected here for easy reference.

Optimizer settings. We use the AdamW optimizer with a learning rate of 0.00005 (5e-5) and a weight decay of 0.1. In some earlier experiments the Adam optimizer is used with a learning rate of 5e−4 and the same weight decay of 0.1.

Training duration. For the pretraining demonstrations, the number of epochs is set to 1. This is intentionally short—the goal is to verify that the training loop works correctly, not to produce a fully converged model.

Evaluation cadence. Results are printed after every 5 batches, and the evaluation loss is calculated over 5 batches at each evaluation step. This provides a reasonable signal about training progress without spending too much time on evaluation.

Data loader settings. The training data loader is created with a batch size of 2, a maximum sequence length of 256 (matching the reduced context length), a stride of 256, drop_last=True, shuffle=True, and num_workers=0. The validation data loader uses the same batch size and sequence length but without shuffling.

Text generation settings. When generating text to inspect model quality, a temperature of 1.5 is used, and top-K sampling with $K = 50$ restricts token selection to the 50 most likely candidates at each step.

B.5 Summary

The table below consolidates the key hyperparameters used in this book's tutorial setting alongside the full GPT-2 small specification for comparison.

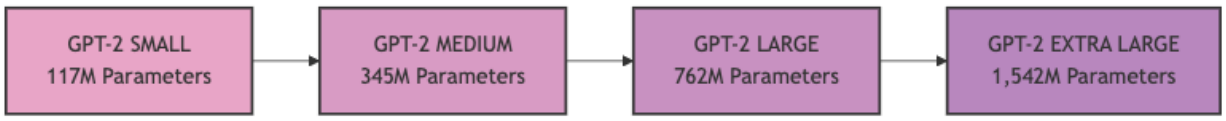


Figure B.2: GPT-2 model variants showing increasing parameter counts from Small (117M) to Extra Large (1,542M).

With these numbers firmly in hand, we are ready to start building the model itself. Every architectural choice made in the coming chapters traces back to one of the values defined here.

Appendix C

Datasets Used in This Series

Every machine learning project lives or dies by its data, and building a language model from scratch is no exception. This chapter catalogs every dataset that appears throughout this series—what each one contains, where it comes from, how large it is, and what role it plays. Having this reference in one place will save you from hunting through later chapters when you need to recall why a particular dataset was chosen or how to obtain it.

We cover four distinct categories: a small literary text used for pre-training demonstrations, a spam-classification corpus used for classification fine-tuning, a compact instruction-following dataset used for instruction fine-tuning, and the massive corpora used to train GPT-3 in the real world. The first three are datasets you will actually work with in code; the last provides context for understanding the scale gap between educational examples and production systems.

C.1 The Verdict: Pre-Training Dataset

The primary dataset for pre-training demonstrations throughout this series is *The Verdict*, a short story by Edith Wharton. Published in 1908 and now in the public domain, it is freely available to download and is included directly in the companion repository for this series.

The choice of such a small text is deliberate: a short work is used specifically to ensure fast execution on laptops, so you can run every pre-training experiment on consumer hardware in a reasonable amount of time.

In concrete terms, the text file contains **20,479 total characters**. When tokenized using byte pair encoding (specifically the tiktoken encoder), the story produces **5,145 tokens**. That is an extremely short corpus by the standards of language model training—a production model would be trained on hundreds of billions of tokens—but for learning how the data pipeline, model architecture, and training loop all fit together, it is entirely suitable.

[Diagram: Comparison of model sizes: The Verdict (5,145 tokens) versus GPT-3 (300 billion tokens) on a logarithmic scale.]

C.2 SMS Spam Collection: Classification Fine-Tuning Dataset

When the series moves from pre-training to fine-tuning for classification, a different dataset takes center stage: the **SMS Spam Collection**. This corpus demonstrates how a pre-trained language model can be adapted to perform binary text classification—distinguishing spam messages from legitimate ones (commonly called "ham").

The dataset contains **5,572 messages** in total, of which 425 spam messages were manually extracted from the Grumbletext website. It is distributed as a tab-separated values file named SMSSpamCollection.tsv, which can be loaded directly into a pandas DataFrame for manipulation.

One practical challenge with spam datasets is class imbalance: legitimate messages typically outnumber spam by a wide margin. To address this, the dataset is balanced before training. After balancing, each class contains **747 instances**, yielding 1,494 examples in total for the classification experiments.

C.3 Instruction Fine-Tuning Dataset

The third dataset appears in the instruction fine-tuning portion of the series, where the goal is to teach a model to follow natural-language instructions rather than simply continue text. The dataset used for these demonstrations consists of **1,100 instruction–input–output pairs**.

Each example has three fields: an instruction describing the task, an optional input providing context, and an output giving the expected response. This format is the backbone of instruction fine-tuning and mirrors the structure used by much larger, publicly available datasets.

For comparison, the **Stanford Alpaca** repository contains **52,000 instruction–output pairs**—roughly **50 times larger** than the 1,100-pair dataset used here. The smaller dataset is chosen for the same reason *The Verdict* is used for pre-training: it keeps training times manageable on a laptop while still illustrating every step of the fine-tuning pipeline.

[Diagram: Comparison of dataset sizes: this series uses 1,100 instruction-following pairs versus Stanford Alpaca's 52,000 pairs.]

C.4 GPT-3 Pre-Training Corpora: Real-World Scale

To ground your intuition about what production pre-training actually looks like, it is worth examining the data that went into GPT-3. In total, GPT-3 was trained on **300 billion tokens**, assembled from several sources and weighted differently during training.

Common Crawl

The dominant source is **Common Crawl**, a free and open repository of web crawl data that has maintained over **250 million pages** spanning **17 years** since 2007. GPT-3 draws **410 billion tokens** from Common Crawl, constituting **60%** of the entire training dataset—making it the backbone of the training run.

WebText2

The second major source is **WebText2**, an enhanced version of the original WebText corpus covering Reddit submissions from 2005 to 2020. GPT-3 uses **19 billion words** from WebText2, accounting for **22%** of the training data.

Books and Wikipedia

The remaining **18–19%** of GPT-3's training data comes from books and Wikipedia. Within the books component, the Books1 corpus alone contributes **12 billion tokens** at an **8% weight**.

The table below summarizes the full composition of GPT-3's training data:

Source	Tokens	Weight
Common Crawl	410 billion	60%
WebText2	19 billion words	22%
Books1	12 billion	8%
Books + Wikipedia (remainder)	—	~10%
Total	**300 billion tokens**	**100%**

It is worth noting that the weighting scheme matters as much as raw token counts. A source can be sampled more or less frequently than its size alone would suggest, allowing the training process to emphasize higher-quality text—such as curated books or filtered web pages—relative to noisier sources.

C.5 Downloading and Licensing

The Verdict is in the public domain and is included directly in the series repository. The SMS Spam Collection is a well-known benchmark dataset available for research use, and the 1,100-pair instruction dataset is provided alongside the series materials. Common Crawl is a free and open repository, though working with it at full scale requires storage and compute resources well beyond a typical laptop setup.

For the purposes of this series, you will only need *The Verdict* and the SMS Spam Collection. Everything else is either provided in the repository or serves as reference material describing production-scale systems rather than something you will run locally.

C.6 Summary

Dataset	Size	Purpose
The Verdict (Edith Wharton, 1908)	20,479 chars / 5,145 tokens	Pre-training demonstrations

Dataset	Size	Purpose
SMS Spam Collection	5,572 messages (balanced: 747 per class)	Classification fine-tuning
Instruction dataset	1,100 pairs	Instruction fine-tuning
Stanford Alpaca	52,000 pairs	Reference / comparison
GPT-3 corpora	300 billion tokens total	Production-scale context

With these datasets cataloged—and a clear sense of why each was chosen—you are ready to move into the chapters that build the data-loading and tokenization pipelines that feed them into a model.